

GCS-Saker 서버 기술 입문 가이드

WebRTC부터 OCP까지, 코딩 초보자를 위한 250페이지 학습 자료

GCS-Saker

Server Technology Beginner Guide

실시간 관제 서버를 그림과 원칙으로 배우는 입문서

Version 0.1 / 250 pages / Generated for A4AI GCS-Saker

읽는 방법

처음에는 기술 이름을 외우기보다, 각 기술이 어느 계층에 있고 어떤 문제를
풀이는지 그림으로 따라가면 된다.

이 문서가 다루는 범위

서버에 실제 적용된 기술과 앞으로 유지해야 할 개발 원칙

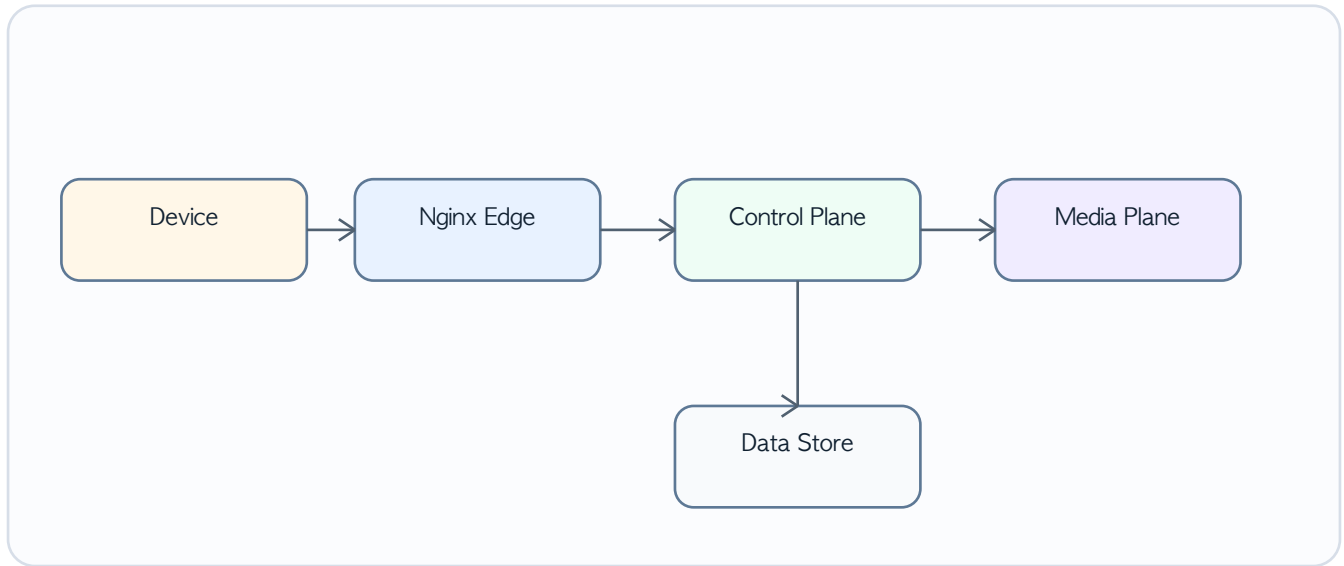
- WebRTC, WHIP/WHEP, STUN/TURN, MediaMTX, HLS fallback 같은 실시간 스트리밍 기술.
- Nginx, Docker Compose, Spring/Kotlin, Go, Python, Redis, MySQL, MQTT 같은 서버 기술.
- JWT, HttpOnly cookie, CSRF, CORS, CSP, HTTPS 같은 보안 기술.
- SOLID, OCP, DTO/VO/domain, 불변 객체, 일급 컬렉션, 디자인 패턴 같은 코드 설계 원칙.
- unit, integration, smoke, runtime validation, coverage 같은 테스트 체계.

이 문서가 의도적으로 하지 않는 것

- 운영 비밀번호, token, TURN credential, 서버 private 상세 값은 적지 않는다.
- 특정 secret 값을 예제로 넣지 않는다.
- 기술을 과장하지 않고 현재 구조와 후속 과제를 구분한다.

전체 시스템 한 장 요약

외부 입구, control plane, media plane, data plane



네 개의 큰 영역

- Entry: Nginx가 HTTPS 단일 입구를 담당한다.
- Control Plane: 인증, 권한, 스트림 목록, 운영 상태를 판단한다.
- Media Plane: 영상과 음성 packet을 빠르게 전달한다.
- Data Plane: MySQL, Redis, MQTT, PostGIS 후보가 데이터를 저장하거나 전달한다.

목차 1

1장부터 13장

- 01. GCS-Saker 전체 구조 - 관제 시스템의 큰 지도
- 02. HTTP, HTTPS, Nginx Edge - 외부 입구를 하나로 묶는 법
- 03. Docker Compose와 네트워크 - 여러 서비스를 한 번에 운영하기
- 04. React와 TypeScript Dashboard - 관제사가 실제로 보는 화면
- 05. WebRTC 기본 - 브라우저 실시간 미디어 연결
- 06. STUN, TURN, NAT - 공유기 뒤 장비를 연결하는 법
- 07. MediaMTX, WHIP, WHEP, HLS - 미디어 서버와 표준 입출력
- 08. 초저지연 스트리밍 - 빠르게 보이게 만드는 선택들
- 09. 인증, 인가, JWT 보안 - 누가 무엇을 볼 수 있는가
- 10. Spring Boot와 Kotlin Auth-Policy - 엄격한 정책 서버
- 11. Go Media-Control - 빠른 스트림 제어 API
- 12. Python FastAPI Legacy Bridge - 기존 기능을 안전하게 이어가기
- 13. MySQL과 정형 데이터 - 오래 보관할 장부

목차 2

14장부터 부록

- 14. Redis와 캐시/세션 - 빠른 메모리 저장소
- 15. MQTT와 이벤트 메시징 - 장비 이벤트를 가볍게 주고받기
- 16. 지도, GPS, 시간 동기화 - 상황을 같은 시각과 위치에 맞추기
- 17. TDD와 테스트 전략 - 실행 가능한 상태를 지키는 안전망
- 18. 운영 관측과 장애 대응 - 문제가 났을 때 빨리 알아차리기
- 19. SOLID와 OCP - 오래 버티는 코드의 원칙
- 20. 객체지향과 디자인 패턴 - 복잡한 코드를 이름 있는 구조로 정리하기
- 21. DTO, VO, API Contract - 외부 약속과 내부 규칙 분리
- 22. 동시성과 멀티스레드 - 동시에 일어나면 안 되는 일을 막기
- 23. 폐쇄망 배포 - 인터넷 없이도 돌아가는 시스템
- 24. 성능 튜닝 - 빠르고 덜 낭비하게 만들기
- 25. 운영 Runbook과 릴리스 - 개발 결과를 안전하게 서버에 올리기

부록

- 용어 사전 1
- 용어 사전 2
- 용어 사전 3
- 운영 명령 모음
- WebRTC 장애 분류
- 오디오 지연 분류
- DB 튜닝 요약
- Redis 튜닝 요약
- 프론트 렌더링 점검
- 보안 체크리스트
- 폐쇄망 체크리스트
- 테스트 체크리스트
- PR 작성 템플릿
- Issue 작성 템플릿
- 서버 배포 템플릿
- 장애 보고 템플릿
- 초보자 4주 학습 계획
- 코드 리뷰 관점
- 성능 리뷰 관점
- 마지막 한 장

01. GCS-Saker 전체 구조 - 개념 지도

관제 시스템의 큰 지도

이 페이지의 핵심

처음에는 서비스가 많아 복잡해 보이지만, 모든 구성요소는 현장 데이터를 안전하고 빠르게 대시보드에 전달하기 위해 존재한다.

왜 필요한가

처음에는 서비스가 많아 복잡해 보이지만, 모든 구성요소는 현장 데이터를 안전하고 빠르게 대시보드에 전달하기 위해 존재한다.

서버에서 맡는 역할

Nginx, dashboard, auth-policy, media-control, backend, MediaMTX, coturn, Redis, MySQL이 각자 책임을 나눠 갖는다.

초보자 비유

건물에 비유하면 Nginx는 정문, 인증 서버는 출입증 검사대, MediaMTX는 영상 중계실, Redis와 MySQL은 기록실이다.

기억할 문장

- GCS-Saker 전체 구조는 '관제 시스템의 큰 지도'를 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

01. GCS-Saker 전체 구조 - 서버 적용 위치

관제 시스템의 큰 지도

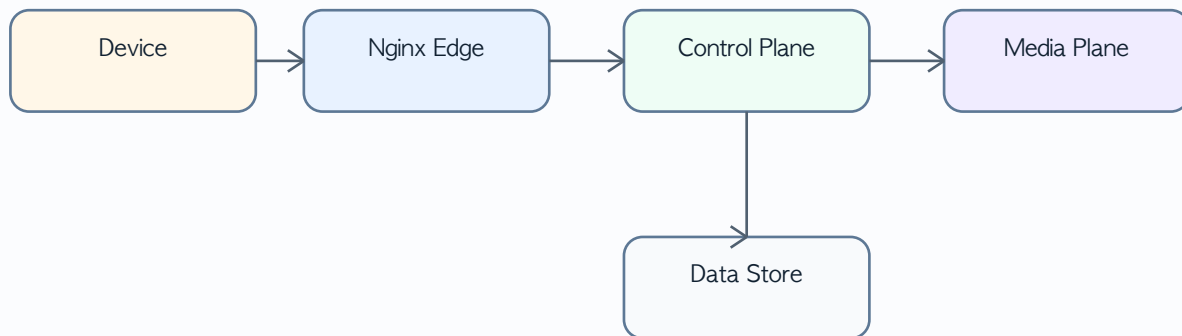


그림 해석: 관제 시스템의 큰 지도 - 외부 HTTPS 응답, healthz/readyz, first-frame latency, container health를 함께 본다.

서버 적용 위치

Nginx, dashboard, auth-policy, media-control, backend, MediaMTX, coturn, Redis, MySQL이 각자 책임을 나눠 갖는다.

연결해서 볼 것

GCS-Saker 전체 구조는 단독 기술이 아니라 Nginx, Docker Compose, 인증, 미디어 경로, 테스트와 함께 동작한다.

운영 지표

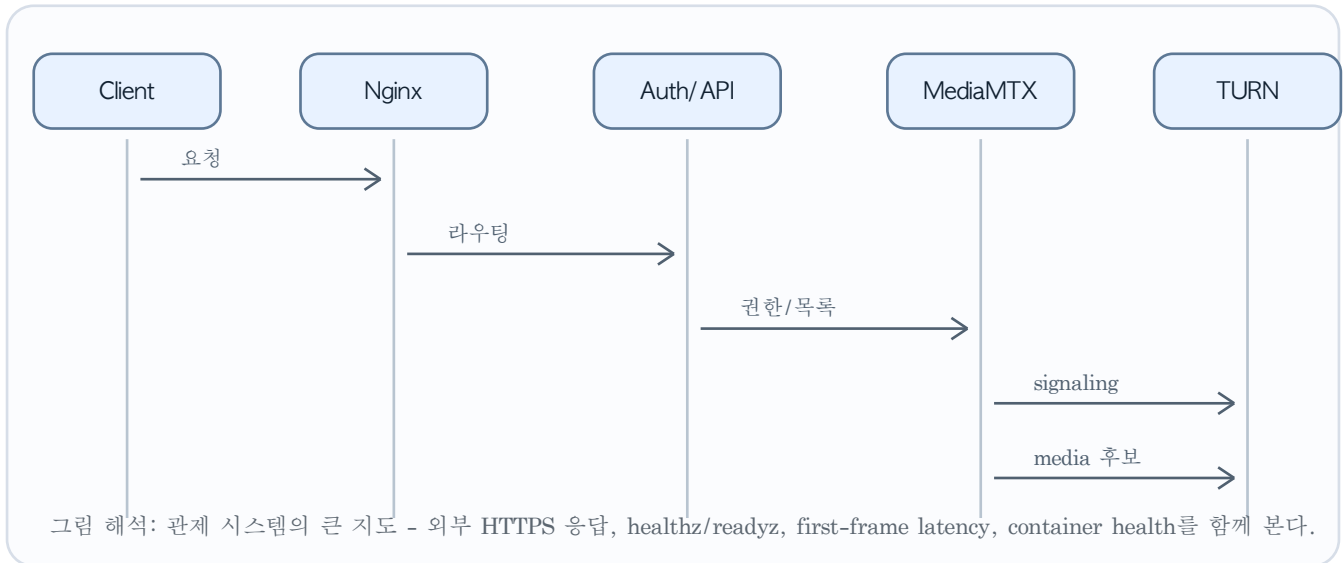
외부 HTTPS 응답, healthz/readyz, first-frame latency, container health를 함께 본다.

기억할 문장

- GCS-Saker 전체 구조는 '관제 시스템의 큰 지도'를 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

01. GCS-Saker 전체 구조 - 요청/데이터 흐름

관제 시스템의 큰 지도



흐름 읽기

요청은 항상 입구, 정책 판단, 실제 처리, 응답의 순서로 본다. GCS-Saker 전체 구조에서도 같은 순서로 로그를 따라가면 원인 파악이 빨라진다.

확인할 데이터

외부 HTTPS 응답, healthz/readyz, first-frame latency, container health를 함께 본다.

다음 단계

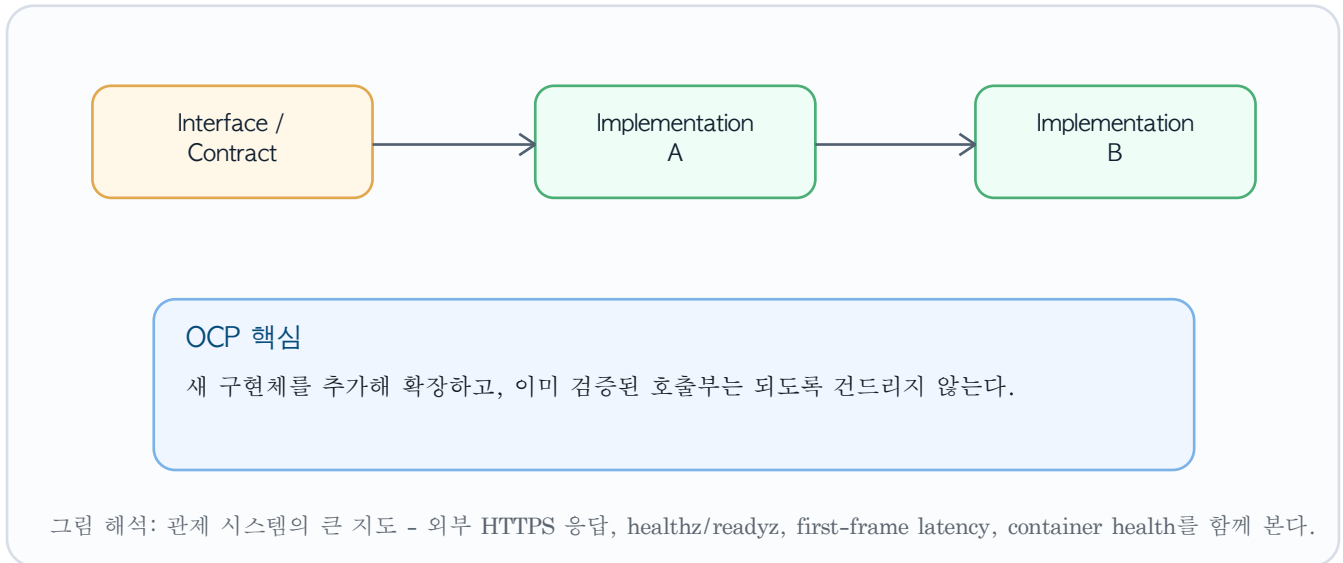
먼저 Nginx route와 Docker Compose 서비스 이름을 외우지 말고 그림으로 연결해 본다.

기억할 문장

- GCS-Saker 전체 구조는 '관제 시스템의 큰 지도'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

01. GCS-Saker 전체 구조 - 코드 작성 원칙

관제 시스템의 큰 지도



코드 원칙

큰 함수를 만들기보다 계층과 책임을 먼저 나눈다. 화면, 정책, 미디어, 저장소가 서로의 일을 침범하지 않게 한다.

설계 연결

SRP는 책임을 나누고, OCP는 새 구현체를 붙이기 쉽게 만들며, DIP는 구체 구현 대신 interface를 바라보게 만든다.

작은 예시

문자열 endpoint를 컨트롤러에 직접 쓰지 말고 RouteContract 또는 ApiRoutes 같은 상수 객체에 모은다.

기억할 문장

- GCS-Saker 전체 구조는 '관제 시스템의 큰 지도'를 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

01. GCS-Saker 전체 구조 - 테스트와 검증

관제 시스템의 큰 지도

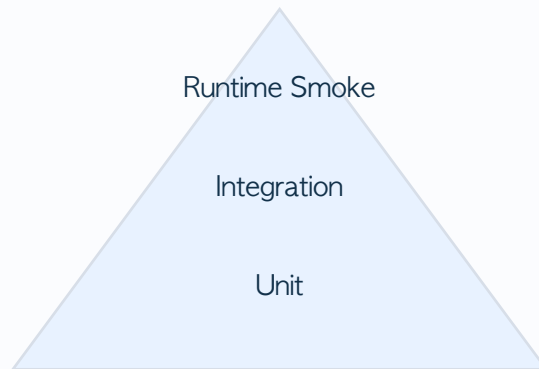


그림 해석: 관제 시스템의 큰 지도 - 외부 HTTPS 응답, healthz/readyz, first-frame latency, container health를 함께 본다.

테스트 원칙

서비스별 health, API contract, runtime smoke를 분리해 어디가 깨졌는지 빠르게 찾는다.

TDD 관점

먼저 실패하는 계약 테스트를 만들고, 그 계약을 만족하는 가장 작은 구현을 작성한 뒤, 실제 runtime smoke로 연결을 확인한다.

통과 기준

외부 HTTPS 응답, healthz/readyz, first-frame latency, container health를 함께 본다.가 기대 범위 안에 있고, 실패 시 원인이 로그와 테스트 이름으로 드러나야 한다.

기억할 문장

- GCS-Saker 전체 구조는 '관제 시스템의 큰 지도'를 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

01. GCS-Saker 전체 구조 - 운영 체크

관제 시스템의 큰 지도

이 페이지의 핵심

처음에는 서비스가 많아 복잡해 보이지만, 모든 구성요소는 현장 데이터를 안전하고 빠르게 대시보드에 전달하기 위해 존재한다.

운영 원칙

운영자는 한 화면에서 API, signaling, media, DB, cache 상태를 구분해서 봐야 한다.

장애 대응

status code, container health, application log, network path, secret/config 누락을 순서대로 확인한다.

기록할 것

명령어, 실행 시각, 결과, 실패 원인, 수정 파일, 남은 리스크를 보고서에 남긴다.

기억할 문장

- GCS-Saker 전체 구조는 '관제 시스템의 큰 지도'를 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

01. GCS-Saker 전체 구조 - 자주 나는 실수

관제 시스템의 큰 지도

이 페이지의 핵심

처음에는 서비스가 많아 복잡해 보이지만, 모든 구성요소는 현장 데이터를 안전하고 빠르게 대시보드에 전달하기 위해 존재한다.

자주 나는 실수

모든 요청을 하나의 백엔드로 몰아넣으면 실시간 미디어 지연과 장애 범위가 커진다.

왜 위험한가

초반에는 빠르게 보이지만, 운영 중 장애 원인을 숨기거나 변경 범위를 키워 결국 더 오래 걸리게 만든다.

예방 습관

계약값 중앙화, DTO 분리, 테스트 추가, 로그/지표 확인을 PR 기준으로 둔다.

기억할 문장

- GCS-Saker 전체 구조는 '관제 시스템의 큰 지도'를 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

01. GCS-Saker 전체 구조 - 초보자 실습

관제 시스템의 큰 지도

이 페이지의 핵심

처음에는 서비스가 많아 복잡해 보이지만, 모든 구성요소는 현장 데이터를 안전하고 빠르게 대시보드에 전달하기 위해 존재한다.

따라 해보기

먼저 Nginx route와 Docker Compose 서비스 이름을 외우지 말고 그림으로 연결해 본다.

관찰하기

작업 전후로 외부 HTTPS 응답, healthz/readyz, first-frame latency, container health를 함께 본다.를 기록한다. 숫자를 모르면 개선인지 착각인지 구분할 수 없다.

질문 만들기

'GCS-Saker 전체 구조이 깨지면 사용자는 무엇을 보게 되는가?'를 한 문장으로 답해 본다.

기억할 문장

- GCS-Saker 전체 구조는 '관제 시스템의 큰 지도'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

01. GCS-Saker 전체 구조 - 복습 체크리스트

관제 시스템의 큰 지도

이 페이지의 핵심

처음에는 서비스가 많아 복잡해 보이지만, 모든 구성요소는 현장 데이터를 안전하고 빠르게 대시보드에 전달하기 위해 존재한다.

복습 질문

GCS-Saker 전체 구조는 control plane, media plane, data plane, ops plane
중 어디에 가까운가?

체크리스트

경로가 상수화되어 있는가? DTO가 분리되어 있는가? 실패 테스트가 있는가? 운영 지표가 있는가? secret이 노출되지 않았는가?

다음 학습

관련 코드 파일 하나와 테스트 파일 하나를 함께 열어 구현과 검증이 어떻게 연결되는지 읽는다.

기억할 문장

- GCS-Saker 전체 구조는 '관제 시스템의 큰 지도'를 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

02. HTTP, HTTPS, Nginx Edge - 개념 지도

외부 입구를 하나로 묶는 법

이 페이지의 핵심

브라우저와 서버는 HTTP로 대화하지만 운영에서는 암호화와 경로 분리가 필요하다.

왜 필요한가

브라우저와 서버는 HTTP로 대화하지만 운영에서는 암호화와 경로 분리가 필요하다.

서버에서 맡는 역할

Nginx는 443 포트 하나로 들어온 요청을 내부의 dashboard, auth-policy, media-control, MediaMTX로 보낸다.

초보자 비유

백화점 안내 데스크가 층과 매장을 안내하듯, Nginx는 URL path를 보고 내부 서비스를 찾아준다.

기억할 문장

- HTTP, HTTPS, Nginx Edge은 '외부 입구를 하나로 묶는 법'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

02. HTTP, HTTPS, Nginx Edge - 서버 적용 위치

외부 입구를 하나로 묶는 법

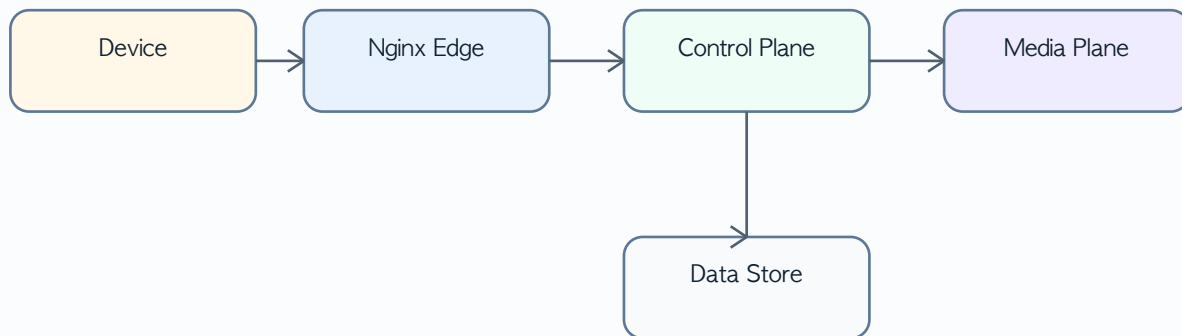


그림 해석: 외부 입구를 하나로 묶는 법 - HTTP status, TLS handshake, proxy timeout, upstream health를 본다.

서버 적용 위치

Nginx는 443 포트 하나로 들어온 요청을 내부의 dashboard, auth-policy, media-control, MediaMTX로 보낸다.

연결해서 볼 것

HTTP, HTTPS, Nginx Edge은 단독 기술이 아니라 Nginx, Docker Compose, 인증, 미디어 경로, 테스트와 함께 동작한다.

운영 지표

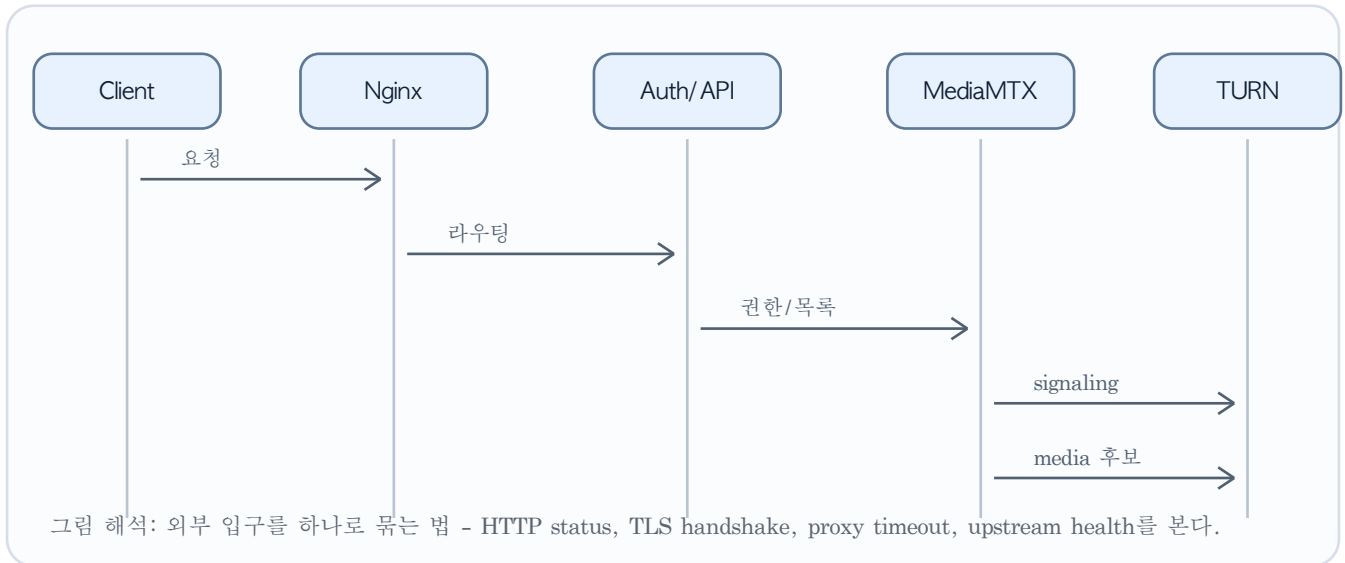
HTTP status, TLS handshake, proxy timeout, upstream health를 본다.

기억할 문장

- HTTP, HTTPS, Nginx Edge은 '외부 입구를 하나로 묶는 법'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

02. HTTP, HTTPS, Nginx Edge - 요청/데이터 흐름

외부 입구를 하나로 묶는 법



흐름 읽기

요청은 항상 입구, 정책 판단, 실제 처리, 응답의 순서로 본다. HTTP, HTTPS, Nginx Edge에서도 같은 순서로 로그를 따라가면 원인 파악이 빨라진다.

확인할 데이터

HTTP status, TLS handshake, proxy timeout, upstream health를 본다.

다음 단계

curl로 각 경로의 status code를 확인하면서 reverse proxy 감각을 익힌다.

기억할 문장

- HTTP, HTTPS, Nginx Edge은 '외부 입구를 하나로 묶는 법'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

02. HTTP, HTTPS, Nginx Edge - 코드 작성 원칙

외부 입구를 하나로 묶는 법



OCP 핵심

새 구현체를 추가해 확장하고, 이미 검증된 호출부는 되도록 건드리지 않는다.

그림 해석: 외부 입구를 하나로 묶는 법 - HTTP status, TLS handshake, proxy timeout, upstream health를 본다.

코드 원칙

서비스 코드에 외부 주소를 홀쭉리지 말고 route contract와 config로 관리한다.

설계 연결

SRP는 책임을 나누고, OCP는 새 구현체를 붙이기 쉽게 만들며, DIP는 구체 구현 대신 interface를 바라보게 만든다.

작은 예시

문자열 endpoint를 컨트롤러에 직접 쓰지 말고 RouteContract 또는 ApiRoutes 같은 상수 객체에 모은다.

기억할 문장

- HTTP, HTTPS, Nginx Edge은 '외부 입구를 하나로 묶는 법'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

02. HTTP, HTTPS, Nginx Edge - 테스트와 검증

외부 입구를 하나로 묶는 법

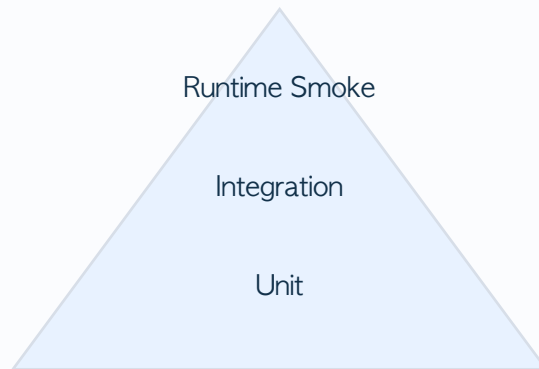


그림 해석: 외부 입구를 하나로 묶는 법 - HTTP status, TLS handshake, proxy timeout, upstream health를 본다.

테스트 원칙

Nginx contract test는 /, /auth-policy, /media-control, /webrtc, /hls 경로가 유지되는지 확인한다.

TDD 관점

먼저 실패하는 계약 테스트를 만들고, 그 계약을 만족하는 가장 작은 구현을 작성한 뒤, 실제 runtime smoke로 연결을 확인한다.

통과 기준

HTTP status, TLS handshake, proxy timeout, upstream health를 본다.가
기대 범위 안에 있고, 실패 시 원인이 로그와 테스트 이름으로 드러나야 한다.

기억할 문장

- HTTP, HTTPS, Nginx Edge은 '외부 입구를 하나로 묶는 법'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

02. HTTP, HTTPS, Nginx Edge - 운영 체크

외부 입구를 하나로 묶는 법

이 페이지의 핵심

브라우저와 서버는 HTTP로 대화하지만 운영에서는 암호화와 경로 분리가 필요하다.

운영 원칙

TLS 인증서, CSP, HSTS, X-Frame-Options 같은 보안 헤더를 edge에서 일관되게 관리한다.

장애 대응

status code, container health, application log, network path,
secret/config 누락을 순서대로 확인한다.

기록할 것

명령어, 실행 시각, 결과, 실패 원인, 수정 파일, 남은 리스크를 보고서에 남긴다.

기억할 문장

- HTTP, HTTPS, Nginx Edge은 '외부 입구를 하나로 묶는 법'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

02. HTTP, HTTPS, Nginx Edge - 자주 나는 실수

외부 입구를 하나로 묶는 법

이 페이지의 핵심

브라우저와 서버는 HTTP로 대화하지만 운영에서는 암호화와 경로 분리가 필요하다.

자주 나는 실수

3000, 8001, 8888 같은 내부 포트를 직접 공개하면 공격 표면이 넓어진다.

왜 위험한가

초반에는 빠르게 보이지만, 운영 중 장애 원인을 숨기거나 변경 범위를 키워 결국 더 오래 걸리게 만든다.

예방 습관

계약값 중앙화, DTO 분리, 테스트 추가, 로그/지표 확인을 PR 기준으로 둔다.

기억할 문장

- HTTP, HTTPS, Nginx Edge은 '외부 입구를 하나로 묶는 법'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

02. HTTP, HTTPS, Nginx Edge - 초보자 실습

외부 입구를 하나로 묶는 법

이 페이지의 핵심

브라우저와 서버는 HTTP로 대화하지만 운영에서는 암호화와 경로 분리가 필요하다.

따라 해보기

curl로 각 경로의 status code를 확인하면서 reverse proxy 감각을 익힌다.

관찰하기

작업 전후로 HTTP status, TLS handshake, proxy timeout, upstream health를 본다.를 기록한다. 숫자를 모르면 개선인지 착각인지 구분할 수 없다.

질문 만들기

'HTTP, HTTPS, Nginx Edge이 깨지면 사용자는 무엇을 보게 되는가?'를 한 문장으로 답해 본다.

기억할 문장

- HTTP, HTTPS, Nginx Edge은 '외부 입구를 하나로 묶는 법'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

02. HTTP, HTTPS, Nginx Edge - 복습 체크리스트

외부 입구를 하나로 묶는 법

이 페이지의 핵심

브라우저와 서버는 HTTP로 대화하지만 운영에서는 암호화와 경로 분리가 필요하다.

복습 질문

HTTP, HTTPS, Nginx Edge은 control plane, media plane, data plane, ops plane 중 어디에 가까운가?

체크리스트

경로가 상수화되어 있는가? DTO가 분리되어 있는가? 실패 테스트가 있는가? 운영 지표가 있는가? secret이 노출되지 않았는가?

다음 학습

관련 코드 파일 하나와 테스트 파일 하나를 함께 열어 구현과 검증이 어떻게 연결되는지 읽는다.

기억할 문장

- HTTP, HTTPS, Nginx Edge은 '외부 입구를 하나로 묶는 법'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

03. Docker Compose와 네트워크 - 개념 지도

여러 서비스를 한 번에 운영하기

이 페이지의 핵심

한 서버 안에서도 DB, cache, auth, media, dashboard를 프로세스별로 나눠야 장애와 확장이 쉬워진다.

왜 필요한가

한 서버 안에서도 DB, cache, auth, media, dashboard를 프로세스별로 나눠야 장애와 확장이 쉬워진다.

서버에서 맡는 역할

Compose는 control-net과 media-net을 나눠 서비스가 필요한 네트워크에만 붙도록 한다.

초보자 비유

여러 전자기기를 멀티탭에 꽂되, 전원 순서와 연결선을 정리해 둔 것과 같다.

기억할 문장

- Docker Compose와 네트워크는 '여러 서비스를 한 번에 운영하기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

03. Docker Compose와 네트워크 - 서버 적용 위치

여러 서비스를 한 번에 운영하기

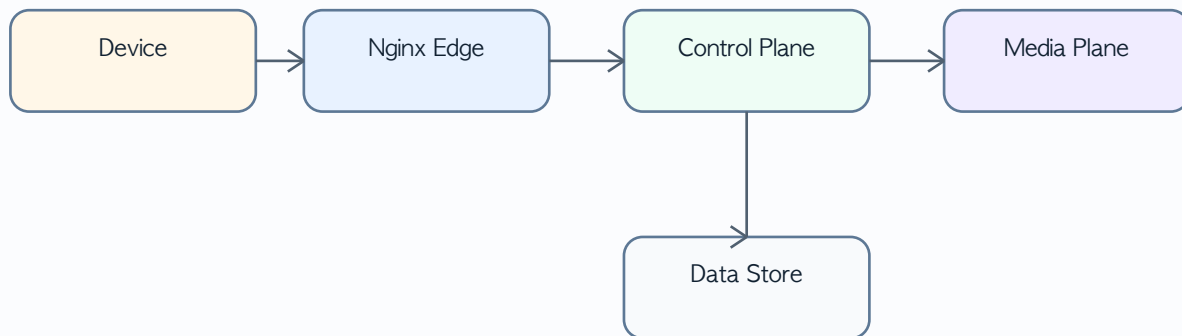


그림 해석: 여러 서비스를 한 번에 운영하기 - container health, restart count, image tag, network binding, exposed port를 본다.

서버 적용 위치

Compose는 control-net과 media-net을 나눠 서비스가 필요한 네트워크에만 붙도록 한다.

연결해서 볼 것

Docker Compose와 네트워크는 단독 기술이 아니라 Nginx, Docker Compose, 인증, 미디어 경로, 테스트와 함께 동작한다.

운영 지표

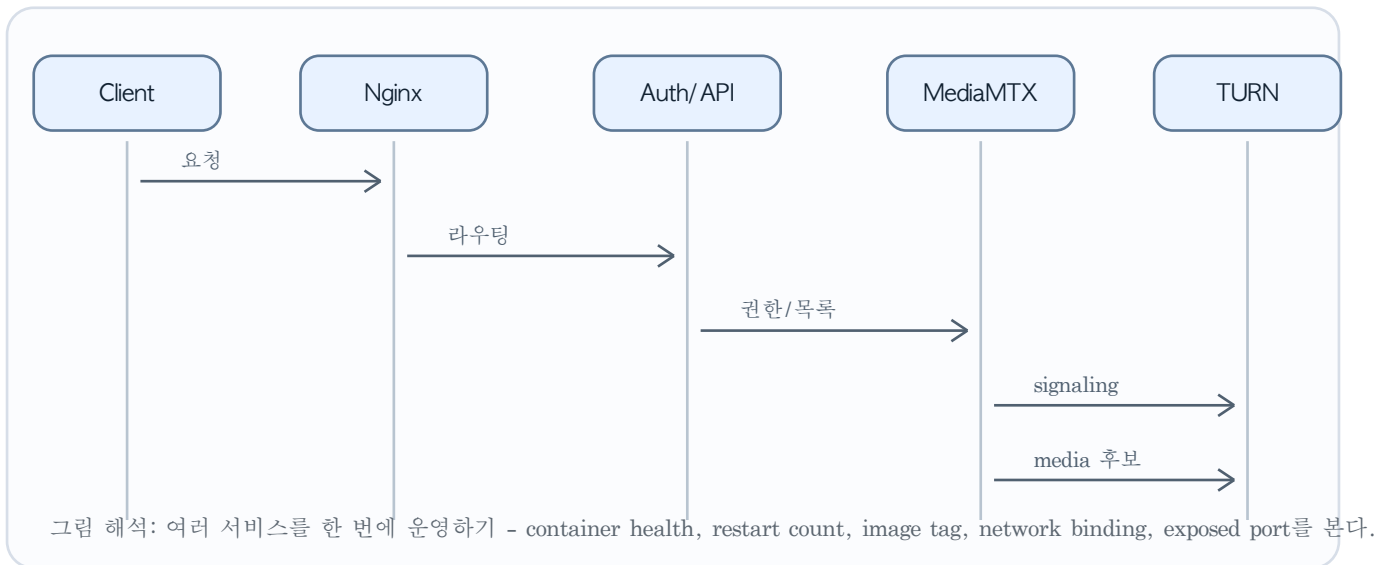
container health, restart count, image tag, network binding, exposed port를 본다.

기억할 문장

- Docker Compose와 네트워크는 '여러 서비스를 한 번에 운영하기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

03. Docker Compose와 네트워크 - 요청/데이터 흐름

여러 서비스를 한 번에 운영하기



흐름 읽기

요청은 항상 입구, 정책 판단, 실제 처리, 응답의 순서로 본다. Docker Compose와 네트워크에서도 같은 순서로 로그를 따라가면 원인 파악이 빨라진다.

확인할 데이터

container health, restart count, image tag, network binding, exposed port를 본다.

다음 단계

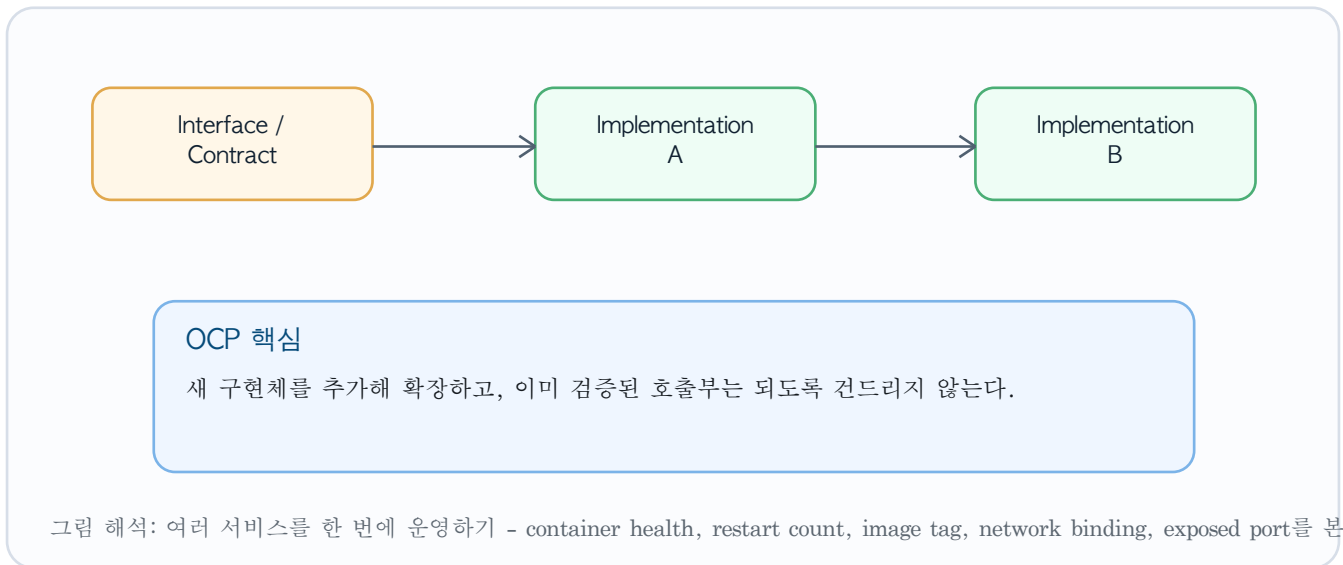
compose 파일에서 각 서비스가 어느 network에 붙는지 색칠해 본다.

기억할 문장

- Docker Compose와 네트워크는 '여러 서비스를 한 번에 운영하기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

03. Docker Compose와 네트워크 - 코드 작성 원칙

여러 서비스를 한 번에 운영하기



코드 원칙

환경 값은 .env와 example로 분리하고, secret은 GitHub에 올리지 않는다.

설계 연결

SRP는 책임을 나누고, OCP는 새 구현체를 붙이기 쉽게 만들며, DIP는 구체 구현 대신 interface를 바라보게 만든다.

작은 예시

문자열 endpoint를 컨트롤러에 직접 쓰지 말고 RouteContract 또는 ApiRoutes 같은 상수 객체에 모은다.

기억할 문장

- Docker Compose와 네트워크는 '여러 서비스를 한 번에 운영하기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

03. Docker Compose와 네트워크 - 테스트와 검증

여러 서비스를 한 번에 운영하기

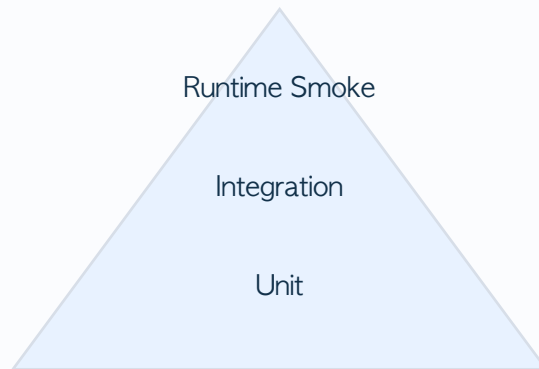


그림 해석: 여러 서비스를 한 번에 운영하기 - container health, restart count, image tag, network binding, exposed port를 본다.

테스트 원칙

docker compose config --quiet로 문법과 변수 누락을 먼저 잡는다.

TDD 관점

먼저 실패하는 계약 테스트를 만들고, 그 계약을 만족하는 가장 작은 구현을 작성한 뒤, 실제 runtime smoke로 연결을 확인한다.

통과 기준

container health, restart count, image tag, network binding, exposed port를 본다.가 기대 범위 안에 있고, 실패 시 원인이 로그와 테스트 이름으로 드러나야 한다.

기억할 문장

- Docker Compose와 네트워크는 '여러 서비스를 한 번에 운영하기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

03. Docker Compose와 네트워크 - 운영 체크

여러 서비스를 한 번에 운영하기

이 페이지의 핵심

한 서버 안에서도 DB, cache, auth, media, dashboard를
프로세스별로 나눠야 장애와 확장이 쉬워진다.

운영 원칙

컨테이너는 restart policy와 healthcheck를 가져야 운영자가 상태를 믿고 볼 수 있다.

장애 대응

status code, container health, application log, network path,
secret/config 누락을 순서대로 확인한다.

기록할 것

명령어, 실행 시각, 결과, 실패 원인, 수정 파일, 남은 리스크를 보고서에 남긴다.

기억할 문장

- Docker Compose와 네트워크는 '여러 서비스를 한 번에 운영하기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

03. Docker Compose와 네트워크 - 자주 나는 실수

여러 서비스를 한 번에 운영하기

이 페이지의 핵심

한 서버 안에서도 DB, cache, auth, media, dashboard를
프로세스별로 나눠야 장애와 확장이 쉬워진다.

자주 나는 실수

depends_on만 믿고 실제 준비 상태를 확인하지 않으면 시작 직후 500 오류가 생긴다.

왜 위험한가

초반에는 빠르게 보이지만, 운영 중 장애 원인을 숨기거나 변경 범위를 키워 결국 더 오래 걸리게 만든다.

예방 습관

계약값 중앙화, DTO 분리, 테스트 추가, 로그/지표 확인을 PR 기준으로 둔다.

기억할 문장

- Docker Compose와 네트워크은 '여러 서비스를 한 번에 운영하기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

03. Docker Compose와 네트워크 - 초보자 실습

여러 서비스를 한 번에 운영하기

이 페이지의 핵심

한 서버 안에서도 DB, cache, auth, media, dashboard를
프로세스별로 나눠야 장애와 확장이 쉬워진다.

따라 해보기

compose 파일에서 각 서비스가 어느 network에 붙는지 색칠해 본다.

관찰하기

작업 전후로 container health, restart count, image tag, network
binding, exposed port를 본다.를 기록한다. 숫자를 모르면 개선인지 착각인지 구분할 수 없다.

질문 만들기

'Docker Compose와 네트워크이 깨지면 사용자는 무엇을 보게 되는가?'를 한 문장으로 답해 본다.

기억할 문장

- Docker Compose와 네트워크는 '여러 서비스를 한 번에 운영하기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

03. Docker Compose와 네트워크 - 복습 체크리스트

여러 서비스를 한 번에 운영하기

이 페이지의 핵심

한 서버 안에서도 DB, cache, auth, media, dashboard를
프로세스별로 나눠야 장애와 확장이 쉬워진다.

복습 질문

Docker Compose와 네트워크는 control plane, media plane, data plane, ops plane 중 어디에 가까운가?

체크리스트

경로가 상수화되어 있는가? DTO가 분리되어 있는가? 실패 테스트가 있는가? 운영 지표가 있는가? secret이 노출되지 않았는가?

다음 학습

관련 코드 파일 하나와 테스트 파일 하나를 함께 열어 구현과 검증이 어떻게 연결되는지 읽는다.

기억할 문장

- Docker Compose와 네트워크는 '여러 서비스를 한 번에 운영하기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

04. React와 TypeScript Dashboard - 개념 지도

관제사가 실제로 보는 화면

이 페이지의 핵심

관제 화면은 실시간 스트림, 지도, 자산 트리, 이벤트 로그, 서버 상태를 동시에 다뤄야 한다.

왜 필요한가

관제 화면은 실시간 스트림, 지도, 자산 트리, 이벤트 로그, 서버 상태를 동시에 다뤄야 한다.

서버에서 맡는 역할

React component는 화면 조각, custom hook은 WebRTC와 API 상태, TypeScript는 타입 안전성을 담당한다.

초보자 비유

레고 블록처럼 작은 컴포넌트를 조합해 큰 대시보드를 만든다.

기억할 문장

- React와 TypeScript Dashboard은 '관제사가 실제로 보는 화면'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

04. React와 TypeScript Dashboard - 서버 적용 위치

관제사가 실제로 보는 화면

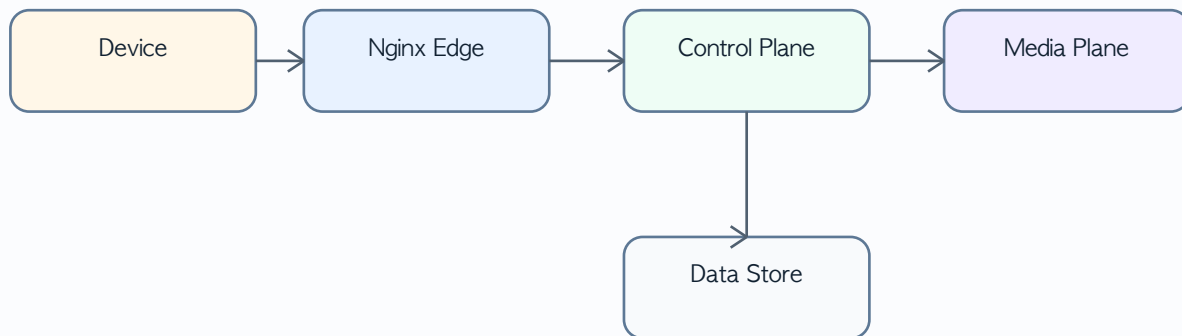


그림 해석: 관제사가 실제로 보는 화면 - render count, bundle chunk size, first frame, UI interaction delay를 본다.

서버 적용 위치

React component는 화면 조각, custom hook은 WebRTC와 API 상태, TypeScript는 타입 안전성을 담당한다.

연결해서 볼 것

React와 TypeScript Dashboard은 단독 기술이 아니라 Nginx, Docker Compose, 인증, 미디어 경로, 테스트와 함께 동작한다.

운영 지표

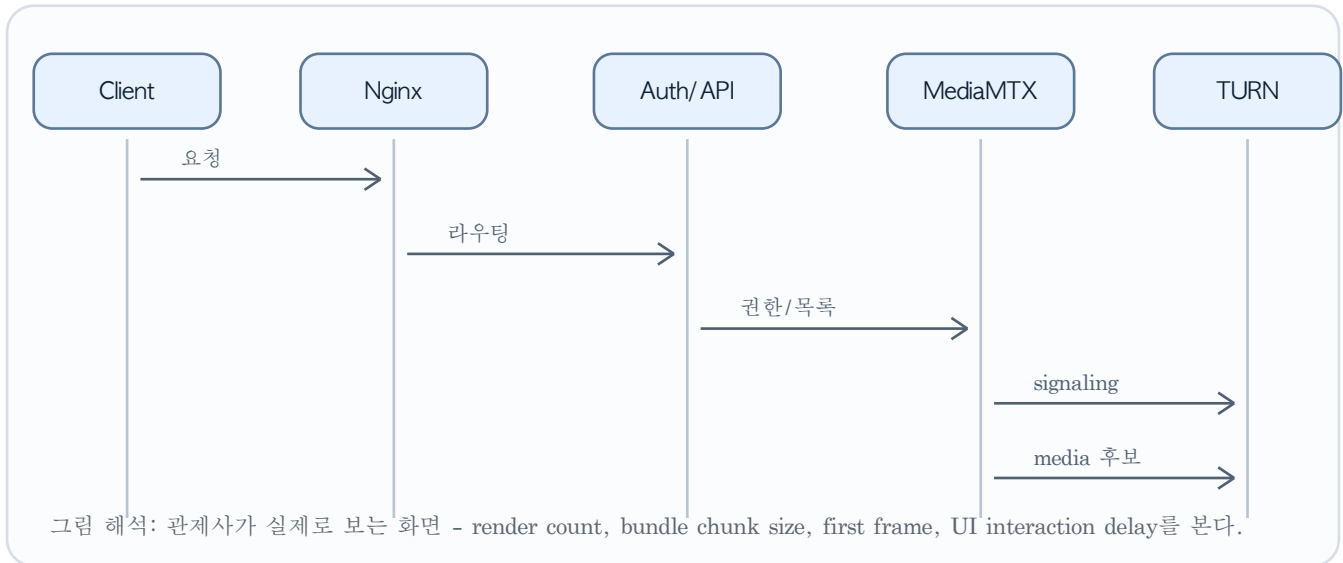
render count, bundle chunk size, first frame, UI interaction delay를 본다.

기억할 문장

- React와 TypeScript Dashboard은 '관제사가 실제로 보는 화면'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

04. React와 TypeScript Dashboard - 요청/데이터 흐름

관제사가 실제로 보는 화면



흐름 읽기

요청은 항상 입구, 정책 판단, 실제 처리, 응답의 순서로 본다. React와 TypeScript Dashboard에서도 같은 순서로 로그를 따라가면 원인 파악이 빨라진다.

확인할 데이터

render count, bundle chunk size, first frame, UI interaction delay를 본다.

다음 단계

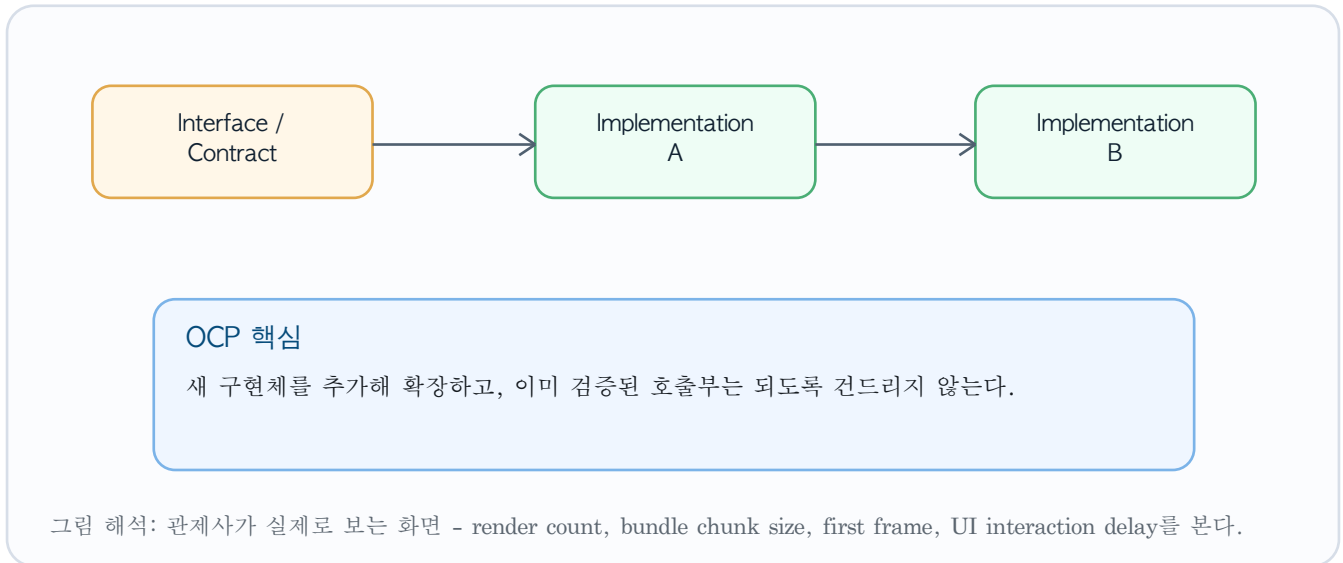
한 컴포넌트를 열어 props, state, effect, render가 어떤 순서로 움직이는지 추적해 본다.

기억할 문장

- React와 TypeScript Dashboard은 '관제사가 실제로 보는 화면'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

04. React와 TypeScript Dashboard - 코드 작성 원칙

관제사가 실제로 보는 화면



코드 원칙

API route, query key, status enum을 상수화하고 컴포넌트 하나에 상태를 과하게 몰아넣지 않는다.

설계 연결

SRP는 책임을 나누고, OCP는 새 구현체를 붙이기 쉽게 만들며, DIP는 구체 구현 대신 interface를 바라보게 만든다.

작은 예시

문자열 endpoint를 컨트롤러에 직접 쓰지 말고 RouteContract 또는 ApiRoutes 같은 상수 객체에 모은다.

기억할 문장

- React와 TypeScript Dashboard은 '관제사가 실제로 보는 화면'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

04. React와 TypeScript Dashboard - 테스트와 검증

관제사가 실제로 보는 화면

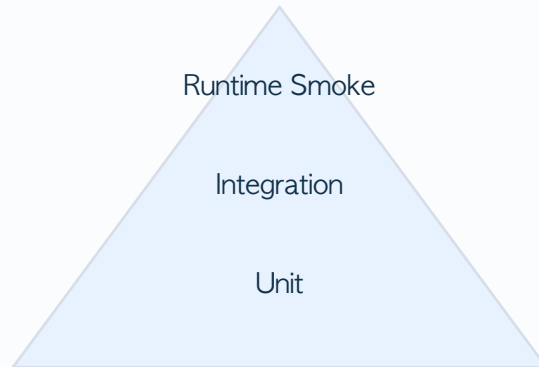


그림 해석: 관제사가 실제로 보는 화면 - render count, bundle chunk size, first frame, UI interaction delay를 본다.

테스트 원칙

Vitest와 Testing Library로 UI 동작, auth 상태, stream status, map focus를 검증한다.

TDD 관점

먼저 실패하는 계약 테스트를 만들고, 그 계약을 만족하는 가장 작은 구현을 작성한 뒤, 실제 runtime smoke로 연결을 확인한다.

통과 기준

render count, bundle chunk size, first frame, UI interaction delay를 본다.가 기대 범위 안에 있고, 실패 시 원인이 로그와 테스트 이름으로 드러나야 한다.

기억할 문장

- React와 TypeScript Dashboard은 '관제사가 실제로 보는 화면'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

04. React와 TypeScript Dashboard - 운영 체크

관제사가 실제로 보는 화면

이 페이지의 핵심

관제 화면은 실시간 스트림, 지도, 자산 트리, 이벤트 로그, 서버 상태를 동시에 다뤄야 한다.

운영 원칙

영상 위에 텍스트가 겹치지 않도록 플레이어 overlay는 최소화하고 데이터는 data 속성과 상태 콜백으로 노출한다.

장애 대응

status code, container health, application log, network path, secret/config 누락을 순서대로 확인한다.

기록할 것

명령어, 실행 시각, 결과, 실패 원인, 수정 파일, 남은 리스크를 보고서에 남긴다.

기억할 문장

- React와 TypeScript Dashboard은 '관제사가 실제로 보는 화면'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

04. React와 TypeScript Dashboard - 자주 나는 실수

관제사가 실제로 보는 화면

이 페이지의 핵심

관제 화면은 실시간 스트림, 지도, 자산 트리, 이벤트 로그, 서버 상태를 동시에 다뤄야 한다.

자주 나는 실수

useEffect가 너무 많으면 불필요한 렌더링과 메모리 누수가 생길 수 있다.

왜 위험한가

초반에는 빠르게 보이지만, 운영 중 장애 원인을 숨기거나 변경 범위를 키워 결국 더 오래 걸리게 만든다.

예방 습관

계약값 중앙화, DTO 분리, 테스트 추가, 로그/지표 확인을 PR 기준으로 둔다.

기억할 문장

- React와 TypeScript Dashboard은 '관제사가 실제로 보는 화면'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

04. React와 TypeScript Dashboard - 초보자 실습

관제사가 실제로 보는 화면

이 페이지의 핵심

관제 화면은 실시간 스트림, 지도, 자산 트리, 이벤트 로그, 서버 상태를 동시에 다뤄야 한다.

따라 해보기

한 컴포넌트를 열어 props, state, effect, render가 어떤 순서로 움직이는지 추적해 본다.

관찰하기

작업 전후로 render count, bundle chunk size, first frame, UI interaction delay를 본다.를 기록한다. 숫자를 모르면 개선인지 착각인지 구분할 수 없다.

질문 만들기

'React와 TypeScript Dashboard이 깨지면 사용자는 무엇을 보게 되는가?'를 한 문장으로 답해 본다.

기억할 문장

- React와 TypeScript Dashboard은 '관제사가 실제로 보는 화면'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

04. React와 TypeScript Dashboard - 복습 체크리스트

관제사가 실제로 보는 화면

이 페이지의 핵심

관제 화면은 실시간 스트림, 지도, 자산 트리, 이벤트 로그, 서버 상태를 동시에 다뤄야 한다.

복습 질문

React와 TypeScript Dashboard은 control plane, media plane, data plane, ops plane 중 어디에 가까운가?

체크리스트

경로가 상수화되어 있는가? DTO가 분리되어 있는가? 실패 테스트가 있는가? 운영 지표가 있는가? secret이 노출되지 않았는가?

다음 학습

관련 코드 파일 하나와 테스트 파일 하나를 함께 열어 구현과 검증이 어떻게 연결되는지 읽는다.

기억할 문장

- React와 TypeScript Dashboard은 '관제사가 실제로 보는 화면'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

05. WebRTC 기본 - 개념 지도

브라우저 실시간 미디어 연결

이 페이지의 핵심

WebRTC는 브라우저에서 영상과 음성을 낮은 지연으로 주고받기 위한 표준 기술이다.

왜 필요한가

WebRTC는 브라우저에서 영상과 음성을 낮은 지연으로 주고받기 위한 표준 기술이다.

서버에서 맡는 역할

GCS-Saker에서는 publisher가 WHIP로 송출하고 dashboard가 WHEP로 수신한다.

초보자 비유

전화 통화를 시작하기 전에 서로 번호와 연결 방법을 맞추는 과정과 비슷하다.

기억할 문장

- WebRTC 기본은 '브라우저 실시간 미디어 연결'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

05. WebRTC 기본 - 서버 적용 위치

브라우저 실시간 미디어 연결

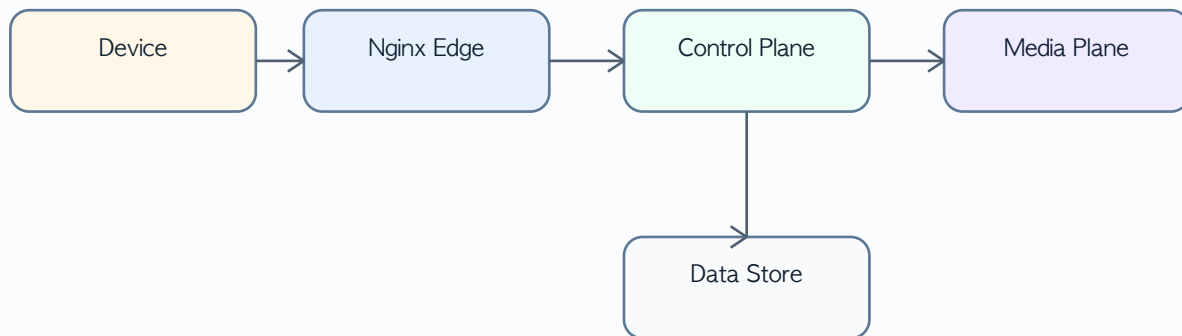


그림 해석: 브라우저 실시간 미디어 연결 - offer latency, answer latency, ICE connected latency, first-frame latency를 본다.

서버 적용 위치

GCS-Saker에서는 publisher가 WHIP로 송출하고 dashboard가 WHEP로 수신한다.

연결해서 볼 것

WebRTC 기본은 단독 기술이 아니라 Nginx, Docker Compose, 인증, 미디어 경로, 테스트와 함께 동작한다.

운영 지표

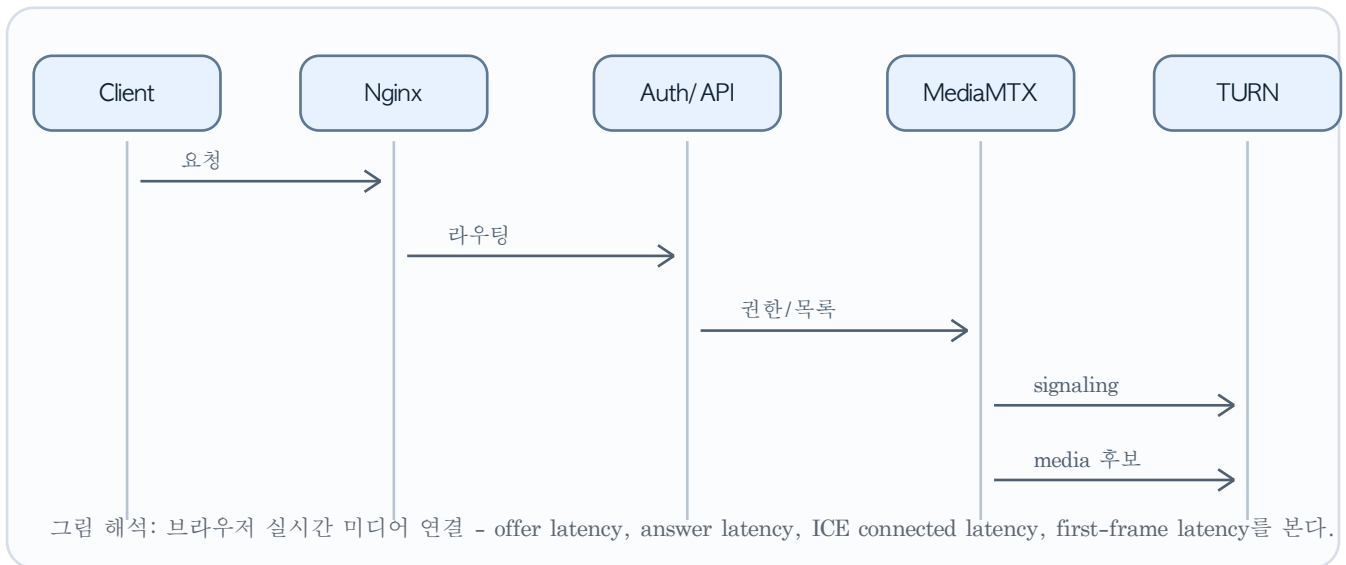
offer latency, answer latency, ICE connected latency, first-frame latency를 본다.

기억할 문장

- WebRTC 기본은 '브라우저 실시간 미디어 연결'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

05. WebRTC 기본 - 요청/데이터 흐름

브라우저 실시간 미디어 연결



흐름 읽기

요청은 항상 입구, 정책 판단, 실제 처리, 응답의 순서로 본다. WebRTC 기본에서도 같은 순서로 로그를 따라가면 원인 파악이 빨라진다.

확인할 데이터

offer latency, answer latency, ICE connected latency, first-frame latency를 본다.

다음 단계

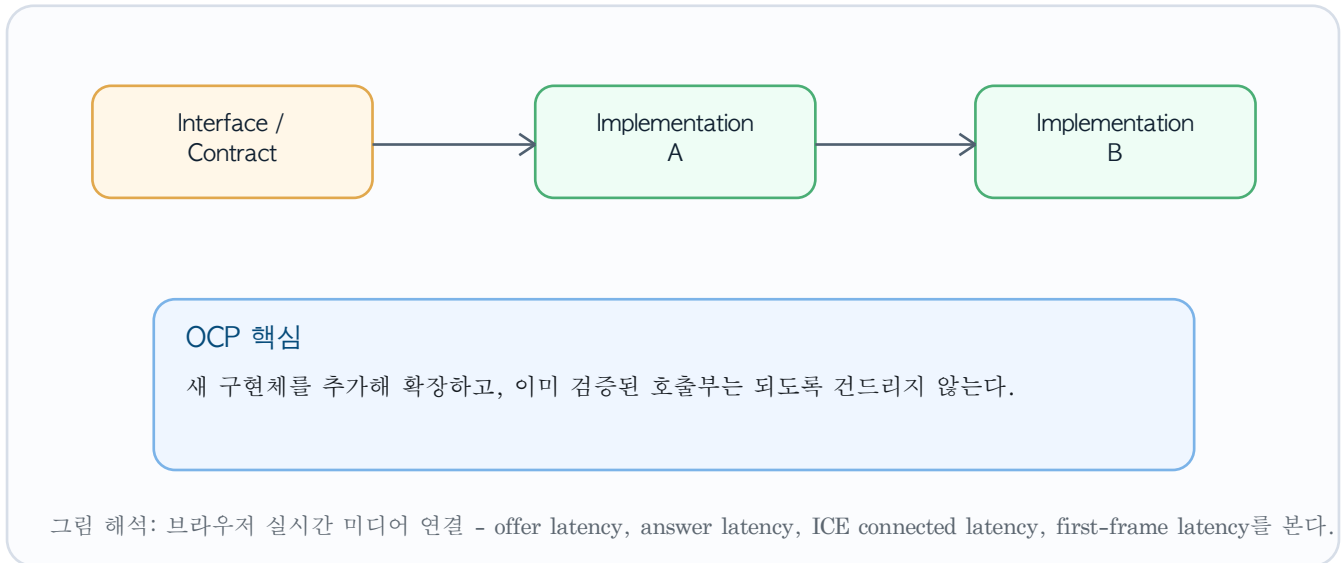
브라우저 개발자 도구의 webrtc-internals를 켜고 candidate와 stats를 관찰한다.

기억할 문장

- WebRTC 기본은 '브라우저 실시간 미디어 연결'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

05. WebRTC 기본 - 코드 작성 원칙

브라우저 실시간 미디어 연결



코드 원칙

signaling, ICE, media track, stats를 분리해 생각하고, UI 컴포넌트는 hook이 만든 상태만 사용한다.

설계 연결

SRP는 책임을 나누고, OCP는 새 구현체를 붙이기 쉽게 만들며, DIP는 구체 구현 대신 interface를 바라보게 만든다.

작은 예시

문자열 endpoint를 컨트롤러에 직접 쓰지 말고 RouteContract 또는 ApiRoutes 같은 상수 객체에 모은다.

기억할 문장

- WebRTC 기본은 '브라우저 실시간 미디어 연결'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

05. WebRTC 기본 - 테스트와 검증

브라우저 실시간 미디어 연결

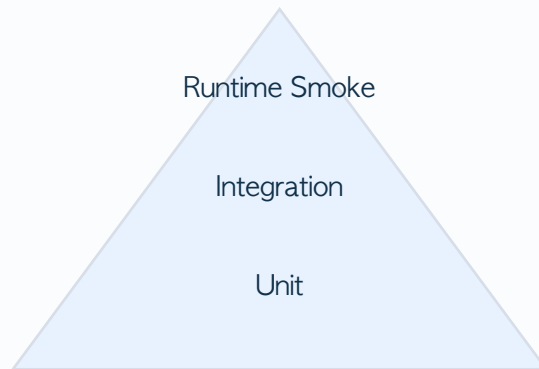


그림 해석: 브라우저 실시간 미디어 연결 - offer latency, answer latency, ICE connected latency, first-frame latency를 본다.

테스트 원칙

WHIP/WHEP smoke와 getStats test로 signaling과 media 수신 상태를 따로 검증한다.

TDD 관점

먼저 실패하는 계약 테스트를 만들고, 그 계약을 만족하는 가장 작은 구현을 작성한 뒤, 실제 runtime smoke로 연결을 확인한다.

통과 기준

offer latency, answer latency, ICE connected latency, first-frame latency를 본다.가 기대 범위 안에 있고, 실패 시 원인이 로그와 테스트 이름으로 드러나야 한다.

기억할 문장

- WebRTC 기본은 '브라우저 실시간 미디어 연결'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

05. WebRTC 기본 - 운영 체크

브라우저 실시간 미디어 연결

이 페이지의 핵심

WebRTC는 브라우저에서 영상과 음성을 낮은 지연으로 주고받기 위한 표준 기술이다.

운영 원칙

운영자는 first-frame latency, ICE state, candidate type, packet loss를 확인해야 한다.

장애 대응

status code, container health, application log, network path, secret/config 누락을 순서대로 확인한다.

기록할 것

명령어, 실행 시각, 결과, 실패 원인, 수정 파일, 남은 리스크를 보고서에 남긴다.

기억할 문장

- WebRTC 기본은 '브라우저 실시간 미디어 연결'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

05. WebRTC 기본 - 자주 나는 실수

브라우저 실시간 미디어 연결

이 페이지의 핵심

WebRTC는 브라우저에서 영상과 음성을 낮은 지연으로 주고받기 위한 표준 기술이다.

자주 나는 실수

signaling HTTP가 200이어도 media packet이 연결되지 않으면 영상은 나오지 않는다.

왜 위험한가

초반에는 빠르게 보이지만, 운영 중 장애 원인을 숨기거나 변경 범위를 키워 결국 더 오래 걸리게 만든다.

예방 습관

계약값 중앙화, DTO 분리, 테스트 추가, 로그/지표 확인을 PR 기준으로 둔다.

기억할 문장

- WebRTC 기본은 '브라우저 실시간 미디어 연결'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

05. WebRTC 기본 - 초보자 실습

브라우저 실시간 미디어 연결

이 페이지의 핵심

WebRTC는 브라우저에서 영상과 음성을 낮은 지연으로 주고받기 위한 표준 기술이다.

따라 해보기

브라우저 개발자 도구의 `webrtc-internals`를 켜고 `candidate`와 `stats`를 관찰한다.

관찰하기

작업 전후로 `offer latency`, `answer latency`, `ICE connected latency`, `first-frame latency`를 본다.를 기록한다. 숫자를 모르면 개선인지 착각인지 구분할 수 없다.

질문 만들기

'WebRTC 기본이 깨지면 사용자는 무엇을 보게 되는가?'를 한 문장으로 답해 본다.

기억할 문장

- WebRTC 기본은 '브라우저 실시간 미디어 연결'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

05. WebRTC 기본 - 복습 체크리스트

브라우저 실시간 미디어 연결

이 페이지의 핵심

WebRTC는 브라우저에서 영상과 음성을 낮은 지연으로 주고받기 위한 표준 기술이다.

복습 질문

WebRTC 기본은 control plane, media plane, data plane, ops plane 중 어디에 가까운가?

체크리스트

경로가 상수화되어 있는가? DTO가 분리되어 있는가? 실패 테스트가 있는가? 운영 지표가 있는가? secret이 노출되지 않았는가?

다음 학습

관련 코드 파일 하나와 테스트 파일 하나를 함께 열어 구현과 검증이 어떻게 연결되는지 읽는다.

기억할 문장

- WebRTC 기본은 '브라우저 실시간 미디어 연결'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

06. STUN, TURN, NAT - 개념 지도

공유기 뒤 장비를 연결하는 법

이 페이지의 핵심

현장 장비와 관제 브라우저는 대부분 NAT와 방화벽 뒤에 있으므로 직접 연결이 항상 되지는 않는다.

왜 필요한가

현장 장비와 관제 브라우저는 대부분 NAT와 방화벽 뒤에 있으므로 직접 연결이 항상 되지는 않는다.

서버에서 맡는 역할

STUN은 외부에서 보이는 주소 후보를 찾고, TURN은 직접 연결 실패 시 미디어를 relay한다.

초보자 비유

STUN은 거울, TURN은 우회 택배라고 생각하면 쉽다.

기억할 문장

- STUN, TURN, NAT은 '공유기 뒤 장비를 연결하는 법'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

06. STUN, TURN, NAT - 서버 적용 위치

공유기 뒤 장비를 연결하는 법

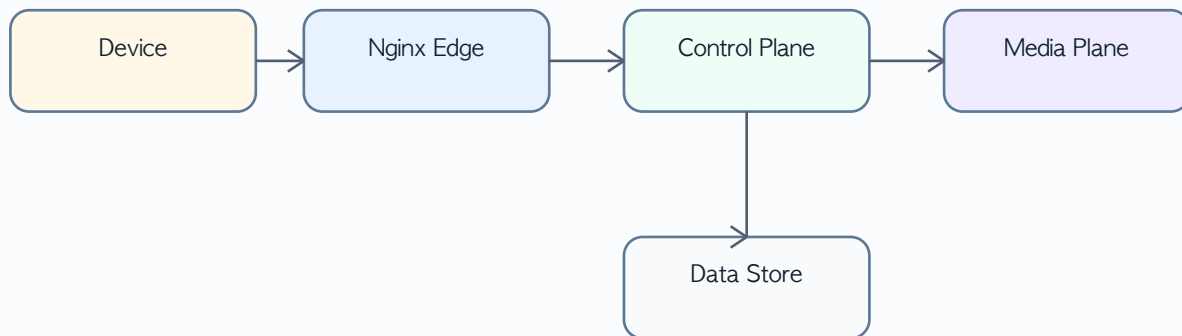


그림 해석: 공유기 뒤 장비를 연결하는 법 - allocation latency, relay candidate count, packet loss, relay bandwidth를 본다.

서버 적용 위치

STUN은 외부에서 보이는 주소 후보를 찾고, TURN은 직접 연결 실패 시 미디어를 relay한다.

연결해서 볼 것

STUN, TURN, NAT은 단독 기술이 아니라 Nginx, Docker Compose, 인증, 미디어 경로, 테스트와 함께 동작한다.

운영 지표

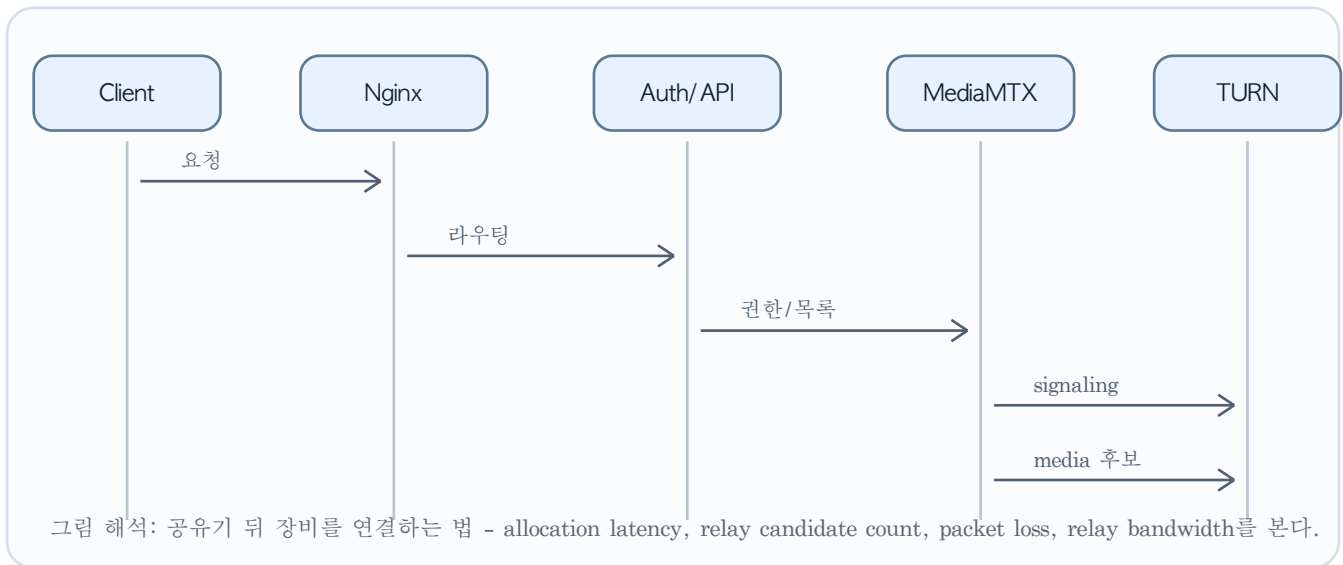
allocation latency, relay candidate count, packet loss, relay bandwidth를 본다.

기억할 문장

- STUN, TURN, NAT은 '공유기 뒤 장비를 연결하는 법'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

06. STUN, TURN, NAT - 요청/데이터 흐름

공유기 뒤 장비를 연결하는 법



흐름 읽기

요청은 항상 입구, 정책 판단, 실제 처리, 응답의 순서로 본다. STUN, TURN, NAT에서도 같은 순서로 로그를 따라가면 원인 파악이 빨라진다.

확인할 데이터

allocation latency, relay candidate count, packet loss, relay bandwidth를 본다.

다음 단계

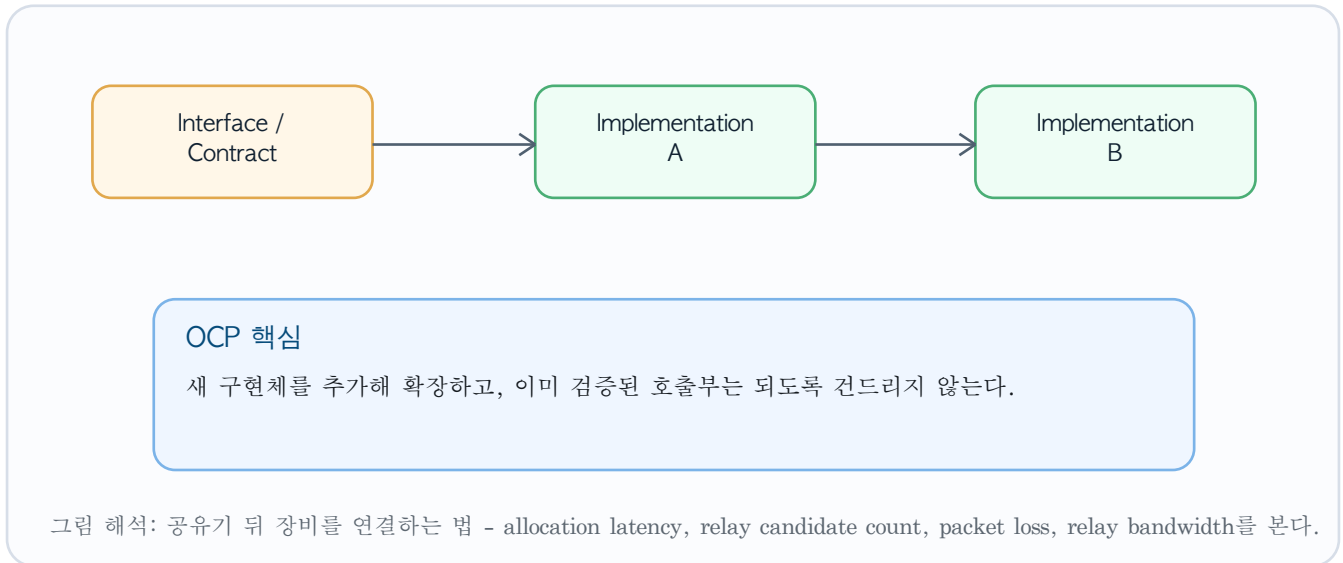
direct STUN 실패와 TURN 성공의 로그 차이를 비교한다.

기억할 문장

- STUN, TURN, NAT은 '공유기 뒤 장비를 연결하는 법'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

06. STUN, TURN, NAT - 코드 작성 원칙

공유기 뒤 장비를 연결하는 법



코드 원칙

ICE server list는 일급 컬렉션과 contract로 관리해 잘못된 URL과 credential 누락을 막는다.

설계 연결

SRP는 책임을 나누고, OCP는 새 구현체를 붙이기 쉽게 만들며, DIP는 구체 구현 대신 interface를 바라보게 만든다.

작은 예시

문자열 endpoint를 컨트롤러에 직접 쓰지 말고 RouteContract 또는 ApiRoutes 같은 상수 객체에 모은다.

기억할 문장

- STUN, TURN, NAT은 '공유기 뒤 장비를 연결하는 법'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

06. STUN, TURN, NAT - 테스트와 검증

공유기 뒤 장비를 연결하는 법

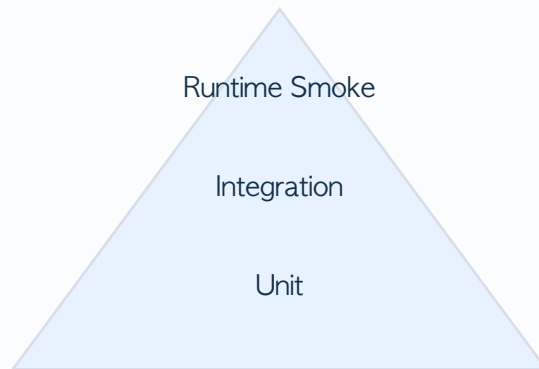


그림 해석: 공유기 뒤 장비를 연결하는 법 - allocation latency, relay candidate count, packet loss, relay bandwidth를 본다.

테스트 원칙

TURN allocation smoke, relay-only WebRTC smoke, candidate summary test를 분리한다.

TDD 관점

먼저 실패하는 계약 테스트를 만들고, 그 계약을 만족하는 가장 작은 구현을 작성한 뒤, 실제 runtime smoke로 연결을 확인한다.

통과 기준

allocation latency, relay candidate count, packet loss, relay bandwidth를 본다.가 기대 범위 안에 있고, 실패 시 원인이 로그와 테스트 이름으로 드러나야 한다.

기억할 문장

- STUN, TURN, NAT은 '공유기 뒤 장비를 연결하는 법'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

06. STUN, TURN, NAT - 운영 체크

공유기 뒤 장비를 연결하는 법

이 페이지의 핵심

현장 장비와 관제 브라우저는 대부분 NAT와 방화벽 뒤에 있으므로 직접 연결이 항상 되지는 않는다.

운영 원칙

포트 3478/3479와 relay range, firewall, advertised IP가 맞는지 운영 기준으로 확인한다.

장애 대응

status code, container health, application log, network path, secret/config 누락을 순서대로 확인한다.

기록할 것

명령어, 실행 시각, 결과, 실패 원인, 수정 파일, 남은 리스크를 보고서에 남긴다.

기억할 문장

- STUN, TURN, NAT은 '공유기 뒤 장비를 연결하는 법'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

06. STUN, TURN, NAT - 자주 나는 실수

공유기 뒤 장비를 연결하는 법

이 페이지의 핵심

현장 장비와 관제 브라우저는 대부분 NAT와 방화벽 뒤에 있으므로 직접 연결이 항상 되지는 않는다.

자주 나는 실수

TURN을 항상 쓰면 안정성은 좋아지지만 서버 트래픽과 지연이 늘 수 있다.

왜 위험한가

초반에는 빠르게 보이지만, 운영 중 장애 원인을 숨기거나 변경 범위를 키워 결국 더 오래 걸리게 만든다.

예방 습관

계약값 중앙화, DTO 분리, 테스트 추가, 로그/지표 확인을 PR 기준으로 둔다.

기억할 문장

- STUN, TURN, NAT은 '공유기 뒤 장비를 연결하는 법'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

06. STUN, TURN, NAT - 초보자 실습

공유기 뒤 장비를 연결하는 법

이 페이지의 핵심

현장 장비와 관제 브라우저는 대부분 NAT와 방화벽 뒤에 있으므로 직접 연결이 항상 되지는 않는다.

따라 해보기

direct STUN 실패와 TURN 성공의 로그 차이를 비교한다.

관찰하기

작업 전후로 allocation latency, relay candidate count, packet loss, relay bandwidth를 본다.를 기록한다. 숫자를 모르면 개선인지 착각인지 구분할 수 없다.

질문 만들기

'STUN, TURN, NAT이 깨지면 사용자는 무엇을 보게 되는가?'를 한 문장으로 답해 본다.

기억할 문장

- STUN, TURN, NAT은 '공유기 뒤 장비를 연결하는 법'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

06. STUN, TURN, NAT - 복습 체크리스트

공유기 뒤 장비를 연결하는 법

이 페이지의 핵심

현장 장비와 관제 브라우저는 대부분 NAT와 방화벽 뒤에 있으므로 직접 연결이 항상 되지는 않는다.

복습 질문

STUN, TURN, NAT은 control plane, media plane, data plane, ops plane
중 어디에 가까운가?

체크리스트

경로가 상수화되어 있는가? DTO가 분리되어 있는가? 실패 테스트가 있는가? 운영 지표가 있는가? secret이 노출되지 않았는가?

다음 학습

관련 코드 파일 하나와 테스트 파일 하나를 함께 열어 구현과 검증이 어떻게 연결되는지 읽는다.

기억할 문장

- STUN, TURN, NAT은 '공유기 뒤 장비를 연결하는 법'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

07. MediaMTX, WHIP, WHEP, HLS - 개념 지도

미디어 서버와 표준 입출력

이 페이지의 핵심

MediaMTX는 미디어 패킷 처리를 앱 서버에서 분리해 실시간성을 지키게 해준다.

왜 필요한가

MediaMTX는 미디어 패킷 처리를 앱 서버에서 분리해 실시간성을 지키게 해준다.

서버에서 맡는 역할

WHIP는 송출, WHEP는 수신, HLS는 WebRTC 실패 시 fallback 경로를 담당한다.

초보자 비유

방송국 중계 장비처럼 여러 입력을 받아 여러 시청 방식으로 내보낸다.

기억할 문장

- MediaMTX, WHIP, WHEP, HLS은 '미디어 서버와 표준 입출력'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

07. MediaMTX, WHIP, WHEP, HLS - 서버 적용 위치

미디어 서버와 표준 입출력

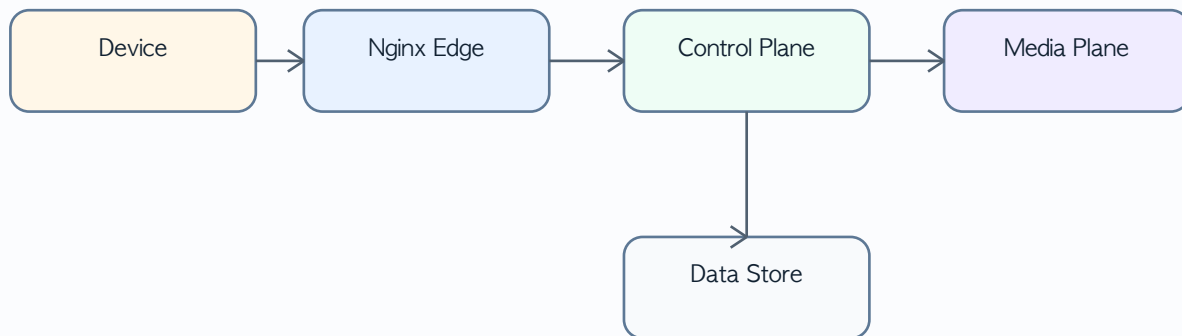


그림 해석: 미디어 서버와 표준 입출력 - WHEP status, HLS playlist status, MediaMTX API response, path count를 본다.

서버 적용 위치

WHIP는 송출, WHEP는 수신, HLS는 WebRTC 실패 시 fallback 경로를 담당한다.

연결해서 볼 것

MediaMTX, WHIP, WHEP, HLS은 단독 기술이 아니라 Nginx, Docker Compose, 인증, 미디어 경로, 테스트와 함께 동작한다.

운영 지표

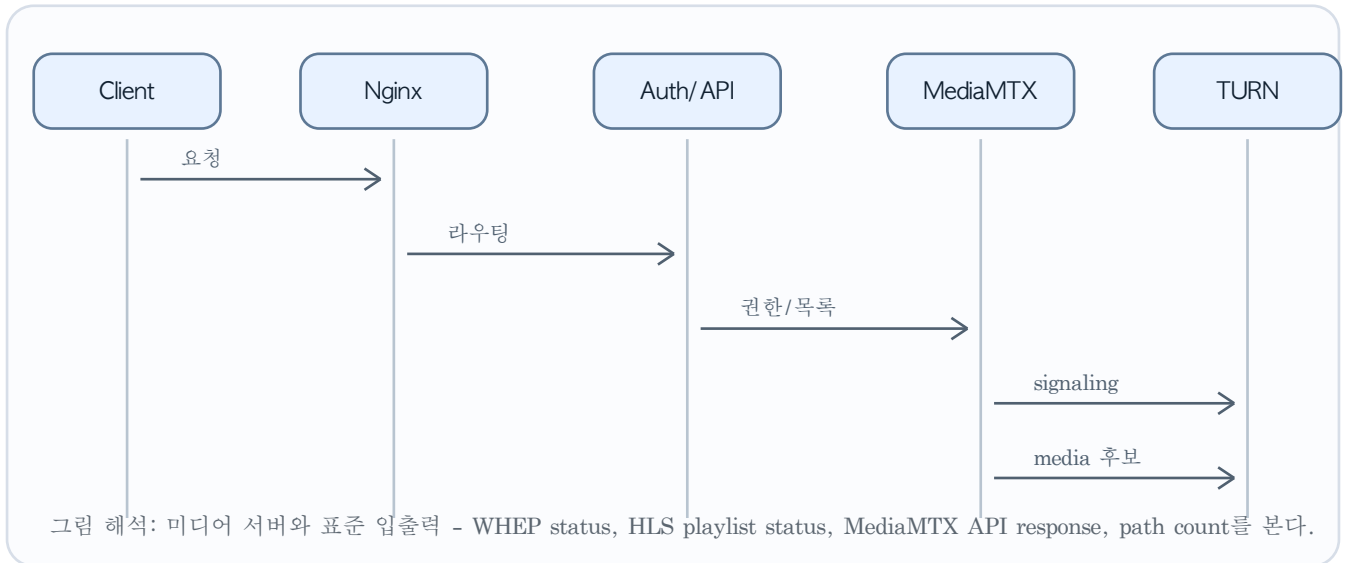
WHEP status, HLS playlist status, MediaMTX API response, path count를 본다.

기억할 문장

- MediaMTX, WHIP, WHEP, HLS은 '미디어 서버와 표준 입출력'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

07. MediaMTX, WHIP, WHEP, HLS - 요청/데이터 흐름

미디어 서버와 표준 입출력



흐름 읽기

요청은 항상 입구, 정책 판단, 실제 처리, 응답의 순서로 본다. MediaMTX, WHIP, WHEP, HLS에서도 같은 순서로 로그를 따라가면 원인 파악이 빨라진다.

확인할 데이터

WHEP status, HLS playlist status, MediaMTX API response, path count를 본다.

다음 단계

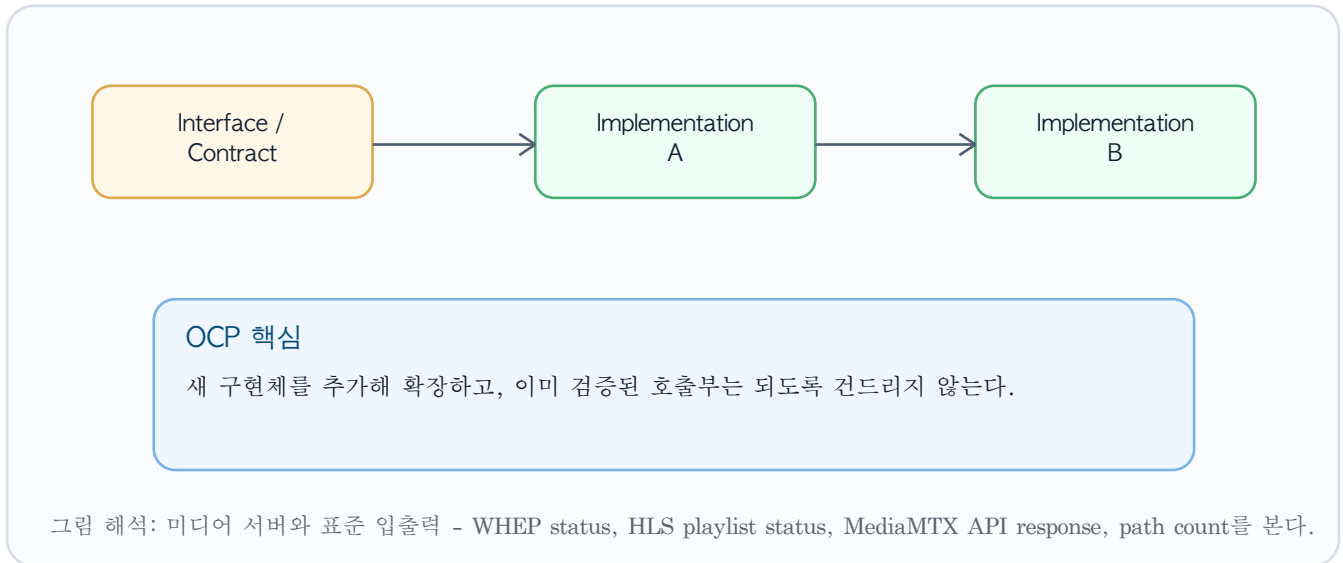
같은 stream path를 WHIP, WHEP, HLS URL로 바꿔 써 보며 차이를 익힌다.

기억할 문장

- MediaMTX, WHIP, WHEP, HLS은 '미디어 서버와 표준 입출력'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

07. MediaMTX, WHIP, WHEP, HLS - 코드 작성 원칙

미디어 서버와 표준 입출력



코드 원칙

MediaMTX API 접근은 adapter로 감싸고, stream path는 값 객체로 검증한다.

설계 연결

SRP는 책임을 나누고, OCP는 새 구현체를 붙이기 쉽게 만들며, DIP는 구체 구현 대신 interface를 바라보게 만든다.

작은 예시

문자열 endpoint를 컨트롤러에 직접 쓰지 말고 RouteContract 또는 ApiRoutes 같은 상수 객체에 모은다.

기억할 문장

- MediaMTX, WHIP, WHEP, HLS은 '미디어 서버와 표준 입출력'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

07. MediaMTX, WHIP, WHEP, HLS - 테스트와 검증

미디어 서버와 표준 입출력

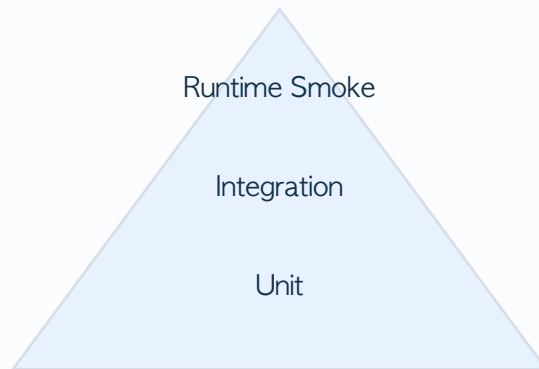


그림 해석: 미디어 서버와 표준 입출력 - WHEP status, HLS playlist status, MediaMTX API response, path count를 본다.

테스트 원칙

publish-play smoke, first-frame smoke, HLS playlist smoke를 각각 둔다.

TDD 관점

먼저 실패하는 계약 테스트를 만들고, 그 계약을 만족하는 가장 작은 구현을 작성한 뒤, 실제 runtime smoke로 연결을 확인한다.

통과 기준

WHEP status, HLS playlist status, MediaMTX API response, path count를 본다.가 기대 범위 안에 있고, 실패 시 원인이 로그와 테스트 이름으로 드러나야 한다.

기억할 문장

- MediaMTX, WHIP, WHEP, HLS은 '미디어 서버와 표준 입출력'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

07. MediaMTX, WHIP, WHEP, HLS - 운영 체크

미디어 서버와 표준 입출력

이 페이지의 핵심

MediaMTX는 미디어 패킷 처리를 앱 서버에서 분리해 실시간성을 지키게 해준다.

운영 원칙

candidate에 private IP가 섞이는지와 HLS fallback이 실제로 가능한지를 운영에서 확인한다.

장애 대응

status code, container health, application log, network path,
secret/config 누락을 순서대로 확인한다.

기록할 것

명령어, 실행 시각, 결과, 실패 원인, 수정 파일, 남은 리스크를 보고서에 남긴다.

기억할 문장

- MediaMTX, WHIP, WHEP, HLS은 '미디어 서버와 표준 입출력'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

07. MediaMTX, WHIP, WHEP, HLS - 자주 나는 실수

미디어 서버와 표준 입출력

이 페이지의 핵심

MediaMTX는 미디어 패킷 처리를 앱 서버에서 분리해 실시간성을 지키게 해준다.

자주 나는 실수

MediaMTX만 정상이라고 WebRTC가 항상 되는 것은 아니다. ICE와 proxy도 함께 맞아야 한다.

왜 위험한가

초반에는 빠르게 보이지만, 운영 중 장애 원인을 숨기거나 변경 범위를 키워 결국 더 오래 걸리게 만든다.

예방 습관

계약값 중앙화, DTO 분리, 테스트 추가, 로그/지표 확인을 PR 기준으로 둔다.

기억할 문장

- MediaMTX, WHIP, WHEP, HLS은 '미디어 서버와 표준 입출력'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

07. MediaMTX, WHIP, WHEP, HLS - 초보자 실습

미디어 서버와 표준 입출력

이 페이지의 핵심

MediaMTX는 미디어 패킷 처리를 앱 서버에서 분리해 실시간성을 지키게 해준다.

따라 해보기

같은 stream path를 WHIP, WHEP, HLS URL로 바꿔 써 보며 차이를 익힌다.

관찰하기

작업 전후로 WHEP status, HLS playlist status, MediaMTX API response, path count를 본다.를 기록한다. 숫자를 모르면 개선인지 착각인지 구분할 수 없다.

질문 만들기

'MediaMTX, WHIP, WHEP, HLS이 깨지면 사용자는 무엇을 보게 되는가?'를 한 문장으로 답해 본다.

기억할 문장

- MediaMTX, WHIP, WHEP, HLS은 '미디어 서버와 표준 입출력'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

07. MediaMTX, WHIP, WHEP, HLS - 복습 체크리스트

미디어 서버와 표준 입출력

이 페이지의 핵심

MediaMTX는 미디어 패킷 처리를 앱 서버에서 분리해 실시간성을 지키게 해준다.

복습 질문

MediaMTX, WHIP, WHEP, HLS은 control plane, media plane, data plane, ops plane 중 어디에 가까운가?

체크리스트

경로가 상수화되어 있는가? DTO가 분리되어 있는가? 실패 테스트가 있는가? 운영 지표가 있는가? secret이 노출되지 않았는가?

다음 학습

관련 코드 파일 하나와 테스트 파일 하나를 함께 열어 구현과 검증이 어떻게 연결되는지 읽는다.

기억할 문장

- MediaMTX, WHIP, WHEP, HLS은 '미디어 서버와 표준 입출력'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

08. 초저지연 스트리밍 - 개념 지도

빠르게 보이게 만드는 선택들

이 페이지의 핵심

관제 시스템은 몇 초 늦은 영상보다 현재에 가까운 영상이 중요하다.

왜 필요한가

관제 시스템은 몇 초 늦은 영상보다 현재에 가까운 영상이 중요하다.

서버에서 맡는 역할

WebRTC를 기본 수신 경로로 두고, HLS는 호환성 fallback으로 둔다.

초보자 비유

고화질 사진보다 실시간 무전이 더 중요한 상황을 떠올리면 된다.

기억할 문장

- 초저지연 스트리밍은 '빠르게 보이게 만드는 선택들'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

08. 초저지연 스트리밍 - 서버 적용 위치

빠르게 보이게 만드는 선택들

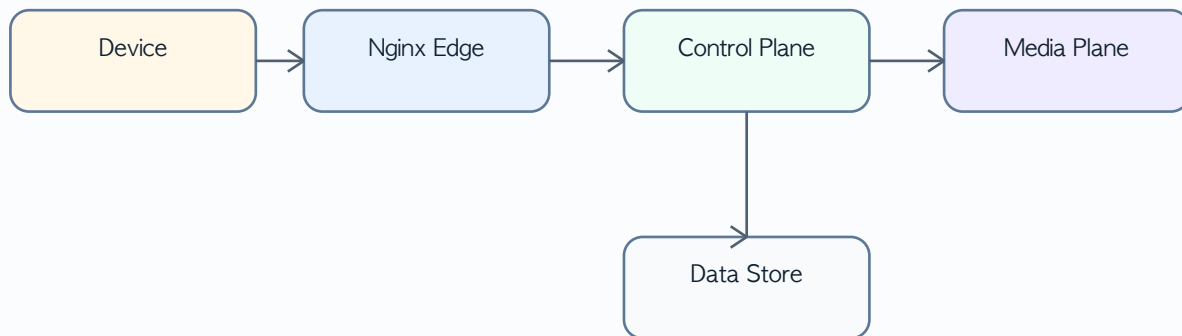


그림 해석: 빠르게 보이게 만드는 선택들 - first frame, jitter, RTT, dropped frame, concealed samples를 본다.

서버 적용 위치

WebRTC를 기본 수신 경로로 두고, HLS는 호환성 fallback으로 둔다.

연결해서 볼 것

초저지연 스트리밍은 단독 기술이 아니라 Nginx, Docker Compose, 인증, 미디어 경로, 테스트와 함께 동작한다.

운영 지표

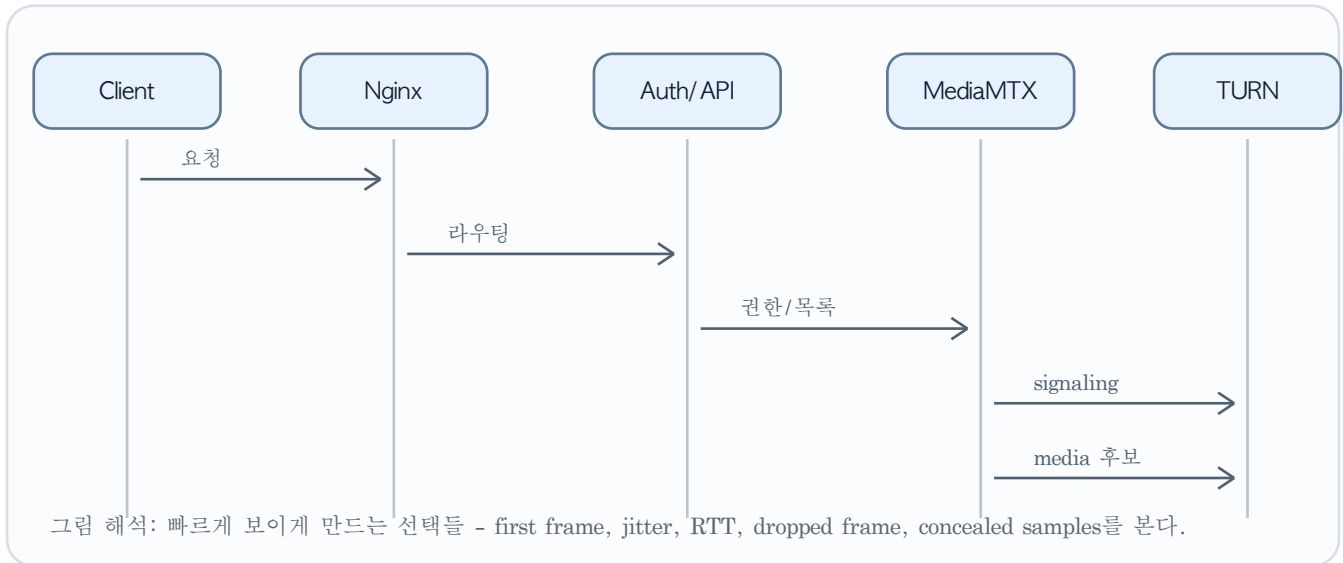
first frame, jitter, RTT, dropped frame, concealed samples를 본다.

기억할 문장

- 초저지연 스트리밍은 '빠르게 보이게 만드는 선택들'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

08. 초저지연 스트리밍 - 요청/데이터 흐름

빠르게 보이게 만드는 선택들



흐름 읽기

요청은 항상 입구, 정책 판단, 실제 처리, 응답의 순서로 본다. 초저지연 스트리밍에서도 같은 순서로 로그를 따라가면 원인 파악이 빨라진다.

확인할 데이터

first frame, jitter, RTT, dropped frame, concealed samples를 본다.

다음 단계

저지연 오디오 모드와 음질 모드를 바꿔 실제 차이를 느껴 본다.

기억할 문장

- 초저지연 스트리밍은 '빠르게 보이게 만드는 선택들'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

08. 초저지연 스트리밍 - 코드 작성 원칙

빠르게 보이게 만드는 선택들



OCP 핵심

새 구현체를 추가해 확장하고, 이미 검증된 호출부는 되도록 건드리지 않는다.

그림 해석: 빠르게 보이게 만드는 선택들 - first frame, jitter, RTT, dropped frame, concealed samples를 본다.

코드 원칙

오디오/비디오 처리 옵션은 **strategy**로 분리해 저지연 모드와 품질 모드를 교체 가능하게 한다.

설계 연결

SRP는 책임을 나누고, OCP는 새 구현체를 붙이기 쉽게 만들며, DIP는 구체 구현 대신 **interface**를 바라보게 만든다.

작은 예시

문자열 **endpoint**를 컨트롤러에 직접 쓰지 말고 **RouteContract** 또는 **ApiRoutes** 같은 상수 객체에 모은다.

기억할 문장

- 초저지연 스트리밍은 '빠르게 보이게 만드는 선택들'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

08. 초저지연 스트리밍 - 테스트와 검증

빠르게 보이게 만드는 선택들

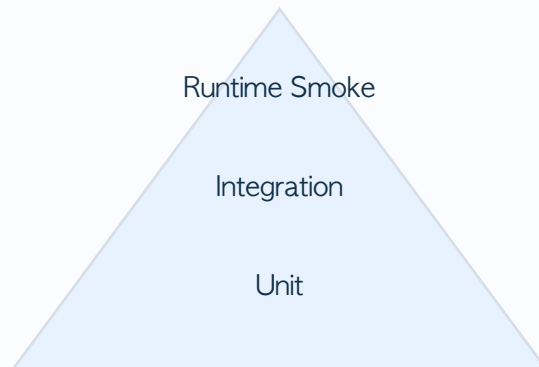


그림 해석: 빠르게 보이게 만드는 선택들 - first frame, jitter, RTT, dropped frame, concealed samples를 본다.

테스트 원칙

오디오 jitter, packet loss, first-frame latency를 자동 테스트와 smoke에서 수집한다.

TDD 관점

먼저 실패하는 계약 테스트를 만들고, 그 계약을 만족하는 가장 작은 구현을 작성한 뒤, 실제 runtime smoke로 연결을 확인한다.

통과 기준

first frame, jitter, RTT, dropped frame, concealed samples를 본다.가
기대 범위 안에 있고, 실패 시 원인이 로그와 테스트 이름으로 드러나야 한다.

기억할 문장

- 초저지연 스트리밍은 '빠르게 보이게 만드는 선택들'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

08. 초저지연 스트리밍 - 운영 체크

빠르게 보이게 만드는 선택들

이 페이지의 핵심

관제 시스템은 몇 초 늦은 영상보다 현재에 가까운 영상이 중요하다.

운영 원칙

지연을 줄일 때 CPU, 네트워크, 음질, 안정성의 trade-off를 기록한다.

장애 대응

status code, container health, application log, network path,
secret/config 누락을 순서대로 확인한다.

기록할 것

명령어, 실행 시각, 결과, 실패 원인, 수정 파일, 남은 리스크를 보고서에 남긴다.

기억할 문장

- 초저지연 스트리밍은 '빠르게 보이게 만드는 선택들'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

08. 초저지연 스트리밍 - 자주 나는 실수

빠르게 보이게 만드는 선택들

이 페이지의 핵심

관제 시스템은 몇 초 늦은 영상보다 현재에 가까운 영상이 중요하다.

자주 나는 실수

모든 필터를 켜면 보기 좋을 수 있지만 실제 상황 판단은 늦어질 수 있다.

왜 위험한가

초반에는 빠르게 보이지만, 운영 중 장애 원인을 숨기거나 변경 범위를 키워 결국 더 오래 걸리게 만든다.

예방 습관

계약값 중앙화, DTO 분리, 테스트 추가, 로그/지표 확인을 PR 기준으로 둔다.

기억할 문장

- 초저지연 스트리밍은 '빠르게 보이게 만드는 선택들'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

08. 초저지연 스트리밍 - 초보자 실습

빠르게 보이게 만드는 선택들

이 페이지의 핵심

관제 시스템은 몇 초 늦은 영상보다 현재에 가까운 영상이 중요하다.

따라 해보기

저지연 오디오 모드와 음질 모드를 바꿔 실제 차이를 느껴 본다.

관찰하기

작업 전후로 first frame, jitter, RTT, dropped frame, concealed samples를 본다.를 기록한다. 숫자를 모르면 개선인지 착각인지 구분할 수 없다.

질문 만들기

'초저지연 스트리밍이 깨지면 사용자는 무엇을 보게 되는가?'를 한 문장으로 답해 본다.

기억할 문장

- 초저지연 스트리밍은 '빠르게 보이게 만드는 선택들'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

08. 초저지연 스트리밍 - 복습 체크리스트

빠르게 보이게 만드는 선택들

이 페이지의 핵심

관제 시스템은 몇 초 늦은 영상보다 현재에 가까운 영상이 중요하다.

복습 질문

초저지연 스트리밍은 control plane, media plane, data plane, ops plane 중 어디에 가까운가?

체크리스트

경로가 상수화되어 있는가? DTO가 분리되어 있는가? 실패 테스트가 있는가? 운영 지표가 있는가? secret이 노출되지 않았는가?

다음 학습

관련 코드 파일 하나와 테스트 파일 하나를 함께 열어 구현과 검증이 어떻게 연결되는지 읽는다.

기억할 문장

- 초저지연 스트리밍은 '빠르게 보이게 만드는 선택들'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

09. 인증, 인가, JWT 보안 - 개념 지도

누가 무엇을 볼 수 있는가

이 페이지의 핵심

실시간 관제 데이터는 민감하므로 로그인뿐 아니라 어떤 그룹의 스트림을 볼 수 있는지까지 관리해야 한다.

왜 필요한가

실시간 관제 데이터는 민감하므로 로그인뿐 아니라 어떤 그룹의 스트림을 볼 수 있는지까지 관리해야 한다.

서버에서 맡는 역할

Spring/Kotlin auth-policy는 JWT 발급, refresh session, group policy, CSRF 방어의 중심이다.

초보자 비유

출입증이 있어도 모든 방에 들어갈 수 없는 것과 같다.

기억할 문장

- 인증, 인가, JWT 보안은 '누가 무엇을 볼 수 있는가'를 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

09. 인증, 인가, JWT 보안 - 서버 적용 위치

누가 무엇을 볼 수 있는가

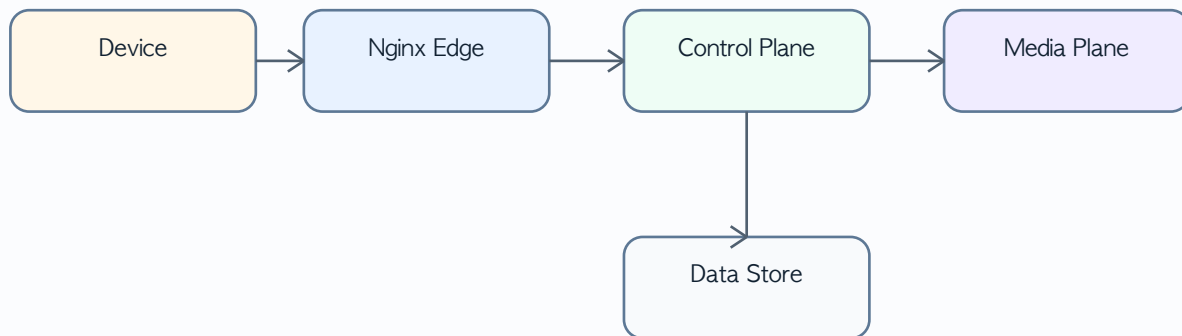


그림 해석: 누가 무엇을 볼 수 있는가 - 401/403 비율, refresh 성공률, session store latency, rejected origin을 본다.

서버 적용 위치

Spring/Kotlin auth-policy는 JWT 발급, refresh session, group policy, CSRF 방어에 중심이다.

연결해서 볼 것

인증, 인가, JWT 보안은 단독 기술이 아니라 Nginx, Docker Compose, 인증, 미디어 경로, 테스트와 함께 동작한다.

운영 지표

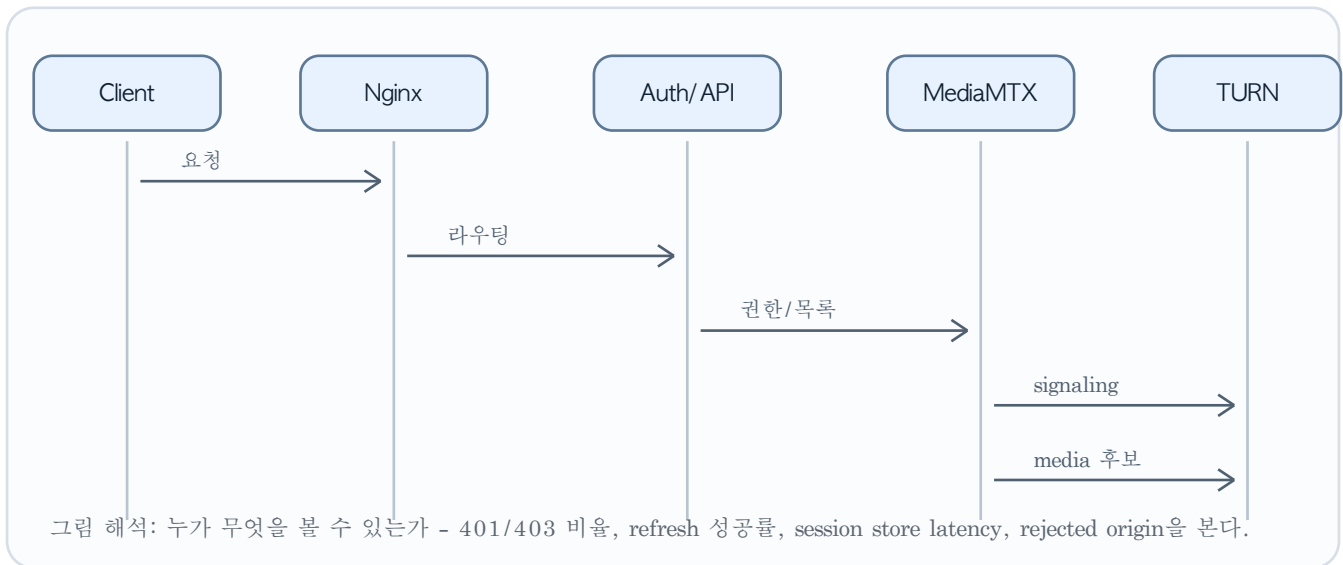
401/403 비율, refresh 성공률, session store latency, rejected origin을 본다.

기억할 문장

- 인증, 인가, JWT 보안은 '누가 무엇을 볼 수 있는가'를 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

09. 인증, 인가, JWT 보안 - 요청/데이터 흐름

누가 무엇을 볼 수 있는가



흐름 읽기

요청은 항상 입구, 정책 판단, 실제 처리, 응답의 순서로 본다. 인증, 인가, JWT 보안에서도 같은 순서로 로그를 따라가면 원인 파악이 빨라진다.

확인할 데이터

401/403 비율, refresh 성공률, session store latency, rejected origin을 본다.

다음 단계

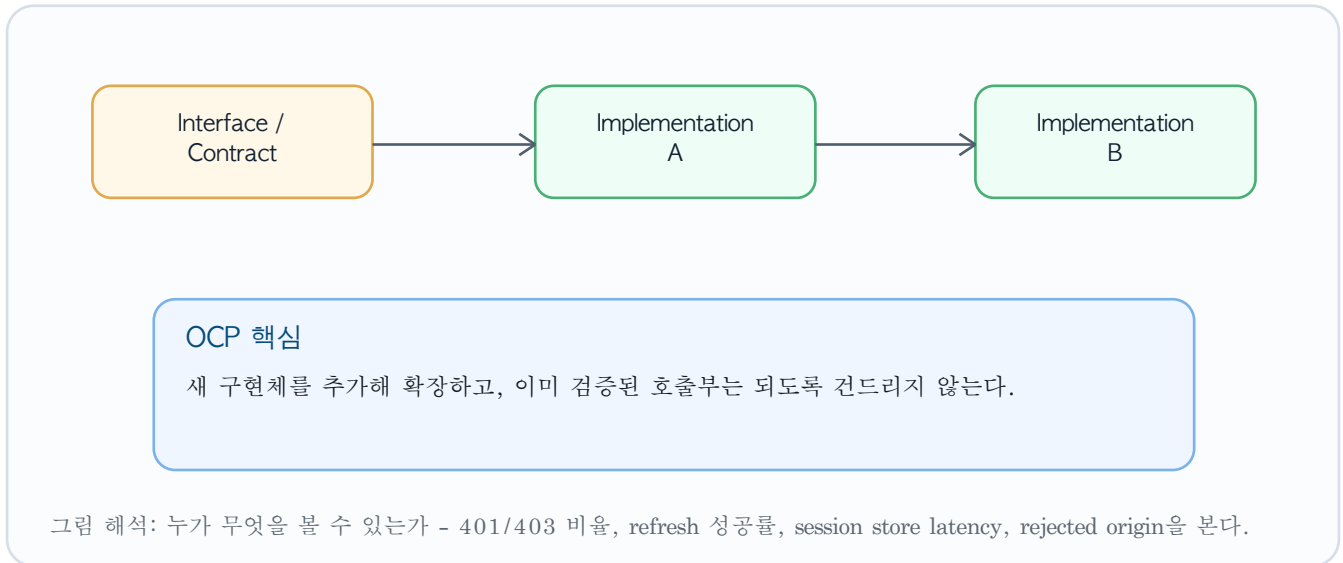
JWT payload와 signature의 차이를 직접 디코딩해서 확인한다.

기억할 문장

- 인증, 인가, JWT 보안은 '누가 무엇을 볼 수 있는가'를 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

09. 인증, 인가, JWT 보안 - 코드 작성 원칙

누가 무엇을 볼 수 있는가



코드 원칙

access token은 짧게, refresh token은 HttpOnly cookie와 Redis session으로 관리한다.

설계 연결

SRP는 책임을 나누고, OCP는 새 구현체를 붙이기 쉽게 만들며, DIP는 구체 구현 대신 interface를 바라보게 만든다.

작은 예시

문자열 endpoint를 컨트롤러에 직접 쓰지 말고 RouteContract 또는 ApiRoutes 같은 상수 객체에 모은다.

기억할 문장

- 인증, 인가, JWT 보안은 '누가 무엇을 볼 수 있는가'를 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

09. 인증, 인가, JWT 보안 - 테스트와 검증

누가 무엇을 볼 수 있는가

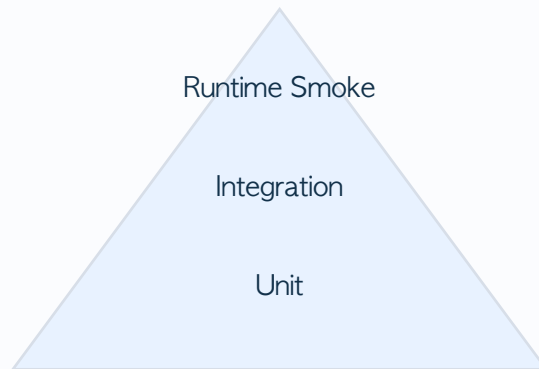


그림 해석: 누가 무엇을 볼 수 있는가 - 401/403 비율, refresh 성공률, session store latency, rejected origin을 본다.

테스트 원칙

로그인, refresh, revoke, unauthorized route, CSRF rejection test를 둔다.

TDD 관점

먼저 실패하는 계약 테스트를 만들고, 그 계약을 만족하는 가장 작은 구현을 작성한 뒤, 실제 runtime smoke로 연결을 확인한다.

통과 기준

401/403 비율, refresh 성공률, session store latency, rejected origin을 본다.가 기대 범위 안에 있고, 실패 시 원인이 로그와 테스트 이름으로 드러나야 한다.

기억할 문장

- 인증, 인가, JWT 보안은 '누가 무엇을 볼 수 있는가'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

09. 인증, 인가, JWT 보안 - 운영 체크

누가 무엇을 볼 수 있는가

이 페이지의 핵심

실시간 관제 데이터는 민감하므로 로그인뿐 아니라 어떤 그룹의 스트림을 볼 수 있는지까지 관리해야 한다.

운영 원칙

HTTPS, CORS, CSP, Origin/Referer 검증, cookie SameSite를 함께 적용한다.

장애 대응

status code, container health, application log, network path,
secret/config 누락을 순서대로 확인한다.

기록할 것

명령어, 실행 시각, 결과, 실패 원인, 수정 파일, 남은 리스크를 보고서에 남긴다.

기억할 문장

- 인증, 인가, JWT 보안은 '누가 무엇을 볼 수 있는가'를 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

09. 인증, 인가, JWT 보안 - 자주 나는 실수

누가 무엇을 볼 수 있는가

이 페이지의 핵심

실시간 관제 데이터는 민감하므로 로그인뿐 아니라 어떤 그룹의 스트림을 볼 수 있는지까지 관리해야 한다.

자주 나는 실수

localStorage에 token을 넣으면 XSS 때 탈취 위험이 커진다.

왜 위험한가

초반에는 빠르게 보이지만, 운영 중 장애 원인을 숨기거나 변경 범위를 키워 결국 더 오래 걸리게 만든다.

예방 습관

계약값 중앙화, DTO 분리, 테스트 추가, 로그/지표 확인을 PR 기준으로 둔다.

기억할 문장

- 인증, 인가, JWT 보안은 '누가 무엇을 볼 수 있는가'를 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

09. 인증, 인가, JWT 보안 - 초보자 실습

누가 무엇을 볼 수 있는가

이 페이지의 핵심

실시간 관제 데이터는 민감하므로 로그인뿐 아니라 어떤 그룹의 스트림을 볼 수 있는지까지 관리해야 한다.

따라 해보기

JWT payload와 signature의 차이를 직접 디코딩해서 확인한다.

관찰하기

작업 전후로 401/403 비율, refresh 성공률, session store latency, rejected origin을 본다.를 기록한다. 숫자를 모르면 개선인지 착각인지 구분할 수 없다.

질문 만들기

'인증, 인가, JWT 보안이 깨지면 사용자는 무엇을 보게 되는가?'를 한 문장으로 답해 본다.

기억할 문장

- 인증, 인가, JWT 보안은 '누가 무엇을 볼 수 있는가'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

09. 인증, 인가, JWT 보안 - 복습 체크리스트

누가 무엇을 볼 수 있는가

이 페이지의 핵심

실시간 관제 데이터는 민감하므로 로그인뿐 아니라 어떤 그룹의 스트림을 볼 수 있는지까지 관리해야 한다.

복습 질문

인증, 인가, JWT 보안은 control plane, media plane, data plane, ops plane
중 어디에 가까운가?

체크리스트

경로가 상수화되어 있는가? DTO가 분리되어 있는가? 실패 테스트가 있는가? 운영 지표가 있는가? secret이 노출되지 않았는가?

다음 학습

관련 코드 파일 하나와 테스트 파일 하나를 함께 열어 구현과 검증이 어떻게 연결되는지 읽는다.

기억할 문장

- 인증, 인가, JWT 보안은 '누가 무엇을 볼 수 있는가'를 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

10. Spring Boot와 Kotlin Auth-Policy - 개념 지도

엄격한 정책 서버

이 페이지의 핵심

인증/인가와 시간 동기화처럼 정확성이 중요한 영역은 타입과 테스트가 강한 구조가 유리하다.

왜 필요한가

인증/인가와 시간 동기화처럼 정확성이 중요한 영역은 타입과 테스트가 강한 구조가 유리하다.

서버에서 맡는 역할

Kotlin auth-policy는 route contract, DTO, domain model, service, repository를 분리한다.

초보자 비유

규칙이 많은 출입 관리 사무소를 코드로 만든 것과 비슷하다.

기억할 문장

- Spring Boot와 Kotlin Auth-Policy은 '엄격한 정책 서버'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

10. Spring Boot와 Kotlin Auth-Policy - 서버 적용 위치

엄격한 정책 서버

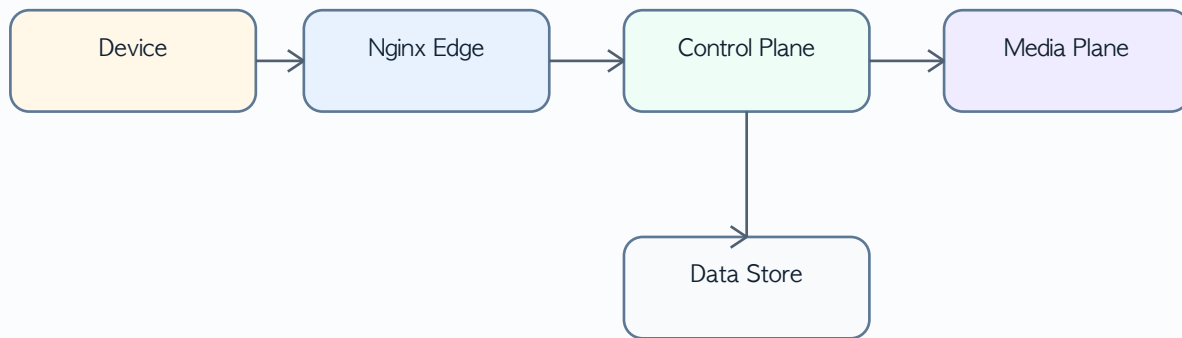


그림 해석: 엄격한 정책 서버 - auth latency, Redis latency, token error, ready state를 본다.

서버 적용 위치

Kotlin auth-policy는 route contract, DTO, domain model, service, repository를 분리한다.

연결해서 볼 것

Spring Boot와 Kotlin Auth-Policy은 단독 기술이 아니라 Nginx, Docker Compose, 인증, 미디어 경로, 테스트와 함께 동작한다.

운영 지표

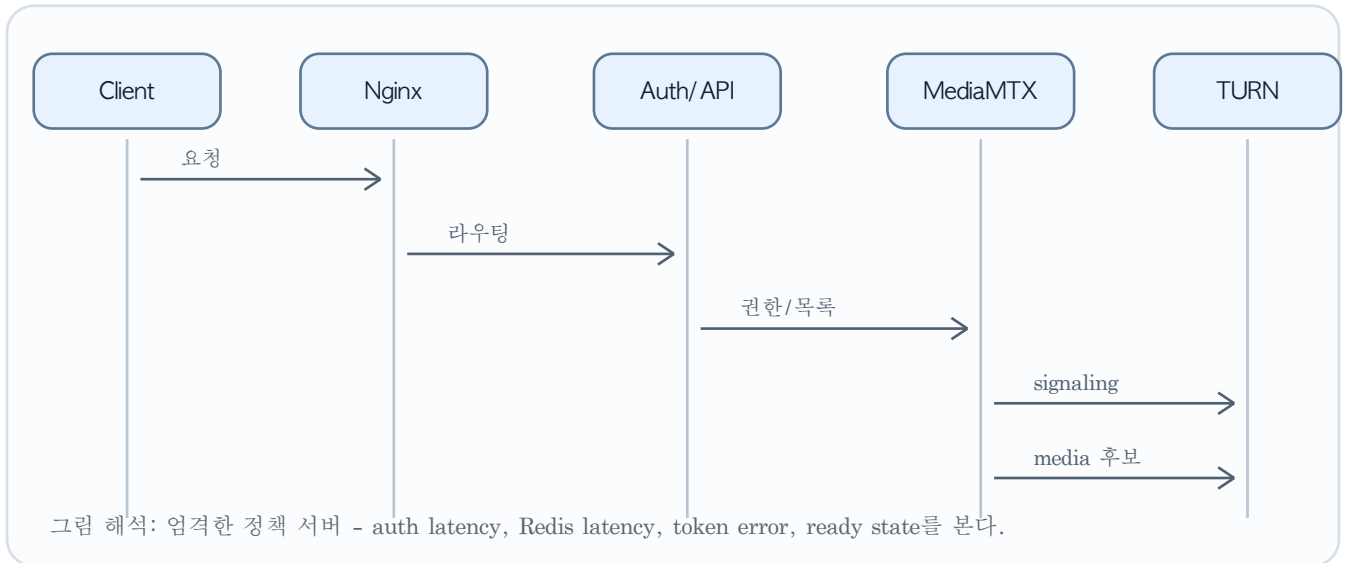
auth latency, Redis latency, token error, ready state를 본다.

기억할 문장

- Spring Boot와 Kotlin Auth-Policy은 '엄격한 정책 서버'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

10. Spring Boot와 Kotlin Auth-Policy - 요청/데이터 흐름

엄격한 정책 서버



흐름 읽기

요청은 항상 입구, 정책 판단, 실제 처리, 응답의 순서로 본다. Spring Boot와 Kotlin Auth-Policy에서도 같은 순서로 로그를 따라가면 원인 파악이 빨라진다.

확인할 데이터

auth latency, Redis latency, token error, ready state를 본다.

다음 단계

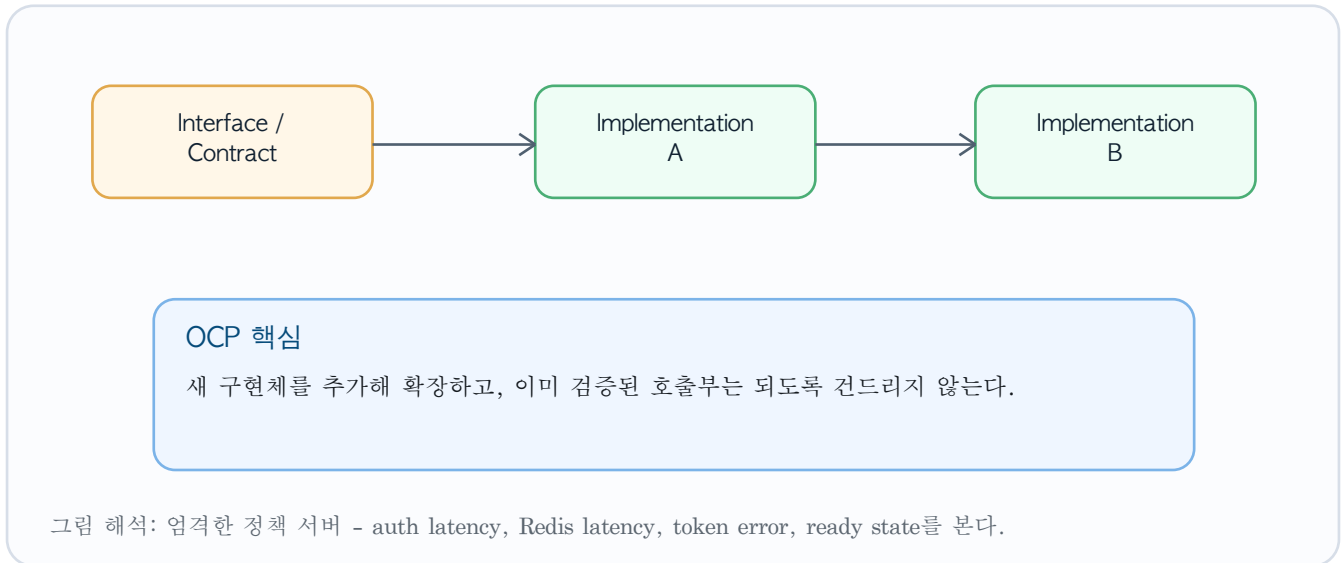
Controller, Service, Repository 파일을 나눠 읽으며 책임을 표시해 본다.

기억할 문장

- Spring Boot와 Kotlin Auth-Policy은 '엄격한 정책 서버'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

10. Spring Boot와 Kotlin Auth-Policy - 코드 작성 원칙

엄격한 정책 서버



코드 원칙

val, data class, require, interface, factory method로 불변성과 계약을 강화한다.

설계 연결

SRP는 책임을 나누고, OCP는 새 구현체를 붙이기 쉽게 만들며, DIP는 구체 구현 대신 interface를 바라보게 만든다.

작은 예시

문자열 endpoint를 컨트롤러에 직접 쓰지 말고 RouteContract 또는 ApiRoutes 같은 상수 객체에 모은다.

기억할 문장

- Spring Boot와 Kotlin Auth-Policy은 '엄격한 정책 서버'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

10. Spring Boot와 Kotlin Auth-Policy - 테스트와 검증

엄격한 정책 서버

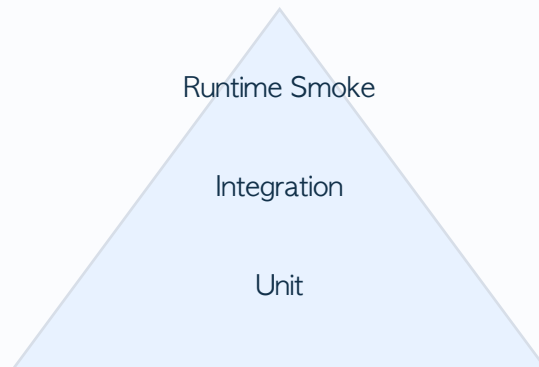


그림 해석: 엄격한 정책 서버 - auth latency, Redis latency, token error, ready state를 본다.

테스트 원칙

Gradle test, Jacoco, route contract test, concurrency test로 회귀를 막는다.

TDD 관점

먼저 실패하는 계약 테스트를 만들고, 그 계약을 만족하는 가장 작은 구현을 작성한 뒤, 실제 runtime smoke로 연결을 확인한다.

통과 기준

auth latency, Redis latency, token error, ready state를 본다.가 기대 범위 안에 있고, 실패 시 원인이 로그와 테스트 이름으로 드러나야 한다.

기억할 문장

- Spring Boot와 Kotlin Auth-Policy은 '엄격한 정책 서버'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

10. Spring Boot와 Kotlin Auth-Policy - 운영 체크

엄격한 정책 서버

이 페이지의 핵심

인증/인가와 시간 동기화처럼 정확성이 중요한 영역은 타입과 테스트가 강한 구조가 유리하다.

운영 원칙

운영에서는 /healthz, /readyz, Redis 연결, JWT secret 주입을 반드시 확인한다.

장애 대응

status code, container health, application log, network path,
secret/config 누락을 순서대로 확인한다.

기록할 것

명령어, 실행 시각, 결과, 실패 원인, 수정 파일, 남은 리스크를 보고서에 남긴다.

기억할 문장

- Spring Boot와 Kotlin Auth-Policy은 '엄격한 정책 서버'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

10. Spring Boot와 Kotlin Auth-Policy - 자주 나는 실수

엄격한 정책 서버

이 페이지의 핵심

인증/인가와 시간 동기화처럼 정확성이 중요한 영역은 타입과 테스트가 강한 구조가 유리하다.

자주 나는 실수

컨트롤러에 문자열 endpoint와 비즈니스 로직을 같이 넣으면 유지보수가 어려워진다.

왜 위험한가

초반에는 빠르게 보이지만, 운영 중 장애 원인을 숨기거나 변경 범위를 키워 결국 더 오래 걸리게 만든다.

예방 습관

계약값 중앙화, DTO 분리, 테스트 추가, 로그/지표 확인을 PR 기준으로 둔다.

기억할 문장

- Spring Boot와 Kotlin Auth-Policy은 '엄격한 정책 서버'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

10. Spring Boot와 Kotlin Auth-Policy - 초보자 실습

엄격한 정책 서버

이 페이지의 핵심

인증/인가와 시간 동기화처럼 정확성이 중요한 영역은 타입과 테스트가 강한 구조가 유리하다.

따라 해보기

Controller, Service, Repository 파일을 나눠 읽으며 책임을 표시해 본다.

관찰하기

작업 전후로 auth latency, Redis latency, token error, ready state를 본다.를 기록한다. 숫자를 모르면 개선인지 착각인지 구분할 수 없다.

질문 만들기

'Spring Boot와 Kotlin Auth-Policy이 깨지면 사용자는 무엇을 보게 되는가?'를 한 문장으로 답해 본다.

기억할 문장

- Spring Boot와 Kotlin Auth-Policy은 '엄격한 정책 서버'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

10. Spring Boot와 Kotlin Auth-Policy - 복습 체크리스트

엄격한 정책 서버

이 페이지의 핵심

인증/인가와 시간 동기화처럼 정확성이 중요한 영역은 타입과 테스트가 강한 구조가 유리하다.

복습 질문

Spring Boot와 Kotlin Auth-Policy은 control plane, media plane, data plane, ops plane 중 어디에 가까운가?

체크리스트

경로가 상수화되어 있는가? DTO가 분리되어 있는가? 실패 테스트가 있는가? 운영 지표가 있는가? secret이 노출되지 않았는가?

다음 학습

관련 코드 파일 하나와 테스트 파일 하나를 함께 열어 구현과 검증이 어떻게 연결되는지 읽는다.

기억할 문장

- Spring Boot와 Kotlin Auth-Policy은 '엄격한 정책 서버'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

11. Go Media-Control - 개념 지도

빠른 스트림 제어 API

이 페이지의 핵심

스트림 목록과 ICE 서버 제공은 빠르고 단순해야 하므로 Go 서비스가 잘 맞는다.

왜 필요한가

스트림 목록과 ICE 서버 제공은 빠르고 단순해야 하므로 Go 서비스가 잘 맞는다.

서버에서 맡는 역할

media-control은 MediaMTX 상태, stream playback URL, ICE server list, cache를 제공한다.

초보자 비유

교통 안내판처럼 지금 갈 수 있는 스트림과 경로를 빠르게 알려준다.

기억할 문장

- Go Media-Control은 '빠른 스트림 제어 API'를 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

11. Go Media-Control - 서버 적용 위치

빠른 스트림 제어 API

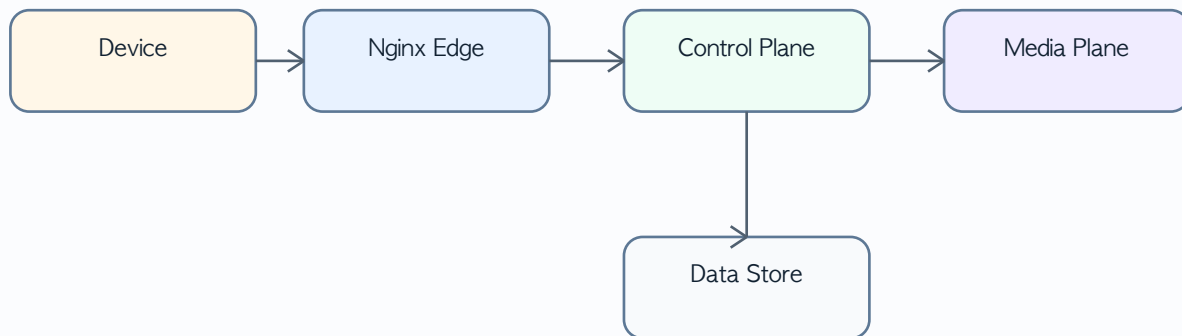


그림 해석: 빠른 스트림 제어 API - stream list latency, cache hit ratio, presence TTL, MediaMTX API latency를 본다.

서버 적용 위치

media-control은 MediaMTX 상태, stream playback URL, ICE server list, cache를 제공한다.

연결해서 볼 것

Go Media-Control은 단독 기술이 아니라 Nginx, Docker Compose, 인증, 미디어 경로, 테스트와 함께 동작한다.

운영 지표

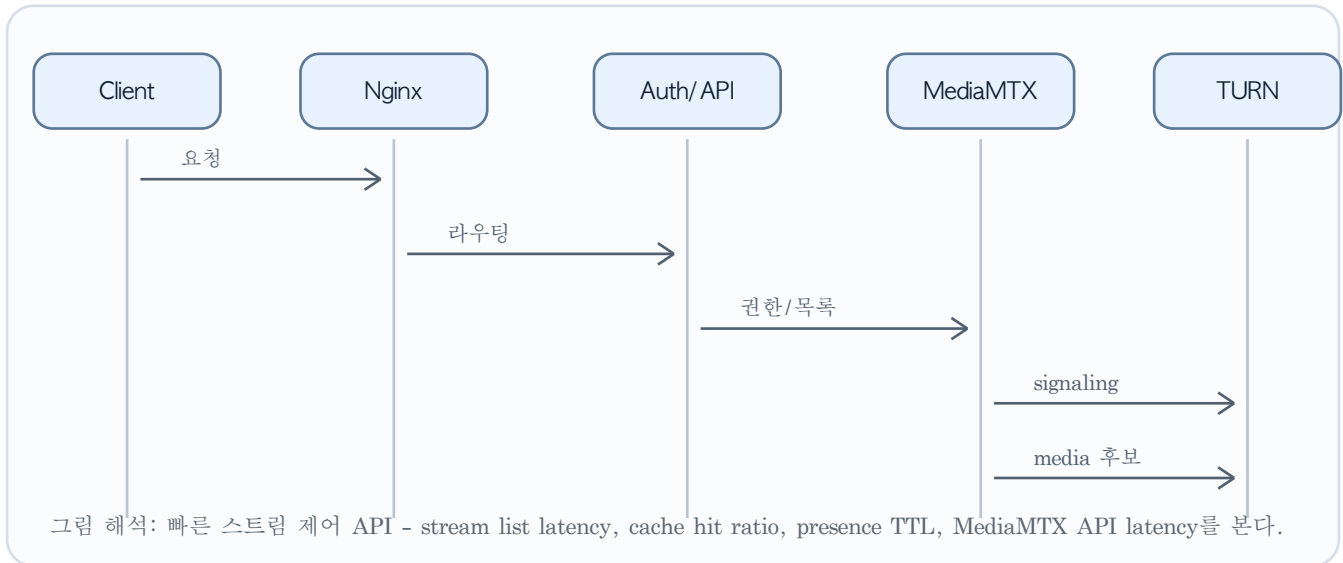
stream list latency, cache hit ratio, presence TTL, MediaMTX API latency를 본다.

기억할 문장

- Go Media-Control은 '빠른 스트림 제어 API'를 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

11. Go Media-Control - 요청/데이터 흐름

빠른 스트림 제어 API



흐름 읽기

요청은 항상 입구, 정책 판단, 실제 처리, 응답의 순서로 본다. Go Media-Control에서도 같은 순서로 로그를 따라가면 원인 파악이 빨라진다.

확인할 데이터

stream list latency, cache hit ratio, presence TTL, MediaMTX API latency를 본다.

다음 단계

Go interface 한 개와 구현체 두 개를 만들어 OCP를 연습한다.

기억할 문장

- Go Media-Control은 '빠른 스트림 제어 API'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

11. Go Media-Control - 코드 작성 원칙

빠른 스트림 제어 API



OCP 핵심

새 구현체를 추가해 확장하고, 이미 검증된 호출부는 되도록 건드리지 않는다.

그림 해석: 빠른 스트림 제어 API - stream list latency, cache hit ratio, presence TTL, MediaMTX API latency를 본다.

코드 원칙

interface와 adapter로 MediaMTX, auth-policy, Redis 의존을 감싼다.

설계 연결

SRP는 책임을 나누고, OCP는 새 구현체를 붙이기 쉽게 만들며, DIP는 구체 구현 대신 interface를 바라보게 만든다.

작은 예시

문자열 endpoint를 컨트롤러에 직접 쓰지 말고 RouteContract 또는 ApiRoutes 같은 상수 객체에 모은다.

기억할 문장

- Go Media-Control은 '빠른 스트림 제어 API'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

11. Go Media-Control - 테스트와 검증

빠른 스트림 제어 API

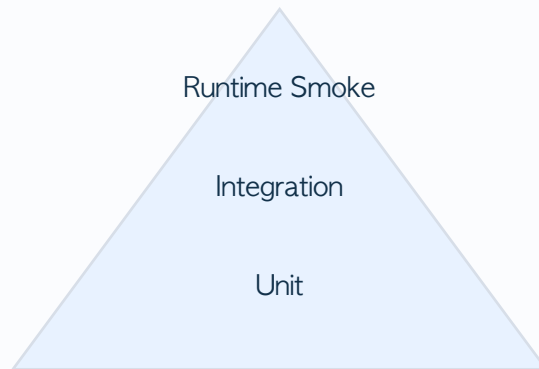


그림 해석: 빠른 스트림 제어 API - stream list latency, cache hit ratio, presence TTL, MediaMTX API latency를 본다.

테스트 원칙

go test와 HTTP API contract test로 JSON field와 error를 고정한다.

TDD 관점

먼저 실패하는 계약 테스트를 만들고, 그 계약을 만족하는 가장 작은 구현을 작성한 뒤, 실제 runtime smoke로 연결을 확인한다.

통과 기준

stream list latency, cache hit ratio, presence TTL, MediaMTX API latency를 본다.가 기대 범위 안에 있고, 실패 시 원인이 로그와 테스트 이름으로 드러나야 한다.

기억할 문장

- Go Media-Control은 '빠른 스트림 제어 API'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

11. Go Media-Control - 운영 체크

빠른 스트림 제어 API

이 페이지의 핵심

스트림 목록과 ICE 서버 제공은 빠르고 단순해야 하므로 Go 서비스가 잘 맞는다.

운영 원칙

Redis cache TTL을 짧게 뒤 최신성과 속도 사이 균형을 잡는다.

장애 대응

status code, container health, application log, network path, secret/config 누락을 순서대로 확인한다.

기록할 것

명령어, 실행 시각, 결과, 실패 원인, 수정 파일, 남은 리스크를 보고서에 남긴다.

기억할 문장

- Go Media-Control은 '빠른 스트림 제어 API'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

11. Go Media-Control - 자주 나는 실수

빠른 스트림 제어 API

이 페이지의 핵심

스트림 목록과 ICE 서버 제공은 빠르고 단순해야 하므로 Go 서비스가 잘 맞는다.

자주 나는 실수

cache TTL이 너무 길면 끊긴 스트림이 계속 살아 있는 것처럼 보일 수 있다.

왜 위험한가

초반에는 빠르게 보이지만, 운영 중 장애 원인을 숨기거나 변경 범위를 키워 결국 더 오래 걸리게 만든다.

예방 습관

계약값 중앙화, DTO 분리, 테스트 추가, 로그/지표 확인을 PR 기준으로 둔다.

기억할 문장

- Go Media-Control은 '빠른 스트림 제어 API'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

11. Go Media-Control - 초보자 실습

빠른 스트림 제어 API

이 페이지의 핵심

스트림 목록과 ICE 서버 제공은 빠르고 단순해야 하므로 Go 서비스가 잘 맞는다.

따라 해보기

Go interface 한 개와 구현체 두 개를 만들어 OCP를 연습한다.

관찰하기

작업 전후로 stream list latency, cache hit ratio, presence TTL, MediaMTX API latency를 본다.를 기록한다. 숫자를 모르면 개선인지 착각인지 구분할 수 없다.

질문 만들기

'Go Media-Control이 깨지면 사용자는 무엇을 보게 되는가?'를 한 문장으로 답해 본다.

기억할 문장

- Go Media-Control은 '빠른 스트림 제어 API'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

11. Go Media-Control - 복습 체크리스트

빠른 스트림 제어 API

이 페이지의 핵심

스트림 목록과 ICE 서버 제공은 빠르고 단순해야 하므로 Go 서비스가 잘 맞는다.

복습 질문

Go Media-Control은 control plane, media plane, data plane, ops plane 중 어디에 가까운가?

체크리스트

경로가 상수화되어 있는가? DTO가 분리되어 있는가? 실패 테스트가 있는가? 운영 지표가 있는가? secret이 노출되지 않았는가?

다음 학습

관련 코드 파일 하나와 테스트 파일 하나를 함께 열어 구현과 검증이 어떻게 연결되는지 읽는다.

기억할 문장

- Go Media-Control은 '빠른 스트림 제어 API'를 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

12. Python FastAPI Legacy Bridge - 개념 지도

기존 기능을 안전하게 이어가기

이 페이지의 핵심

언어 이전 중에도 기존 API와 DB를 바로 버리면 운영 안정성이 무너질 수 있다.

왜 필요한가

언어 이전 중에도 기존 API와 DB를 바로 버리면 운영 안정성이 무너질 수 있다.

서버에서 맡는 역할

FastAPI backend는 legacy API, MySQL schema, MQTT 연동을 점진적으로 이어주는 다리다.

초보자 비유

공사 중 임시 우회도로처럼 기존 기능을 유지하며 새 길을 만든다.

기억할 문장

- Python FastAPI Legacy Bridge은 '기존 기능을 안전하게 이어가기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

12. Python FastAPI Legacy Bridge - 서버 적용 위치

기존 기능을 안전하게 이어가기

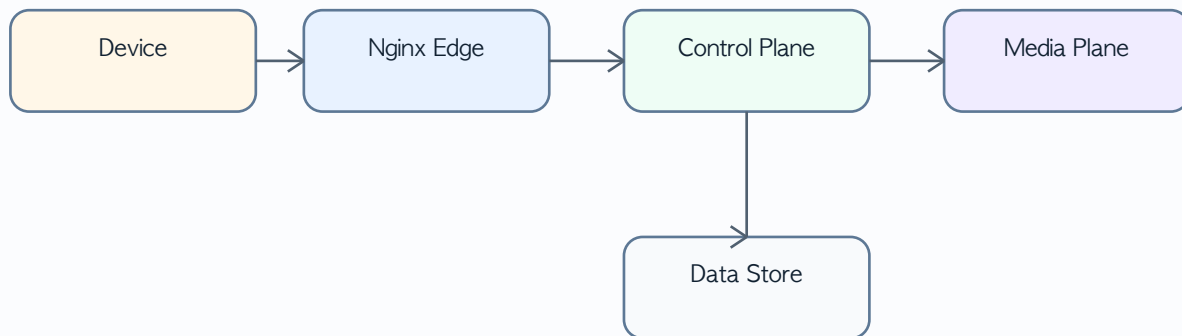


그림 해석: 기존 기능을 안전하게 이어가기 - test coverage, deprecated route call count, 500 error count를 본다.

서버 적용 위치

FastAPI backend는 legacy API, MySQL schema, MQTT 연동을 점진적으로 이어주는 다리다.

연결해서 볼 것

Python FastAPI Legacy Bridge은 단독 기술이 아니라 Nginx, Docker Compose, 인증, 미디어 경로, 테스트와 함께 동작한다.

운영 지표

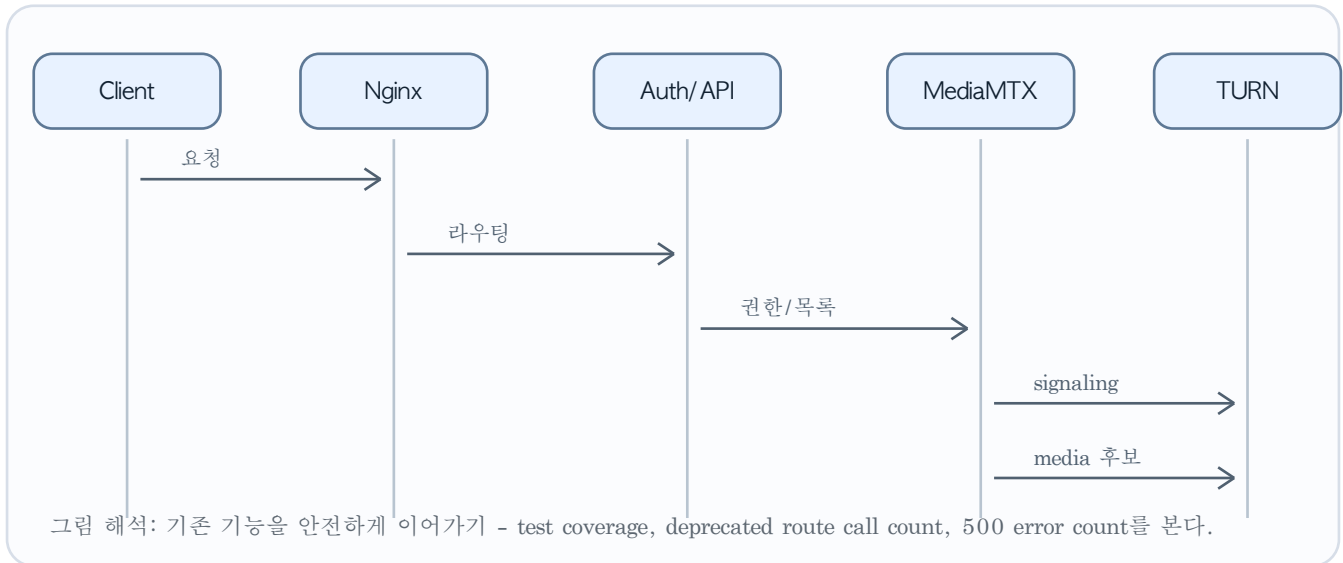
test coverage, deprecated route call count, 500 error count를 본다.

기억할 문장

- Python FastAPI Legacy Bridge은 '기존 기능을 안전하게 이어가기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

12. Python FastAPI Legacy Bridge - 요청/데이터 흐름

기존 기능을 안전하게 이어가기



흐름 읽기

요청은 항상 입구, 정책 판단, 실제 처리, 응답의 순서로 본다. Python FastAPI Legacy Bridge에서도 같은 순서로 로그를 따라가면 원인 파악이 빨라진다.

확인할 데이터

test coverage, deprecated route call count, 500 error count를 본다.

다음 단계

Pydantic DTO와 SQLAlchemy model의 차이를 비교해 본다.

기억할 문장

- Python FastAPI Legacy Bridge은 '기존 기능을 안전하게 이어가기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

12. Python FastAPI Legacy Bridge - 코드 작성 원칙

기존 기능을 안전하게 이어가기



OCP 핵심

새 구현체를 추가해 확장하고, 이미 검증된 호출부는 되도록 건드리지 않는다.

그림 해석: 기존 기능을 안전하게 이어가기 - test coverage, deprecated route call count, 500 error count를 본다.

코드 원칙

Python에서도 Pydantic model, type hint, mypy, DTO 분리로 런타임 오류를 줄인다.

설계 연결

SRP는 책임을 나누고, OCP는 새 구현체를 붙이기 쉽게 만들며, DIP는 구체 구현 대신 interface를 바라보게 만든다.

작은 예시

문자열 endpoint를 컨트롤러에 직접 쓰지 말고 RouteContract 또는 ApiRoutes 같은 상수 객체에 모은다.

기억할 문장

- Python FastAPI Legacy Bridge은 '기존 기능을 안전하게 이어가기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

12. Python FastAPI Legacy Bridge - 테스트와 검증

기존 기능을 안전하게 이어가기

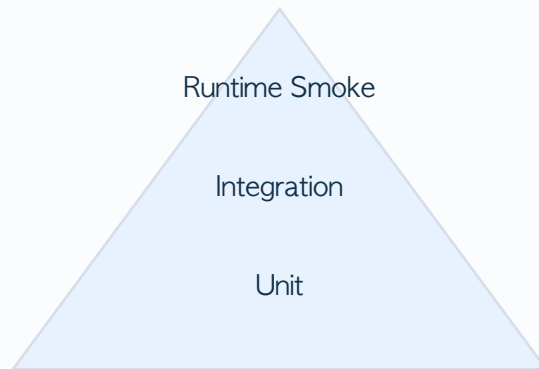


그림 해석: 기존 기능을 안전하게 이어가기 - test coverage, deprecated route call count, 500 error count를 본다.

테스트 원칙

pytest, coverage, mypy, integration smoke로 기존 기능 parity를 확인한다.

TDD 관점

먼저 실패하는 계약 테스트를 만들고, 그 계약을 만족하는 가장 작은 구현을 작성한 뒤, 실제 runtime smoke로 연결을 확인한다.

통과 기준

test coverage, deprecated route call count, 500 error count를 본다.가
기대 범위 안에 있고, 실패 시 원인이 로그와 테스트 이름으로 드러나야 한다.

기억할 문장

- Python FastAPI Legacy Bridge은 '기존 기능을 안전하게 이어가기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

12. Python FastAPI Legacy Bridge - 운영 체크

기존 기능을 안전하게 이어가기

이 페이지의 핵심

언어 이전 중에도 기존 API와 DB를 바로 버리면 운영 안정성이 무너질 수 있다.

운영 원칙

legacy route에는 deprecation header와 replacement route를 명시한다.

장애 대응

status code, container health, application log, network path,
secret/config 누락을 순서대로 확인한다.

기록할 것

명령어, 실행 시각, 결과, 실패 원인, 수정 파일, 남은 리스크를 보고서에 남긴다.

기억할 문장

- Python FastAPI Legacy Bridge은 '기존 기능을 안전하게 이어가기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

12. Python FastAPI Legacy Bridge - 자주 나는 실수

기존 기능을 안전하게 이어가기

이 페이지의 핵심

언어 이전 중에도 기존 API와 DB를 바로 버리면 운영 안정성이 무너질 수 있다.

자주 나는 실수

임시 코드가 문서 없이 오래 남으면 진짜 구조처럼 굳어질 수 있다.

왜 위험한가

초반에는 빠르게 보이지만, 운영 중 장애 원인을 숨기거나 변경 범위를 키워 결국 더 오래 걸리게 만든다.

예방 습관

계약값 중앙화, DTO 분리, 테스트 추가, 로그/지표 확인을 PR 기준으로 둔다.

기억할 문장

- Python FastAPI Legacy Bridge은 '기존 기능을 안전하게 이어가기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

12. Python FastAPI Legacy Bridge - 초보자 실습

기존 기능을 안전하게 이어가기

이 페이지의 핵심

언어 이전 중에도 기존 API와 DB를 바로 버리면 운영 안정성이 무너질 수 있다.

따라 해보기

Pydantic DTO와 SQLAlchemy model의 차이를 비교해 본다.

관찰하기

작업 전후로 test coverage, deprecated route call count, 500 error count를 본다.를 기록한다. 숫자를 모르면 개선인지 착각인지 구분할 수 없다.

질문 만들기

'Python FastAPI Legacy Bridge이 깨지면 사용자는 무엇을 보게 되는가?'를 한 문장으로 답해 본다.

기억할 문장

- Python FastAPI Legacy Bridge은 '기존 기능을 안전하게 이어가기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

12. Python FastAPI Legacy Bridge - 복습 체크리스트

기존 기능을 안전하게 이어가기

이 페이지의 핵심

언어 이전 중에도 기존 API와 DB를 바로 버리면 운영 안정성이 무너질 수 있다.

복습 질문

Python FastAPI Legacy Bridge은 control plane, media plane, data plane, ops plane 중 어디에 가까운가?

체크리스트

경로가 상수화되어 있는가? DTO가 분리되어 있는가? 실패 테스트가 있는가? 운영 지표가 있는가? secret이 노출되지 않았는가?

다음 학습

관련 코드 파일 하나와 테스트 파일 하나를 함께 열어 구현과 검증이 어떻게 연결되는지 읽는다.

기억할 문장

- Python FastAPI Legacy Bridge은 '기존 기능을 안전하게 이어가기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

13. MySQL과 정형 데이터 - 개념 지도

오래 보관할 장부

이 페이지의 핵심

사용자, 회사, 권한 같은 정형 데이터는 안정적인 영구 저장소가 필요하다.

왜 필요한가

사용자, 회사, 권한 같은 정형 데이터는 안정적인 영구 저장소가 필요하다.

서버에서 맡는 역할

MySQL은 legacy schema와 사용자/회사 데이터의 중심 저장소로 쓰인다.

초보자 비유

엑셀 장부보다 훨씬 엄격하고 동시에 여러 사람이 읽고 쓸 수 있는 장부다.

기억할 문장

- MySQL과 정형 데이터는 '오래 보관할 장부'를 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

13. MySQL과 정형 데이터 - 서버 적용 위치

오래 보관할 장부

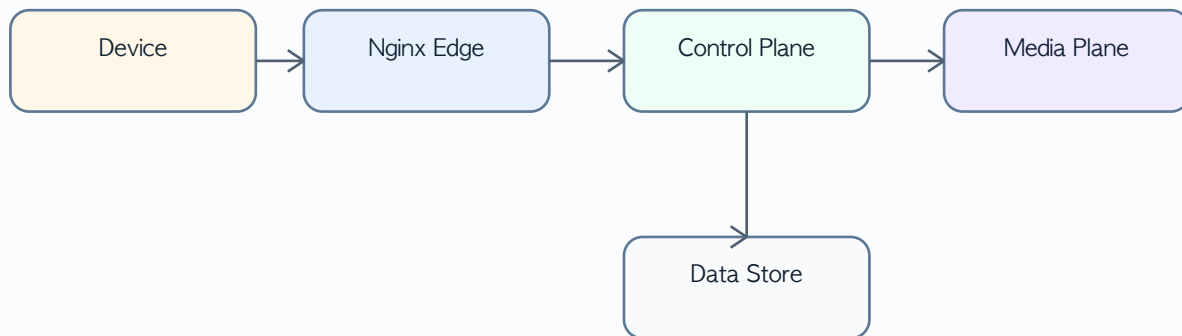


그림 해석: 오래 보관할 장부 - rows examined, full scan, index hit, lock wait, slow query를 본다.

서버 적용 위치

MySQL은 legacy schema와 사용자/회사 데이터의 중심 저장소로 쓰인다.

연결해서 볼 것

MySQL과 정형 데이터는 단독 기술이 아니라 Nginx, Docker Compose, 인증, 미디어 경로, 테스트와 함께 동작한다.

운영 지표

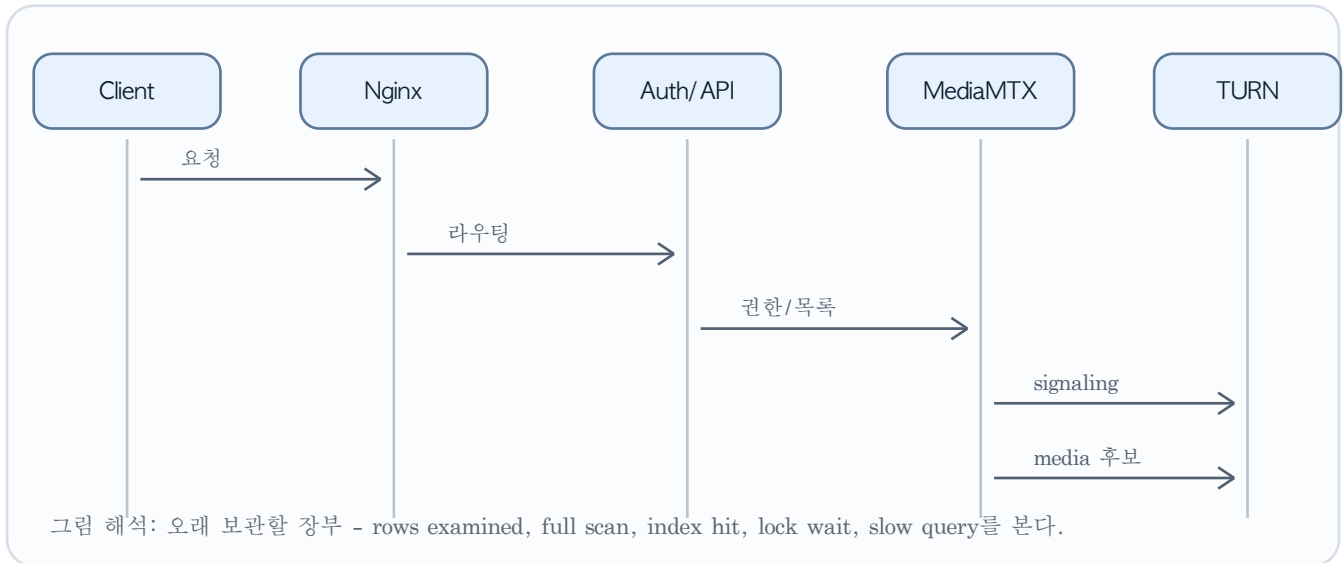
rows examined, full scan, index hit, lock wait, slow query를 본다.

기억할 문장

- MySQL과 정형 데이터는 '오래 보관할 장부'를 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

13. MySQL과 정형 데이터 - 요청/데이터 흐름

오래 보관할 장부



흐름 읽기

요청은 항상 입구, 정책 판단, 실제 처리, 응답의 순서로 본다. MySQL과 정형 데이터에서도 같은 순서로 로그를 따라가면 원인 파악이 빨라진다.

확인할 데이터

rows examined, full scan, index hit, lock wait, slow query를 본다.

다음 단계

EXPLAIN 결과에서 type, key, rows 컬럼부터 읽어 본다.

기억할 문장

- MySQL과 정형 데이터는 '오래 보관할 장부'를 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

13. MySQL과 정형 데이터 - 코드 작성 원칙

오래 보관할 장부



OCP 핵심

새 구현체를 추가해 확장하고, 이미 검증된 호출부는 되도록 건드리지 않는다.

그림 해석: 오래 보관할 장부 - rows examined, full scan, index hit, lock wait, slow query를 본다.

코드 원칙

SELECT *, 묵시적 형변환, 인덱스 컬럼 변형을 피하고 query contract를 테스트한다.

설계 연결

SRP는 책임을 나누고, OCP는 새 구현체를 붙이기 쉽게 만들며, DIP는 구체 구현 대신 interface를 바라보게 만든다.

작은 예시

문자열 endpoint를 컨트롤러에 직접 쓰지 말고 RouteContract 또는 ApiRoutes 같은 상수 객체에 모은다.

기억할 문장

- MySQL과 정형 데이터는 '오래 보관할 장부'를 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

13. MySQL과 정형 데이터 - 테스트와 검증

오래 보관할 장부

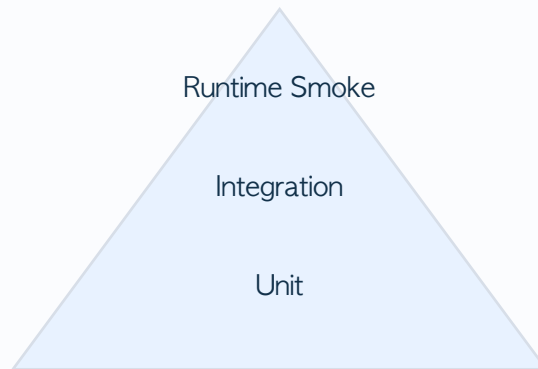


그림 해석: 오래 보관할 장부 - rows examined, full scan, index hit, lock wait, slow query를 본다.

테스트 원칙

실행 계획, 인덱스 사용, no-offset paging, schema contract를 테스트에 반영한다.

TDD 관점

먼저 실패하는 계약 테스트를 만들고, 그 계약을 만족하는 가장 작은 구현을 작성한 뒤, 실제 runtime smoke로 연결을 확인한다.

통과 기준

rows examined, full scan, index hit, lock wait, slow query를 본다.가
기대 범위 안에 있고, 실패 시 원인이 로그와 테스트 이름으로 드러나야 한다.

기억할 문장

- MySQL과 정형 데이터는 '오래 보관할 장부'를 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

13. MySQL과 정형 데이터 - 운영 체크

오래 보관할 장부

이 페이지의 핵심

사용자, 회사, 권한 같은 정형 데이터는 안정적인 영구 저장소가 필요하다.

운영 원칙

백업, migration, 계정 권한, 포트 충돌을 운영에서 주기적으로 확인한다.

장애 대응

status code, container health, application log, network path,
secret/config 누락을 순서대로 확인한다.

기록할 것

명령어, 실행 시각, 결과, 실패 원인, 수정 파일, 남은 리스크를 보고서에 남긴다.

기억할 문장

- MySQL과 정형 데이터는 '오래 보관할 장부'를 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

13. MySQL과 정형 데이터 - 자주 나는 실수

오래 보관할 장부

이 페이지의 핵심

사용자, 회사, 권한 같은 정형 데이터는 안정적인 영구 저장소가 필요하다.

자주 나는 실수

인덱스가 많다고 항상 좋은 것이 아니며 쓰기 성능을 떨어뜨릴 수 있다.

왜 위험한가

초반에는 빠르게 보이지만, 운영 중 장애 원인을 숨기거나 변경 범위를 키워 결국 더 오래 걸리게 만든다.

예방 습관

계약값 중앙화, DTO 분리, 테스트 추가, 로그/지표 확인을 PR 기준으로 둔다.

기억할 문장

- MySQL과 정형 데이터는 '오래 보관할 장부'를 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

13. MySQL과 정형 데이터 - 초보자 실습

오래 보관할 장부

이 페이지의 핵심

사용자, 회사, 권한 같은 정형 데이터는 안정적인 영구 저장소가 필요하다.

따라 해보기

EXPLAIN 결과에서 type, key, rows 컬럼부터 읽어 본다.

관찰하기

작업 전후로 rows examined, full scan, index hit, lock wait, slow query를 본다.를 기록한다. 숫자를 모르면 개선인지 착각인지 구분할 수 없다.

질문 만들기

'MySQL과 정형 데이터이 깨지면 사용자는 무엇을 보게 되는가?'를 한 문장으로 답해 본다.

기억할 문장

- MySQL과 정형 데이터는 '오래 보관할 장부'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

13. MySQL과 정형 데이터 - 복습 체크리스트

오래 보관할 장부

이 페이지의 핵심

사용자, 회사, 권한 같은 정형 데이터는 안정적인 영구 저장소가 필요하다.

복습 질문

MySQL과 정형 데이터는 control plane, media plane, data plane, ops plane 중 어디에 가까운가?

체크리스트

경로가 상수화되어 있는가? DTO가 분리되어 있는가? 실패 테스트가 있는가? 운영 지표가 있는가? secret이 노출되지 않았는가?

다음 학습

관련 코드 파일 하나와 테스트 파일 하나를 함께 열어 구현과 검증이 어떻게 연결되는지 읽는다.

기억할 문장

- MySQL과 정형 데이터는 '오래 보관할 장부'를 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

14. Redis와 캐시/세션 - 개념 지도

빠른 메모리 저장소

이 페이지의 핵심

실시간 화면은 최신 상태와 세션을 빠르게 읽어야 하므로 Redis가 유리하다.

왜 필요한가

실시간 화면은 최신 상태와 세션을 빠르게 읽어야 하므로 Redis가 유리하다.

서버에서 맡는 역할

Redis는 refresh session, stream presence, latest status, short TTL cache에 사용된다.

초보자 비유

철판처럼 빠르게 쓰고 지울 수 있지만 영구 장부는 아니다.

기억할 문장

- Redis와 캐시/세션은 '빠른 메모리 저장소'를 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

14. Redis와 캐시/세션 - 서버 적용 위치

빠른 메모리 저장소

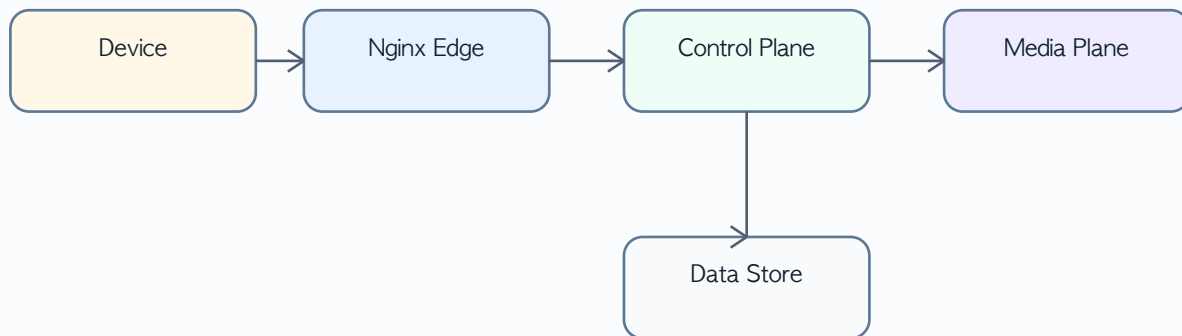


그림 해석: 빠른 메모리 저장소 - cache hit ratio, command latency, memory usage, expired keys를 본다.

서버 적용 위치

Redis는 refresh session, stream presence, latest status, short TTL cache에 사용된다.

연결해서 볼 것

Redis와 캐시/세션은 단독 기술이 아니라 Nginx, Docker Compose, 인증, 미디어 경로, 테스트와 함께 동작한다.

운영 지표

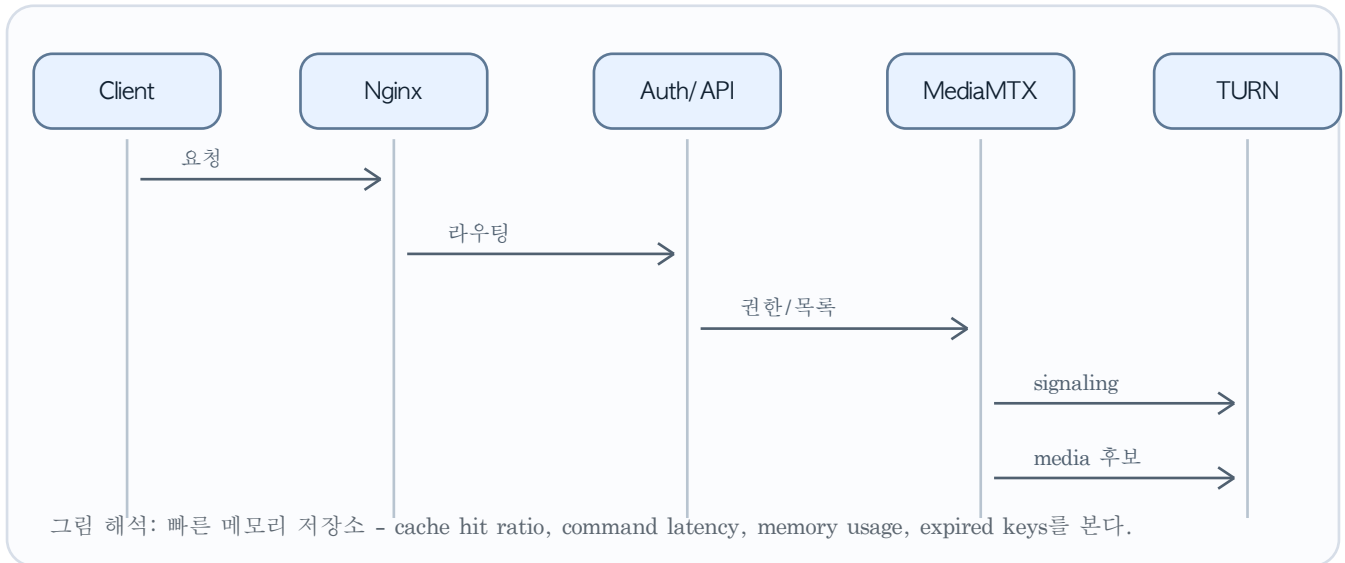
cache hit ratio, command latency, memory usage, expired keys를 본다.

기억할 문장

- Redis와 캐시/세션은 '빠른 메모리 저장소'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

14. Redis와 캐시/세션 - 요청/데이터 흐름

빠른 메모리 저장소



흐름 읽기

요청은 항상 입구, 정책 판단, 실제 처리, 응답의 순서로 본다. Redis와 캐시/세션에서도 같은 순서로 로그를 따라가면 원인 파악이 빨라진다.

확인할 데이터

cache hit ratio, command latency, memory usage, expired keys를 본다.

다음 단계

같은 데이터를 MySQL에서 읽을 때와 Redis에서 읽을 때의 차이를 측정해 본다.

기억할 문장

- Redis와 캐시/세션은 '빠른 메모리 저장소'를 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

14. Redis와 캐시/세션 - 코드 작성 원칙

빠른 메모리 저장소



OCP 핵심

새 구현체를 추가해 확장하고, 이미 검증된 호출부는 되도록 건드리지 않는다.

그림 해석: 빠른 메모리 저장소 - cache hit ratio, command latency, memory usage, expired keys를 본다.

코드 원칙

TTL, key prefix, atomic consume, cache stampede 방지를 contract로 관리한다.

설계 연결

SRP는 책임을 나누고, OCP는 새 구현체를 붙이기 쉽게 만들며, DIP는 구체 구현 대신 interface를 바라보게 만든다.

작은 예시

문자열 endpoint를 컨트롤러에 직접 쓰지 말고 RouteContract 또는 ApiRoutes 같은 상수 객체에 모은다.

기억할 문장

- Redis와 캐시/세션은 '빠른 메모리 저장소'를 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

14. Redis와 캐시/세션 - 테스트와 검증

빠른 메모리 저장소

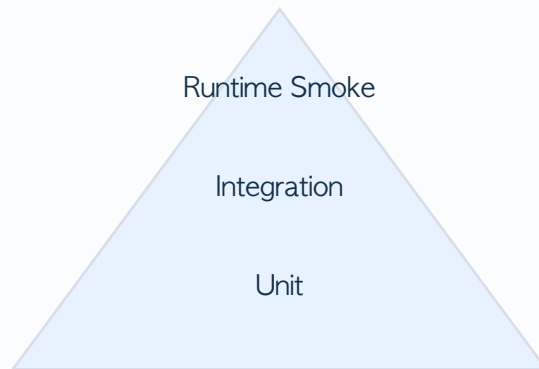


그림 해석: 빠른 메모리 저장소 - cache hit ratio, command latency, memory usage, expired keys를 본다.

테스트 원칙

refresh token 동시 consume, Redis 장애 degraded behavior, cache TTL test를 둔다.

TDD 관점

먼저 실패하는 계약 테스트를 만들고, 그 계약을 만족하는 가장 작은 구현을 작성한 뒤, 실제 runtime smoke로 연결을 확인한다.

통과 기준

cache hit ratio, command latency, memory usage, expired keys를 본다.가 기대 범위 안에 있고, 실패 시 원인이 로그와 테스트 이름으로 드러나야 한다.

기억할 문장

- Redis와 캐시/세션은 '빠른 메모리 저장소'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

14. Redis와 캐시/세션 - 운영 체크

빠른 메모리 저장소

이 페이지의 핵심

실시간 화면은 최신 상태와 세션을 빠르게 읽어야 하므로 Redis가 유리하다.

운영 원칙

maxmemory, eviction policy, slowlog, connection count를 운영에서 확인한다.

장애 대응

status code, container health, application log, network path,
secret/config 누락을 순서대로 확인한다.

기록할 것

명령어, 실행 시각, 결과, 실패 원인, 수정 파일, 남은 리스크를 보고서에 남긴다.

기억할 문장

- Redis와 캐시/세션은 '빠른 메모리 저장소'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

14. Redis와 캐시/세션 - 자주 나는 실수

빠른 메모리 저장소

이 페이지의 핵심

실시간 화면은 최신 상태와 세션을 빠르게 읽어야 하므로 Redis가 유리하다.

자주 나는 실수

KEYS * 같은 명령은 운영 Redis를 멈추게 할 수 있으므로 SCAN 계열을 쓴다.

왜 위험한가

초반에는 빠르게 보이지만, 운영 중 장애 원인을 숨기거나 변경 범위를 키워 결국 더 오래 걸리게 만든다.

예방 습관

계약값 중앙화, DTO 분리, 테스트 추가, 로그/지표 확인을 PR 기준으로 둔다.

기억할 문장

- Redis와 캐시/세션은 '빠른 메모리 저장소'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

14. Redis와 캐시/세션 - 초보자 실습

빠른 메모리 저장소

이 페이지의 핵심

실시간 화면은 최신 상태와 세션을 빠르게 읽어야 하므로 Redis가 유리하다.

따라 해보기

같은 데이터를 MySQL에서 읽을 때와 Redis에서 읽을 때의 차이를 측정해 본다.

관찰하기

작업 전후로 cache hit ratio, command latency, memory usage, expired keys를 본다.를 기록한다. 숫자를 모르면 개선인지 착각인지 구분할 수 없다.

질문 만들기

'Redis와 캐시/세션이 깨지면 사용자는 무엇을 보게 되는가?'를 한 문장으로 답해 본다.

기억할 문장

- Redis와 캐시/세션은 '빠른 메모리 저장소'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

14. Redis와 캐시/세션 - 복습 체크리스트

빠른 메모리 저장소

이 페이지의 핵심

실시간 화면은 최신 상태와 세션을 빠르게 읽어야 하므로 Redis가 유리하다.

복습 질문

Redis와 캐시/세션은 control plane, media plane, data plane, ops plane 중 어디에 가까운가?

체크리스트

경로가 상수화되어 있는가? DTO가 분리되어 있는가? 실패 테스트가 있는가? 운영 지표가 있는가? secret이 노출되지 않았는가?

다음 학습

관련 코드 파일 하나와 테스트 파일 하나를 함께 열어 구현과 검증이 어떻게 연결되는지 읽는다.

기억할 문장

- Redis와 캐시/세션은 '빠른 메모리 저장소'를 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

15. MQTT와 이벤트 메시징 - 개념 지도

장비 이벤트를 가볍게 주고받기

이 페이지의 핵심

드론/로봇/센서 이벤트는 HTTP 요청보다 메시지 버스로 다루는 편이 자연스러운 경우가 많다.

왜 필요한가

드론/로봇/센서 이벤트는 HTTP 요청보다 메시지 버스로 다루는 편이 자연스러운 경우가 많다.

서버에서 맡는 역할

Mosquitto MQTT는 command/event bus 후보로 들어가 있다.

초보자 비유

무전 채널처럼 topic을 정하고 필요한 구독자가 메시지를 듣는다.

기억할 문장

- MQTT와 이벤트 메시징은 '장비 이벤트를 가볍게 주고받기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

15. MQTT와 이벤트 메시징 - 서버 적용 위치

장비 이벤트를 가볍게 주고받기

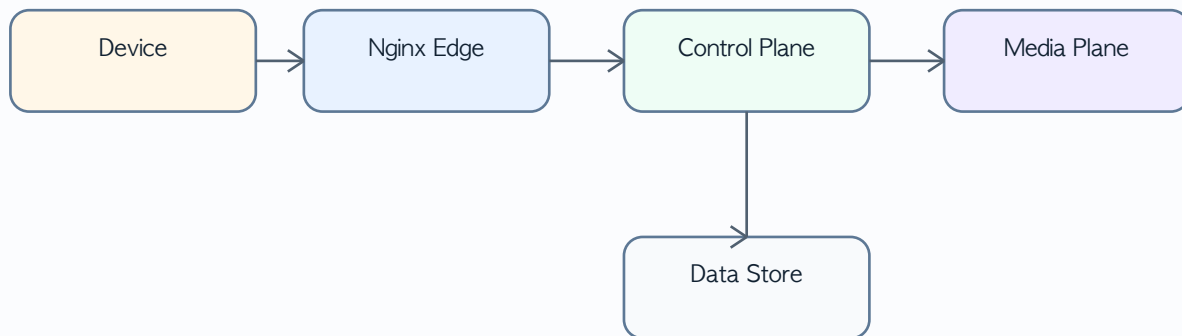


그림 해석: 장비 이벤트를 가볍게 주고받기 - message rate, dropped connection, broker memory, reconnect count를 본다.

서버 적용 위치

Mosquitto MQTT는 command/event bus 후보로 들어가 있다.

연결해서 볼 것

MQTT와 이벤트 메시징은 단독 기술이 아니라 Nginx, Docker Compose, 인증, 미디어 경로, 테스트와 함께 동작한다.

운영 지표

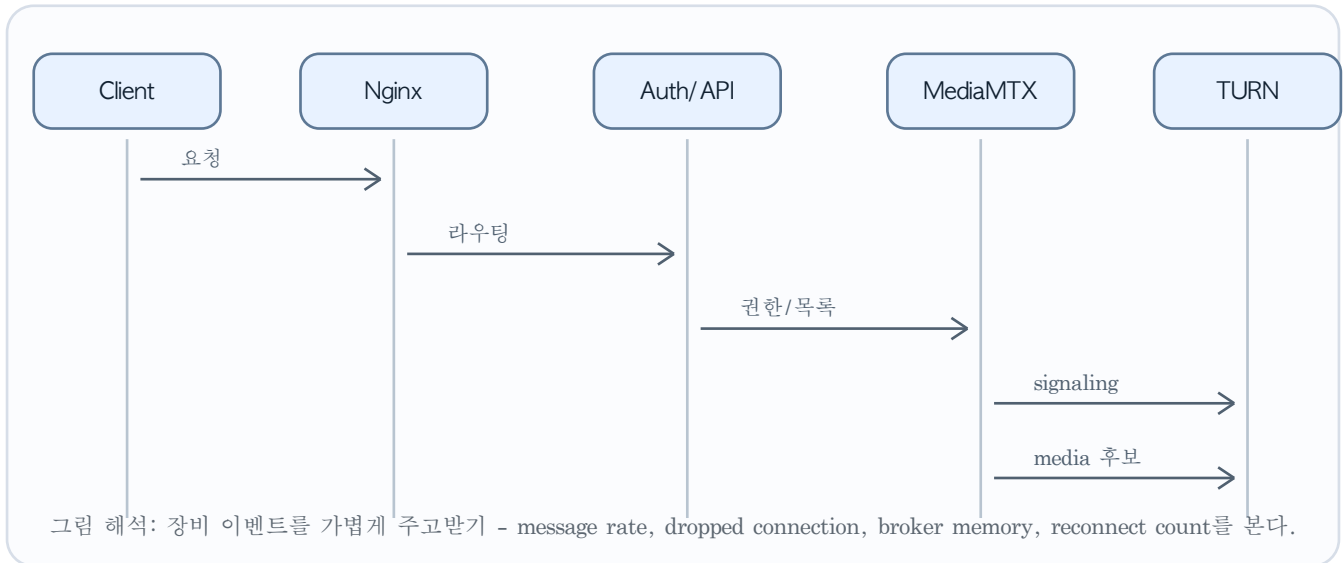
message rate, dropped connection, broker memory, reconnect count를 본다.

기억할 문장

- MQTT와 이벤트 메시징은 '장비 이벤트를 가볍게 주고받기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

15. MQTT와 이벤트 메시징 - 요청/데이터 흐름

장비 이벤트를 가볍게 주고받기



흐름 읽기

요청은 항상 입구, 정책 판단, 실제 처리, 응답의 순서로 본다. MQTT와 이벤트 메시징에서도 같은 순서로 로그를 따라가면 원인 파악이 빨라진다.

확인할 데이터

message rate, dropped connection, broker memory, reconnect count를 본다.

다음 단계

topic 이름을 조직/장비/이벤트 기준으로 나눠 설계해 본다.

기억할 문장

- MQTT와 이벤트 메시징은 '장비 이벤트를 가볍게 주고받기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

15. MQTT와 이벤트 메시징 - 코드 작성 원칙

장비 이벤트를 가볍게 주고받기



OCP 핵심

새 구현체를 추가해 확장하고, 이미 검증된 호출부는 되도록 건드리지 않는다.

그림 해석: 장비 이벤트를 가볍게 주고받기 - message rate, dropped connection, broker memory, reconnect count를 본다.

코드 원칙

topic 이름, payload DTO, QoS 정책을 상수와 contract로 관리해야 한다.

설계 연결

SRP는 책임을 나누고, OCP는 새 구현체를 붙이기 쉽게 만들며, DIP는 구체 구현 대신 interface를 바라보게 만든다.

작은 예시

문자열 endpoint를 컨트롤러에 직접 쓰지 말고 RouteContract 또는 ApiRoutes 같은 상수 객체에 모은다.

기억할 문장

- MQTT와 이벤트 메시징은 '장비 이벤트를 가볍게 주고받기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

15. MQTT와 이벤트 메시징 - 테스트와 검증

장비 이벤트를 가볍게 주고받기

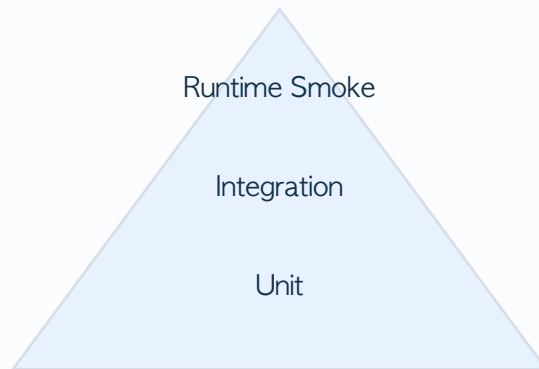


그림 해석: 장비 이벤트를 가볍게 주고받기 - message rate, dropped connection, broker memory, reconnect count를 본다.

테스트 원칙

publish/subscribe smoke, malformed payload rejection, reconnect test를 둔다.

TDD 관점

먼저 실패하는 계약 테스트를 만들고, 그 계약을 만족하는 가장 작은 구현을 작성한 뒤, 실제 runtime smoke로 연결을 확인한다.

통과 기준

message rate, dropped connection, broker memory, reconnect count를 본다.가 기대 범위 안에 있고, 실패 시 원인이 로그와 테스트 이름으로 드러나야 한다.

기억할 문장

- MQTT와 이벤트 메시징은 '장비 이벤트를 가볍게 주고받기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

15. MQTT와 이벤트 메시징 - 운영 체크

장비 이벤트를 가볍게 주고받기

이 페이지의 핵심

드론/로봇/센서 이벤트는 HTTP 요청보다 메시지 버스로 다루는 편이 자연스러운 경우가 많다.

운영 원칙

운영에서는 topic 폭증, retained message, 인증 설정, broker health를 본다.

장애 대응

status code, container health, application log, network path,
secret/config 누락을 순서대로 확인한다.

기록할 것

명령어, 실행 시각, 결과, 실패 원인, 수정 파일, 남은 리스크를 보고서에 남긴다.

기억할 문장

- MQTT와 이벤트 메시징은 '장비 이벤트를 가볍게 주고받기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

15. MQTT와 이벤트 메시징 - 자주 나는 실수

장비 이벤트를 가볍게 주고받기

이 페이지의 핵심

드론/로봇/센서 이벤트는 HTTP 요청보다 메시지 버스로 다루는 편이 자연스러운 경우가 많다.

자주 나는 실수

topic 설계 없이 아무 메시지도 흘리면 나중에 필터링과 권한 관리가 어렵다.

왜 위험한가

초반에는 빠르게 보이지만, 운영 중 장애 원인을 숨기거나 변경 범위를 키워 결국 더 오래 걸리게 만든다.

예방 습관

계약값 중앙화, DTO 분리, 테스트 추가, 로그/지표 확인을 PR 기준으로 둔다.

기억할 문장

- MQTT와 이벤트 메시징은 '장비 이벤트를 가볍게 주고받기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

15. MQTT와 이벤트 메시징 - 초보자 실습

장비 이벤트를 가볍게 주고받기

이 페이지의 핵심

드론/로봇/센서 이벤트는 HTTP 요청보다 메시지 버스로 다루는 편이 자연스러운 경우가 많다.

따라 해보기

topic 이름을 조직/장비/이벤트 기준으로 나눠 설계해 본다.

관찰하기

작업 전후로 message rate, dropped connection, broker memory, reconnect count를 본다.를 기록한다. 숫자를 모르면 개선인지 착각인지 구분할 수 없다.

질문 만들기

'MQTT와 이벤트 메시징이 깨지면 사용자는 무엇을 보게 되는가?'를 한 문장으로 답해 본다.

기억할 문장

- MQTT와 이벤트 메시징은 '장비 이벤트를 가볍게 주고받기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

15. MQTT와 이벤트 메시징 - 복습 체크리스트

장비 이벤트를 가볍게 주고받기

이 페이지의 핵심

드론/로봇/센서 이벤트는 HTTP 요청보다 메시지 버스로 다루는 편이 자연스러운 경우가 많다.

복습 질문

MQTT와 이벤트 메시징은 control plane, media plane, data plane, ops plane 중 어디에 가까운가?

체크리스트

경로가 상수화되어 있는가? DTO가 분리되어 있는가? 실패 테스트가 있는가? 운영 지표가 있는가? secret이 노출되지 않았는가?

다음 학습

관련 코드 파일 하나와 테스트 파일 하나를 함께 열어 구현과 검증이 어떻게 연결되는지 읽는다.

기억할 문장

- MQTT와 이벤트 메시징은 '장비 이벤트를 가볍게 주고받기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

16. 지도, GPS, 시간 동기화 - 개념 지도

상황을 같은 시각과 위치에 맞추기

이 페이지의 핵심

영상, 음성, GPS, AI overlay가 같은 사건을 가리키려면 위치와 시간이 맞아야 한다.

왜 필요한가

영상, 음성, GPS, AI overlay가 같은 사건을 가리키려면 위치와 시간이 맞아야 한다.

서버에서 맡는 역할

dashboard는 stream GPS를 지도 focus와 telemetry panel에 반영하고, time API는 공개망/폐쇄망 모드를 나눈다.

초보자 비유

카메라 영상과 지도 핀이 같은 시계를 보게 만드는 작업이다.

기억할 문장

- 지도, GPS, 시간 동기화는 '상황을 같은 시각과 위치에 맞추기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

16. 지도, GPS, 시간 동기화 - 서버 적용 위치

상황을 같은 시각과 위치에 맞추기

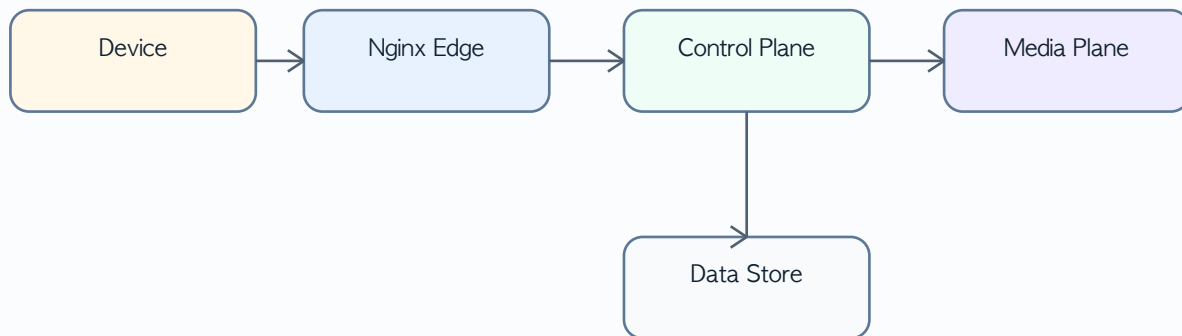


그림 해석: 상황을 같은 시각과 위치에 맞추기 - GPS update latency, drift ms, map tile load, telemetry freshness를 본다.

서버 적용 위치

dashboard는 stream GPS를 지도 focus와 telemetry panel에 반영하고, time API는 공개망/폐쇄망 모드를 나눈다.

연결해서 볼 것

지도, GPS, 시간 동기화는 단독 기술이 아니라 Nginx, Docker Compose, 인증, 미디어 경로, 테스트와 함께 동작한다.

운영 지표

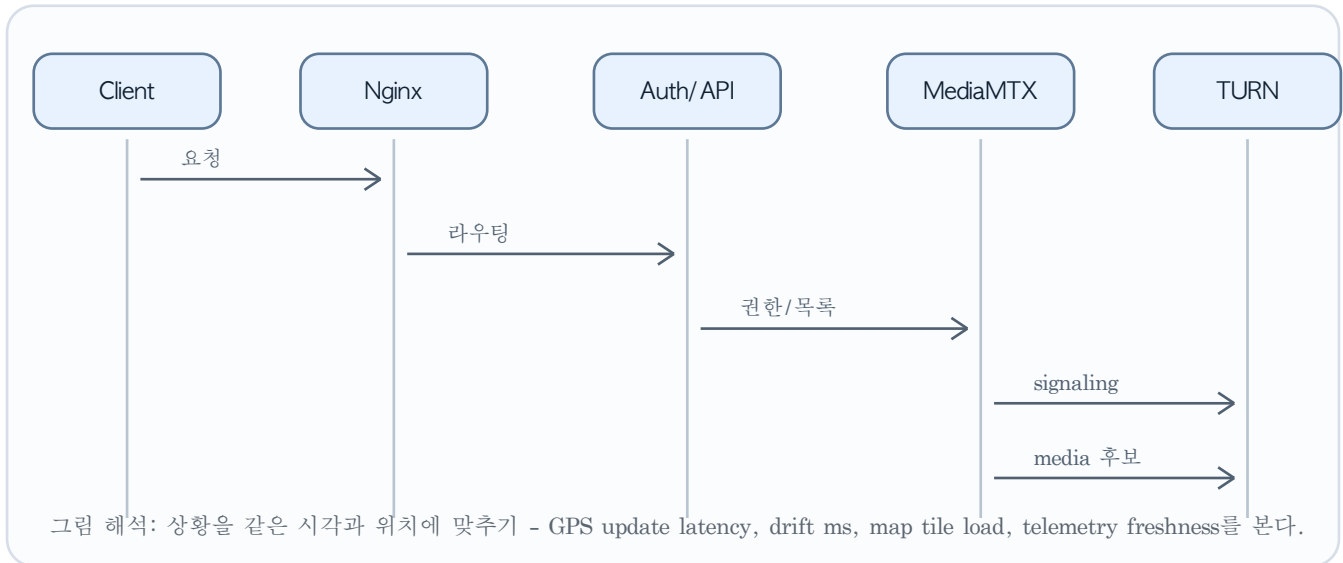
GPS update latency, drift ms, map tile load, telemetry freshness를 본다.

기억할 문장

- 지도, GPS, 시간 동기화는 '상황을 같은 시각과 위치에 맞추기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

16. 지도, GPS, 시간 동기화 - 요청/데이터 흐름

상황을 같은 시각과 위치에 맞추기



흐름 읽기

요청은 항상 입구, 정책 판단, 실제 처리, 응답의 순서로 본다. 지도, GPS, 시간 동기화에서도 같은 순서로 로그를 따라가면 원인 파악이 빨라진다.

확인할 데이터

GPS update latency, drift ms, map tile load, telemetry freshness를 본다.

다음 단계

위도/경도 하나가 지도, 자산 트리, telemetry panel로 이동하는 흐름을 추적한다.

기억할 문장

- 지도, GPS, 시간 동기화는 '상황을 같은 시각과 위치에 맞추기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

16. 지도, GPS, 시간 동기화 - 코드 작성 원칙

상황을 같은 시각과 위치에 맞추기



OCP 핵심

새 구현체를 추가해 확장하고, 이미 검증된 호출부는 되도록 건드리지 않는다.

그림 해석: 상황을 같은 시각과 위치에 맞추기 - GPS update latency, drift ms, map tile load, telemetry freshness를 본다.

코드 원칙

Coordinate, Timestamp, MapProvider 같은 값 객체와 설정 contract를 둔다.

설계 연결

SRP는 책임을 나누고, OCP는 새 구현체를 붙이기 쉽게 만들며, DIP는 구체 구현 대신 interface를 바라보게 만든다.

작은 예시

문자열 endpoint를 컨트롤러에 직접 쓰지 말고 RouteContract 또는 ApiRoutes 같은 상수 객체에 모은다.

기억할 문장

- 지도, GPS, 시간 동기화는 '상황을 같은 시각과 위치에 맞추기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

16. 지도, GPS, 시간 동기화 - 테스트와 검증

상황을 같은 시각과 위치에 맞추기

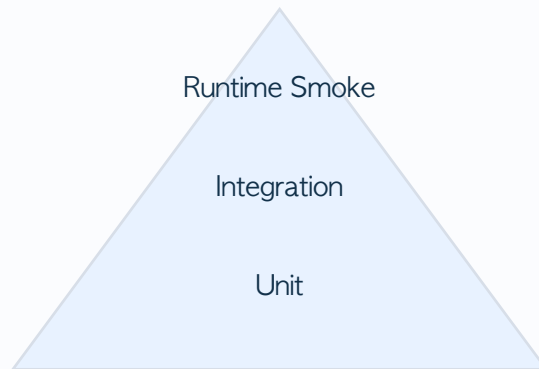


그림 해석: 상황을 같은 시각과 위치에 맞추기 - GPS update latency, drift ms, map tile load, telemetry freshness를 본다.

테스트 원칙

GPS ingest/read, map focus, time sync status, closed-network config test를 둔다.

TDD 관점

먼저 실패하는 계약 테스트를 만들고, 그 계약을 만족하는 가장 작은 구현을 작성한 뒤, 실제 runtime smoke로 연결을 확인한다.

통과 기준

GPS update latency, drift ms, map tile load, telemetry freshness를 본다.가 기대 범위 안에 있고, 실패 시 원인이 로그와 테스트 이름으로 드러나야 한다.

기억할 문장

- 지도, GPS, 시간 동기화는 '상황을 같은 시각과 위치에 맞추기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

16. 지도, GPS, 시간 동기화 - 운영 체크

상황을 같은 시각과 위치에 맞추기

이 페이지의 핵심

영상, 음성, GPS, AI overlay가 같은 사건을 가리키려면 위치와 시간이 맞아야 한다.

운영 원칙

폐쇄망 납품에서는 외부 지도 API와 외부 NTP 의존을 제거할 준비가 필요하다.

장애 대응

status code, container health, application log, network path,
secret/config 누락을 순서대로 확인한다.

기록할 것

명령어, 실행 시각, 결과, 실패 원인, 수정 파일, 남은 리스크를 보고서에 남긴다.

기억할 문장

- 지도, GPS, 시간 동기화는 '상황을 같은 시각과 위치에 맞추기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

16. 지도, GPS, 시간 동기화 - 자주 나는 실수

상황을 같은 시각과 위치에 맞추기

이 페이지의 핵심

영상, 음성, GPS, AI overlay가 같은 사건을 가리키려면 위치와 시간이 맞아야 한다.

자주 나는 실수

좌표계, timestamp 단위, timezone을 섞으면 지도와 영상이 어긋난다.

왜 위험한가

초반에는 빠르게 보이지만, 운영 중 장애 원인을 숨기거나 변경 범위를 키워 결국 더 오래 걸리게 만든다.

예방 습관

계약값 중앙화, DTO 분리, 테스트 추가, 로그/지표 확인을 PR 기준으로 둔다.

기억할 문장

- 지도, GPS, 시간 동기화는 '상황을 같은 시각과 위치에 맞추기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

16. 지도, GPS, 시간 동기화 - 초보자 실습

상황을 같은 시각과 위치에 맞추기

이 페이지의 핵심

영상, 음성, GPS, AI overlay가 같은 사건을 가리키려면 위치와 시간이 맞아야 한다.

따라 해보기

위도/경도 하나가 지도, 자산 트리, telemetry panel로 이동하는 흐름을 추적한다.

관찰하기

작업 전후로 GPS update latency, drift ms, map tile load, telemetry freshness를 본다.를 기록한다. 숫자를 모르면 개선인지 착각인지 구분할 수 없다.

질문 만들기

'지도, GPS, 시간 동기화가 깨지면 사용자는 무엇을 보게 되는가?'를 한 문장으로 답해 본다.

기억할 문장

- 지도, GPS, 시간 동기화는 '상황을 같은 시각과 위치에 맞추기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

16. 지도, GPS, 시간 동기화 - 복습 체크리스트

상황을 같은 시각과 위치에 맞추기

이 페이지의 핵심

영상, 음성, GPS, AI overlay가 같은 사건을 가리키려면 위치와 시간이 맞아야 한다.

복습 질문

지도, GPS, 시간 동기화는 control plane, media plane, data plane, ops plane
중 어디에 가까운가?

체크리스트

경로가 상수화되어 있는가? DTO가 분리되어 있는가? 실패 테스트가 있는가? 운영 지표가 있는가? secret이 노출되지 않았는가?

다음 학습

관련 코드 파일 하나와 테스트 파일 하나를 함께 열어 구현과 검증이 어떻게 연결되는지 읽는다.

기억할 문장

- 지도, GPS, 시간 동기화는 '상황을 같은 시각과 위치에 맞추기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

17. TDD와 테스트 전략 - 개념 지도

실행 가능한 상태를 지키는 안전망

이 페이지의 핵심

실시간 시스템은 작은 변경도 로그인, 송출, 수신, 지도, 상태표시에 영향을 줄 수 있다.

왜 필요한가

실시간 시스템은 작은 변경도 로그인, 송출, 수신, 지도, 상태표시에 영향을 줄 수 있다.

서버에서 맡는 역할

unit, integration, smoke, runtime validation을 계층별로 둔다.

초보자 비유

자동차 출고 전 부품, 조립, 도로주행을 모두 확인하는 것과 같다.

기억할 문장

- TDD와 테스트 전략은 '실행 가능한 상태를 지키는 안전망'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

17. TDD와 테스트 전략 - 서버 적용 위치

실행 가능한 상태를 지키는 안전망

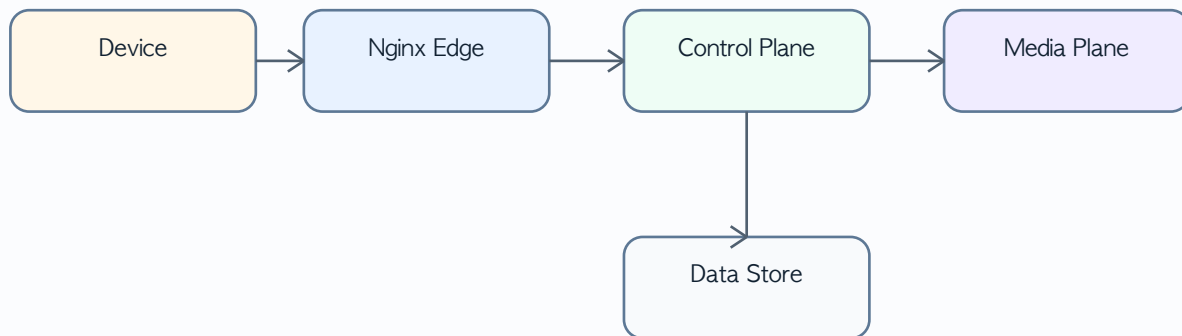


그림 해석: 실행 가능한 상태를 지키는 안전망 - coverage, failed scenario count, smoke latency, flaky test count를 본다.

서버 적용 위치

unit, integration, smoke, runtime validation을 계층별로 둔다.

연결해서 볼 것

TDD와 테스트 전략은 단독 기술이 아니라 Nginx, Docker Compose, 인증, 미디어 경로, 테스트와 함께 동작한다.

운영 지표

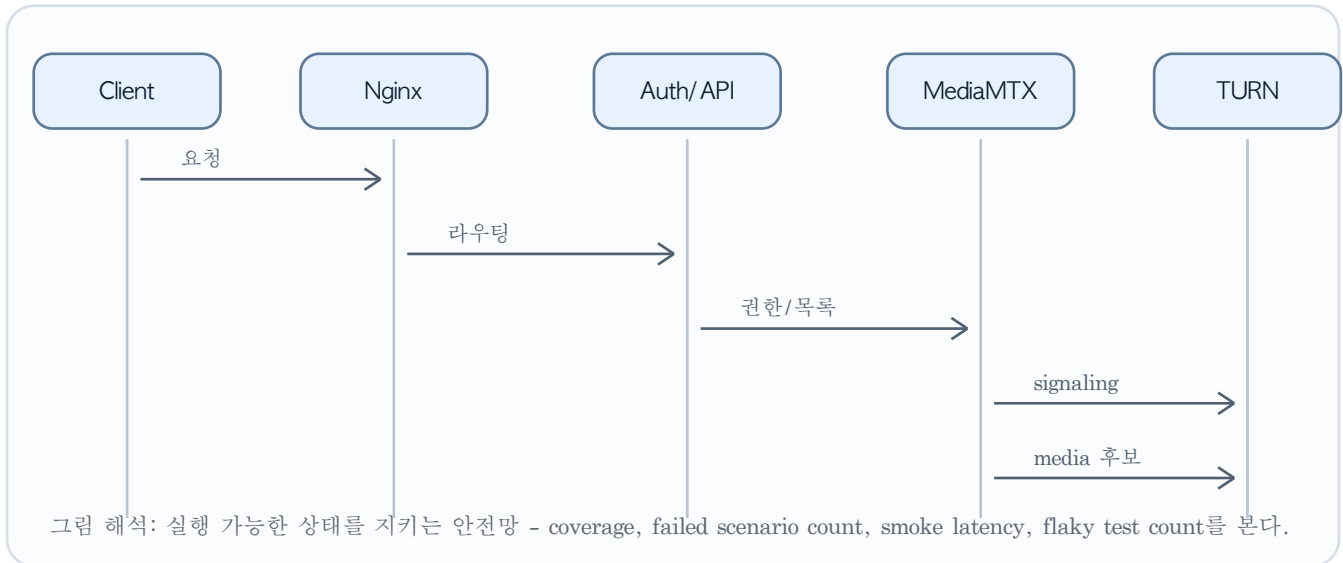
coverage, failed scenario count, smoke latency, flaky test count를 본다.

기억할 문장

- TDD와 테스트 전략은 '실행 가능한 상태를 지키는 안전망'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

17. TDD와 테스트 전략 - 요청/데이터 흐름

실행 가능한 상태를 지키는 안전망



흐름 읽기

요청은 항상 입구, 정책 판단, 실제 처리, 응답의 순서로 본다. TDD와 테스트 전략에서도 같은 순서로 로그를 따라가면 원인 파악이 빨라진다.

확인할 데이터

coverage, failed scenario count, smoke latency, flaky test count를 본다.

다음 단계

하나의 API에 unit, integration, smoke 테스트를 각각 하나씩 연결해 본다.

기억할 문장

- TDD와 테스트 전략은 '실행 가능한 상태를 지키는 안전망'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

17. TDD와 테스트 전략 - 코드 작성 원칙

실행 가능한 상태를 지키는 안전망



OCP 핵심

새 구현체를 추가해 확장하고, 이미 검증된 호출부는 되도록 건드리지 않는다.

그림 해석: 실행 가능한 상태를 지키는 안전망 - coverage, failed scenario count, smoke latency, flaky test count를 본다.

코드 원칙

테스트가 먼저 계약을 설명하고, 코드는 그 계약을 만족하도록 작성한다.

설계 연결

SRP는 책임을 나누고, OCP는 새 구현체를 붙이기 쉽게 만들며, DIP는 구체 구현 대신 interface를 바라보게 만든다.

작은 예시

문자열 endpoint를 컨트롤러에 직접 쓰지 말고 RouteContract 또는 ApiRoutes 같은 상수 객체에 모은다.

기억할 문장

- TDD와 테스트 전략은 '실행 가능한 상태를 지키는 안전망'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

17. TDD와 테스트 전략 - 테스트와 검증

실행 가능한 상태를 지키는 안전망

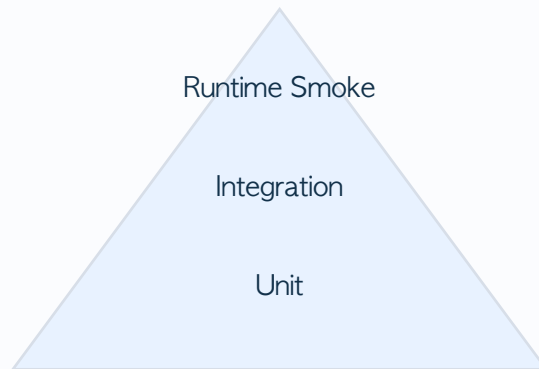


그림 해석: 실행 가능한 상태를 지키는 안전망 - coverage, failed scenario count, smoke latency, flaky test count를 본다.

테스트 원칙

coverage 숫자만 보지 말고 실패 시나리오와 외부 연결 DTO 검증을 포함한다.

TDD 관점

먼저 실패하는 계약 테스트를 만들고, 그 계약을 만족하는 가장 작은 구현을 작성한 뒤, 실제 runtime smoke로 연결을 확인한다.

통과 기준

coverage, failed scenario count, smoke latency, flaky test count를 본다.가 기대 범위 안에 있고, 실패 시 원인이 로그와 테스트 이름으로 드러나야 한다.

기억할 문장

- TDD와 테스트 전략은 '실행 가능한 상태를 지키는 안전망'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

17. TDD와 테스트 전략 - 운영 체크

실행 가능한 상태를 지키는 안전망

이 페이지의 핵심

실시간 시스템은 작은 변경도 로그인, 송출, 수신, 지도, 상태표시에 영향을 줄 수 있다.

운영 원칙

운영 배포 전에는 build 성공뿐 아니라 실제 endpoint와 smoke를 확인한다.

장애 대응

status code, container health, application log, network path, secret/config 누락을 순서대로 확인한다.

기록할 것

명령어, 실행 시각, 결과, 실패 원인, 수정 파일, 남은 리스크를 보고서에 남긴다.

기억할 문장

- TDD와 테스트 전략은 '실행 가능한 상태를 지키는 안전망'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

17. TDD와 테스트 전략 - 자주 나는 실수

실행 가능한 상태를 지키는 안전망

이 페이지의 핵심

실시간 시스템은 작은 변경도 로그인, 송출, 수신, 지도, 상태표시에 영향을 줄 수 있다.

자주 나는 실수

mock만 통과하고 실제 Docker에서 실패하면 테스트 계층이 부족한 것이다.

왜 위험한가

초반에는 빠르게 보이지만, 운영 중 장애 원인을 숨기거나 변경 범위를 키워 결국 더 오래 걸리게 만든다.

예방 습관

계약값 중앙화, DTO 분리, 테스트 추가, 로그/지표 확인을 PR 기준으로 둔다.

기억할 문장

- TDD와 테스트 전략은 '실행 가능한 상태를 지키는 안전망'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

17. TDD와 테스트 전략 - 초보자 실습

실행 가능한 상태를 지키는 안전망

이 페이지의 핵심

실시간 시스템은 작은 변경도 로그인, 송출, 수신, 지도, 상태표시에 영향을 줄 수 있다.

따라 해보기

하나의 API에 unit, integration, smoke 테스트를 각각 하나씩 연결해 본다.

관찰하기

작업 전후로 coverage, failed scenario count, smoke latency, flaky test count를 본다.를 기록한다. 숫자를 모르면 개선인지 착각인지 구분할 수 없다.

질문 만들기

'TDD와 테스트 전략이 깨지면 사용자는 무엇을 보게 되는가?'를 한 문장으로 답해 본다.

기억할 문장

- TDD와 테스트 전략은 '실행 가능한 상태를 지키는 안전망'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

17. TDD와 테스트 전략 - 복습 체크리스트

실행 가능한 상태를 지키는 안전망

이 페이지의 핵심

실시간 시스템은 작은 변경도 로그인, 송출, 수신, 지도, 상태표시에 영향을 줄 수 있다.

복습 질문

TDD와 테스트 전략은 control plane, media plane, data plane, ops plane 중 어디에 가까운가?

체크리스트

경로가 상수화되어 있는가? DTO가 분리되어 있는가? 실패 테스트가 있는가? 운영 지표가 있는가? secret이 노출되지 않았는가?

다음 학습

관련 코드 파일 하나와 테스트 파일 하나를 함께 열어 구현과 검증이 어떻게 연결되는지 읽는다.

기억할 문장

- TDD와 테스트 전략은 '실행 가능한 상태를 지키는 안전망'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

18. 운영 관측과 장애 대응 - 개념 지도

문제가 났을 때 빨리 알아차리기

이 페이지의 핵심

운영에서 중요한 것은 정상 동작뿐 아니라 장애를 감지하고 사용자에게 덜 불편하게 만드는 것이다.

왜 필요한가

운영에서 중요한 것은 정상 동작뿐 아니라 장애를 감지하고 사용자에게 덜 불편하게 만드는 것이다.

서버에서 맡는 역할

healthz, readyz, event log, container health, degraded behavior
runbook이 운영 안정성을 만든다.

초보자 비유

차량 계기판처럼 엔진, 연료, 온도, 경고등을 따로 보는 구조다.

기억할 문장

- 운영 관측과 장애 대응은 '문제가 났을 때 빨리 알아차리기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

18. 운영 관측과 장애 대응 - 서버 적용 위치

문제가 났을 때 빨리 알아차리기

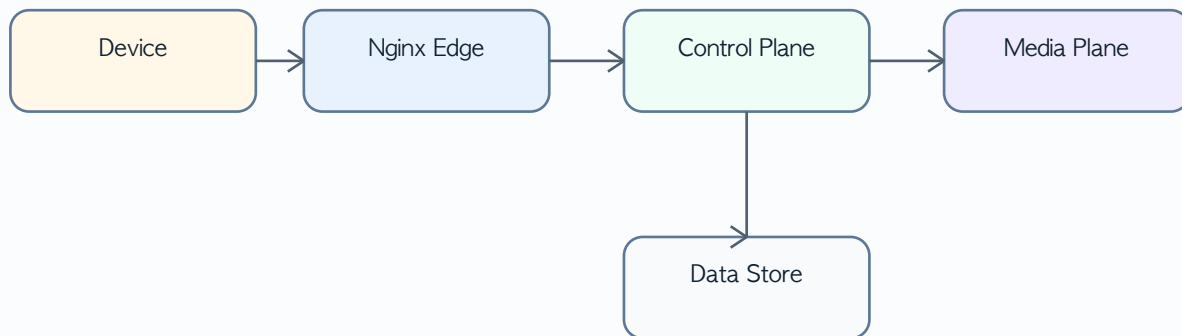


그림 해석: 문제가 났을 때 빨리 알아차리기 - error rate, reconnect count, degraded duration, recovery time objective를 본다.

서버 적용 위치

healthz, readyz, event log, container health, degraded behavior
runbook이 운영 안정성을 만든다.

연결해서 볼 것

운영 관측과 장애 대응은 단독 기술이 아니라 Nginx, Docker Compose, 인증, 미디어 경로, 테스트와 함께 동작한다.

운영 지표

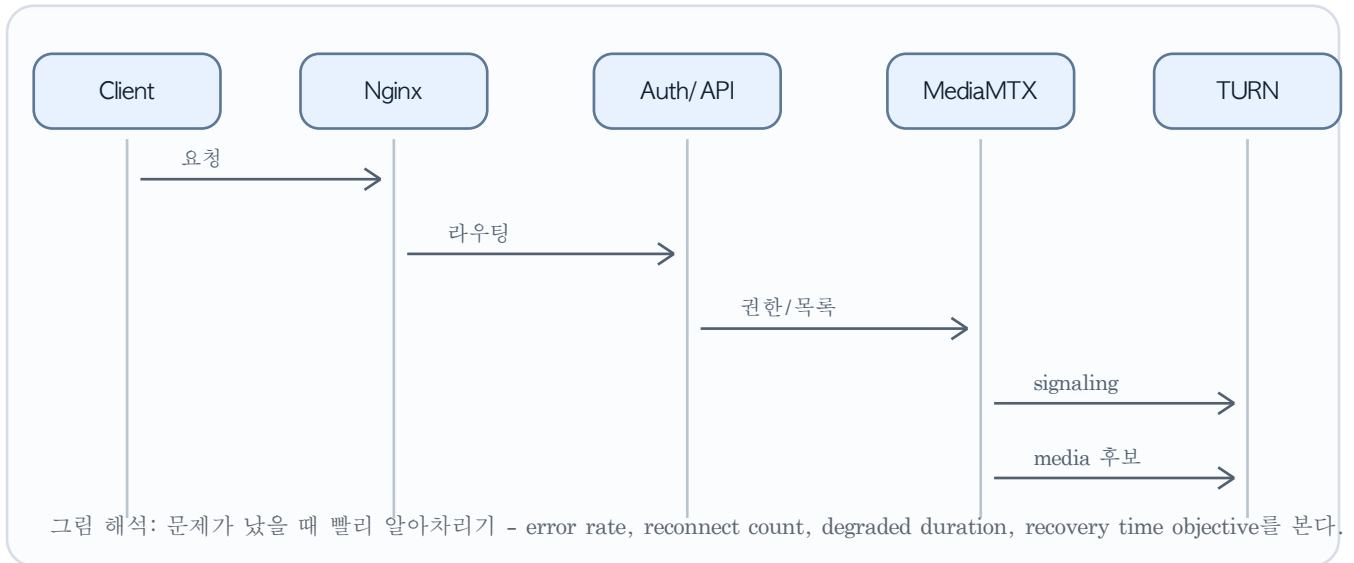
error rate, reconnect count, degraded duration, recovery time
objective를 본다.

기억할 문장

- 운영 관측과 장애 대응은 '문제가 났을 때 빨리 알아차리기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

18. 운영 관측과 장애 대응 - 요청/데이터 흐름

문제가 났을 때 빨리 알아차리기



흐름 읽기

요청은 항상 입구, 정책 판단, 실제 처리, 응답의 순서로 본다. 운영 관측과 장애 대응에서도 같은 순서로 로그를 따라가면 원인 파악이 빨라진다.

확인할 데이터

error rate, reconnect count, degraded duration, recovery time objective를 본다.

다음 단계

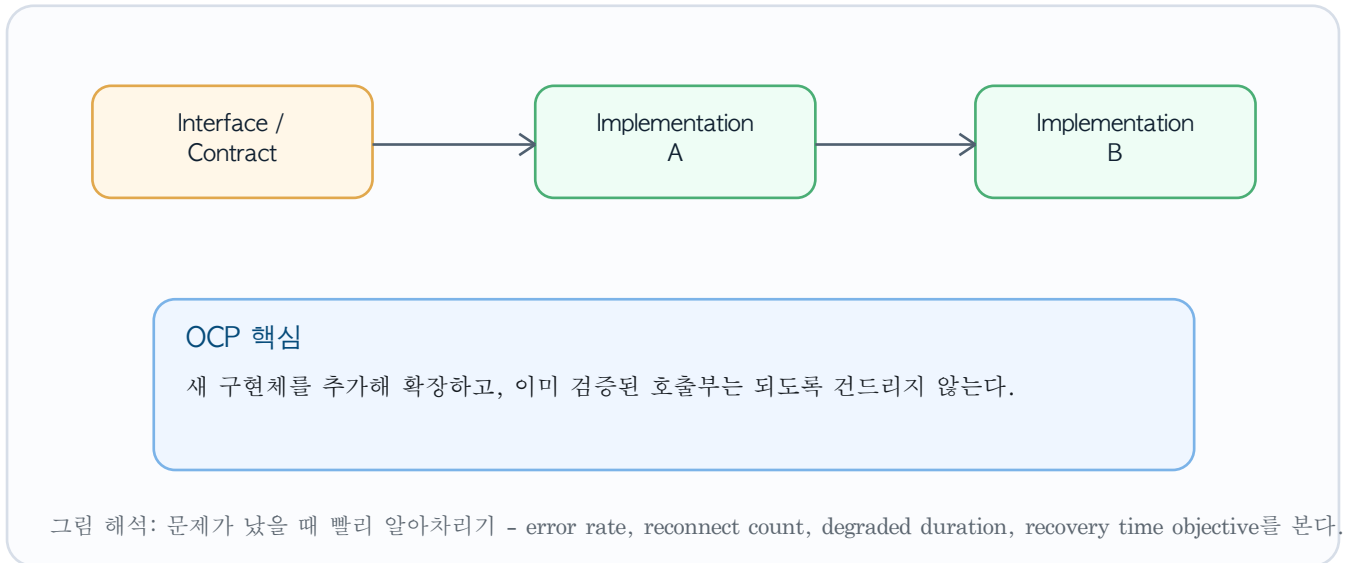
하나의 장애를 가정하고 사용자 화면, 로그, 복구 명령을 순서대로 적어 본다.

기억할 문장

- 운영 관측과 장애 대응은 '문제가 났을 때 빨리 알아차리기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

18. 운영 관측과 장애 대응 - 코드 작성 원칙

문제가 났을 때 빨리 알아차리기



코드 원칙

error type과 status를 custom error/contract로 정리해 로그와 UI가 같은 언어를 쓰게 한다.

설계 연결

SRP는 책임을 나누고, OCP는 새 구현체를 붙이기 쉽게 만들며, DIP는 구체 구현 대신 interface를 바라보게 만든다.

작은 예시

문자열 endpoint를 컨트롤러에 직접 쓰지 말고 RouteContract 또는 ApiRoutes 같은 상수 객체에 모은다.

기억할 문장

- 운영 관측과 장애 대응은 '문제가 났을 때 빨리 알아차리기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

18. 운영 관측과 장애 대응 - 테스트와 검증

문제가 났을 때 빨리 알아차리기

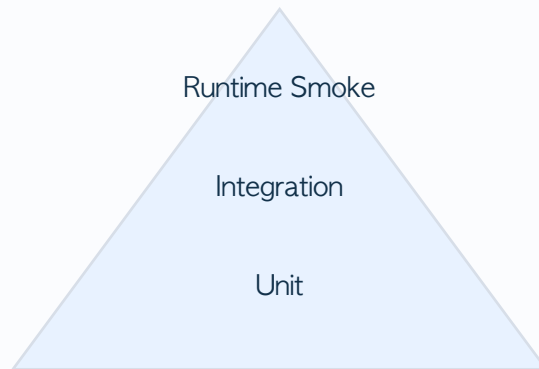


그림 해석: 문제가 났을 때 빨리 알아차리기 - error rate, reconnect count, degraded duration, recovery time objective를 본다.

테스트 원칙

Redis 장애, TURN 장애, MediaMTX 장애, restart scenario를 테스트한다.

TDD 관점

먼저 실패하는 계약 테스트를 만들고, 그 계약을 만족하는 가장 작은 구현을 작성한 뒤, 실제 runtime smoke로 연결을 확인한다.

통과 기준

error rate, reconnect count, degraded duration, recovery time

objective를 본다.가 기대 범위 안에 있고, 실패 시 원인이 로그와 테스트 이름으로 드러나야 한다.

기억할 문장

- 운영 관측과 장애 대응은 '문제가 났을 때 빨리 알아차리기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

18. 운영 관측과 장애 대응 - 운영 체크

문제가 났을 때 빨리 알아차리기

이 페이지의 핵심

운영에서 중요한 것은 정상 동작뿐 아니라 장애를 감지하고 사용자에게 덜 불편하게 만드는 것이다.

운영 원칙

장애 시 즉시 재시작보다 원인 로그 보존과 rollback 경로를 함께 관리한다.

장애 대응

status code, container health, application log, network path,
secret/config 누락을 순서대로 확인한다.

기록할 것

명령어, 실행 시각, 결과, 실패 원인, 수정 파일, 남은 리스크를 보고서에 남긴다.

기억할 문장

- 운영 관측과 장애 대응은 '문제가 났을 때 빨리 알아차리기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

18. 운영 관측과 장애 대응 - 자주 나는 실수

문제가 났을 때 빨리 알아차리기

이 페이지의 핵심

운영에서 중요한 것은 정상 동작뿐 아니라 장애를 감지하고 사용자에게 덜 불편하게 만드는 것이다.

자주 나는 실수

모든 오류를 500으로 던지면 운영자는 무엇이 잘못됐는지 모른다.

왜 위험한가

초반에는 빠르게 보이지만, 운영 중 장애 원인을 숨기거나 변경 범위를 키워 결국 더 오래 걸리게 만든다.

예방 습관

계약값 중앙화, DTO 분리, 테스트 추가, 로그/지표 확인을 PR 기준으로 둔다.

기억할 문장

- 운영 관측과 장애 대응은 '문제가 났을 때 빨리 알아차리기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

18. 운영 관측과 장애 대응 - 초보자 실습

문제가 났을 때 빨리 알아차리기

이 페이지의 핵심

운영에서 중요한 것은 정상 동작뿐 아니라 장애를 감지하고 사용자에게 덜 불편하게 만드는 것이다.

따라 해보기

하나의 장애를 가정하고 사용자 화면, 로그, 복구 명령을 순서대로 적어 본다.

관찰하기

작업 전후로 error rate, reconnect count, degraded duration, recovery time objective를 본다.를 기록한다. 숫자를 모르면 개선인지 착각인지 구분할 수 없다.

질문 만들기

'운영 관측과 장애 대응이 깨지면 사용자는 무엇을 보게 되는가?'를 한 문장으로 답해 본다.

기억할 문장

- 운영 관측과 장애 대응은 '문제가 났을 때 빨리 알아차리기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

18. 운영 관측과 장애 대응 - 복습 체크리스트

문제가 났을 때 빨리 알아차리기

이 페이지의 핵심

운영에서 중요한 것은 정상 동작뿐 아니라 장애를 감지하고 사용자에게 덜 불편하게 만드는 것이다.

복습 질문

운영 관측과 장애 대응은 control plane, media plane, data plane, ops plane 중 어디에 가까운가?

체크리스트

경로가 상수화되어 있는가? DTO가 분리되어 있는가? 실패 테스트가 있는가? 운영 지표가 있는가? secret이 노출되지 않았는가?

다음 학습

관련 코드 파일 하나와 테스트 파일 하나를 함께 열어 구현과 검증이 어떻게 연결되는지 읽는다.

기억할 문장

- 운영 관측과 장애 대응은 '문제가 났을 때 빨리 알아차리기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

19. SOLID와 OCP - 개념 지도

오래 버티는 코드의 원칙

이 페이지의 핵심

기능이 많아질수록 무작정 고치는 코드는 깨지기 쉽다. SOLID는 변경을 작게 만드는 기준이다.

왜 필요한가

기능이 많아질수록 무작정 고치는 코드는 깨지기 쉽다. SOLID는 변경을 작게 만드는 기준이다.

서버에서 맡는 역할

GCS-Saker에서는 route contract, interface, adapter, strategy, DTO 분리로 SOLID를 적용한다.

초보자 비유

콘센트 규격을 맞추면 새 기기를 꽂기 쉬운 것처럼 interface는 교체를 쉽게 한다.

기억할 문장

- SOLID와 OCP은 '오래 버티는 코드의 원칙'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

19. SOLID와 OCP - 서버 적용 위치

오래 버티는 코드의 원칙

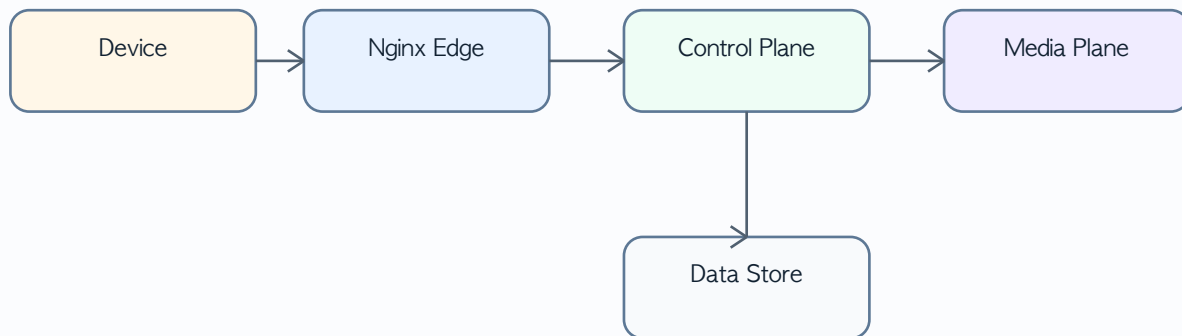


그림 해석: 오래 버티는 코드의 원칙 - modified files per feature, duplicate constants, direct string count, test seam count를 본다.

서버 적용 위치

GCS-Saker에서는 route contract, interface, adapter, strategy, DTO 분리로 SOLID를 적용한다.

연결해서 볼 것

SOLID와 OCP은 단독 기술이 아니라 Nginx, Docker Compose, 인증, 미디어 경로, 테스트와 함께 동작한다.

운영 지표

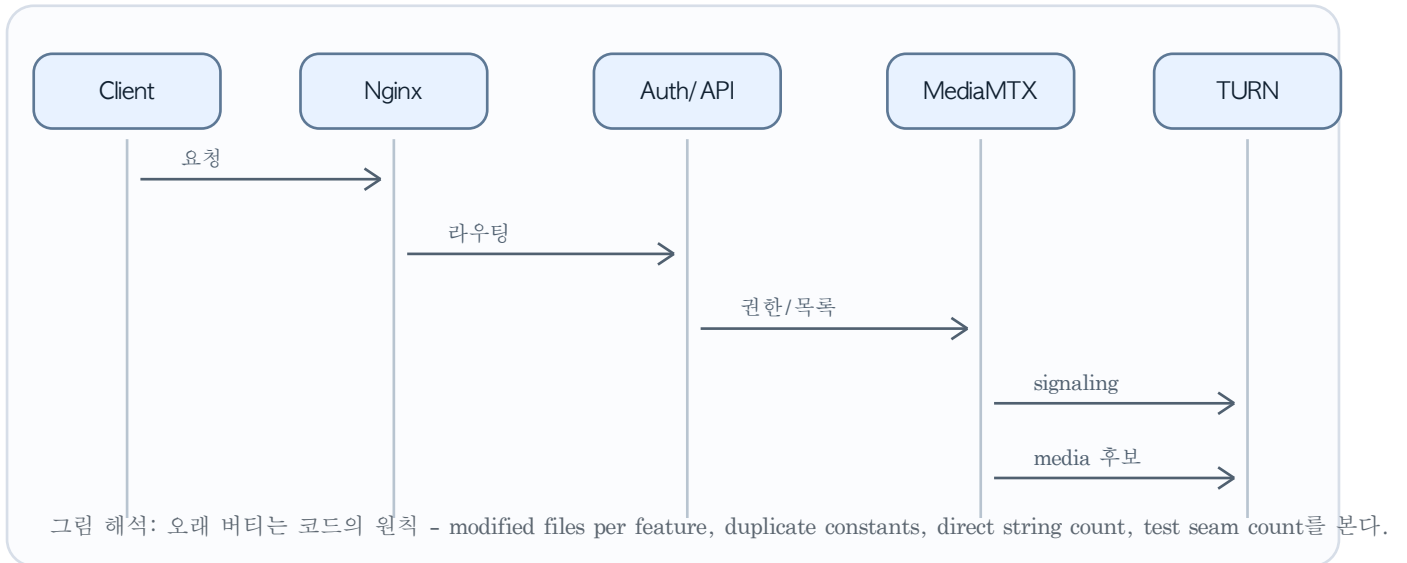
modified files per feature, duplicate constants, direct string count, test seam count를 본다.

기억할 문장

- SOLID와 OCP은 '오래 버티는 코드의 원칙'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

19. SOLID와 OCP - 요청/데이터 흐름

오래 버티는 코드의 원칙



흐름 읽기

요청은 항상 입구, 정책 판단, 실제 처리, 응답의 순서로 본다. SOLID와 OCP에서도 같은 순서로 로그를 따라가면 원인 파악이 빨라진다.

확인할 데이터

modified files per feature, duplicate constants, direct string count, test seam count를 본다.

다음 단계

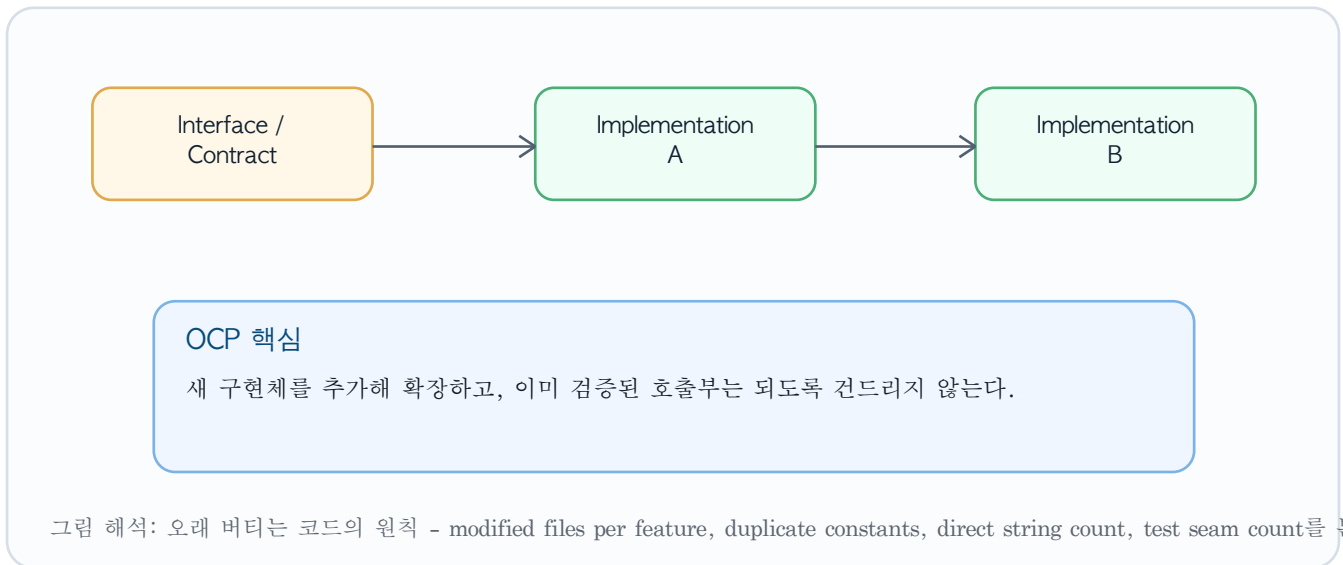
하드코딩된 endpoint 하나를 contract 파일로 옮기고 테스트를 고쳐 본다.

기억할 문장

- SOLID와 OCP은 '오래 버티는 코드의 원칙'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

19. SOLID와 OCP - 코드 작성 원칙

오래 버티는 코드의 원칙



코드 원칙

OCP를 지키려면 새 구현체를 추가할 수 있게 하고 기존 호출부 수정을 줄인다.

설계 연결

SRP는 책임을 나누고, OCP는 새 구현체를 붙이기 쉽게 만들며, DIP는 구체 구현 대신 interface를 바라보게 만든다.

작은 예시

문자열 endpoint를 컨트롤러에 직접 쓰지 말고 RouteContract 또는 ApiRoutes 같은 상수 객체에 모은다.

기억할 문장

- SOLID와 OCP은 '오래 버티는 코드의 원칙'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

19. SOLID와 OCP - 테스트와 검증

오래 버티는 코드의 원칙

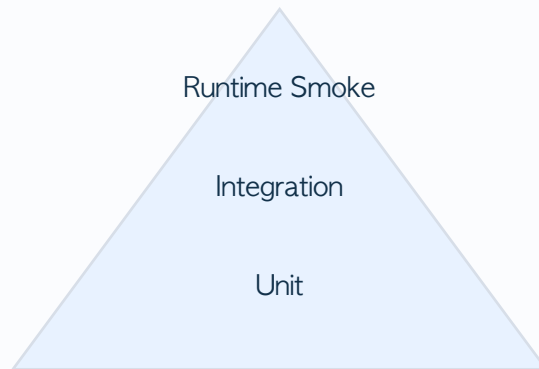


그림 해석: 오래 버티는 코드의 원칙 - modified files per feature, duplicate constants, direct string count, test seam count를 본다.

테스트 원칙

contract test와 mock implementation test로 추상화가 실제로 교체 가능한지 확인한다.

TDD 관점

먼저 실패하는 계약 테스트를 만들고, 그 계약을 만족하는 가장 작은 구현을 작성한 뒤, 실제 runtime smoke로 연결을 확인한다.

통과 기준

modified files per feature, duplicate constants, direct string count, test seam count를 본다.가 기대 범위 안에 있고, 실패 시 원인이 로그와 테스트 이름으로 드러나야 한다.

기억할 문장

- SOLID와 OCP은 '오래 버티는 코드의 원칙'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

19. SOLID와 OCP - 운영 체크

오래 버티는 코드의 원칙

이 페이지의 핵심

기능이 많아질수록 무작정 고치는 코드는 깨지기 쉽다. SOLID는 변경을 작게 만드는 기준이다.

운영 원칙

PR에는 어떤 원칙을 왜 적용했는지, 어떤 변경 범위가 줄었는지 적는다.

장애 대응

status code, container health, application log, network path,
secret/config 누락을 순서대로 확인한다.

기록할 것

명령어, 실행 시각, 결과, 실패 원인, 수정 파일, 남은 리스크를 보고서에 남긴다.

기억할 문장

- SOLID와 OCP은 '오래 버티는 코드의 원칙'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

19. SOLID와 OCP - 자주 나는 실수

오래 버티는 코드의 원칙

이 페이지의 핵심

기능이 많아질수록 무작정 고치는 코드는 깨지기 쉽다. SOLID는 변경을 작게 만드는 기준이다.

자주 나는 실수

패턴 이름만 붙이고 실제 변경 비용이 줄지 않으면 설계가 아니라 장식이다.

왜 위험한가

초반에는 빠르게 보이지만, 운영 중 장애 원인을 숨기거나 변경 범위를 키워 결국 더 오래 걸리게 만든다.

예방 습관

계약값 중앙화, DTO 분리, 테스트 추가, 로그/지표 확인을 PR 기준으로 둔다.

기억할 문장

- SOLID와 OCP은 '오래 버티는 코드의 원칙'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

19. SOLID와 OCP - 초보자 실습

오래 버티는 코드의 원칙

이 페이지의 핵심

기능이 많아질수록 무작정 고치는 코드는 깨지기 쉽다. SOLID는 변경을 작게 만드는 기준이다.

따라 해보기

하드코딩된 endpoint 하나를 contract 파일로 옮기고 테스트를 고쳐 본다.

관찰하기

작업 전후로 modified files per feature, duplicate constants, direct string count, test seam count를 본다.를 기록한다. 숫자를 모르면 개선인지 착각인지 구분할 수 없다.

질문 만들기

'SOLID와 OCP이 깨지면 사용자는 무엇을 보게 되는가?'를 한 문장으로 답해 본다.

기억할 문장

- SOLID와 OCP은 '오래 버티는 코드의 원칙'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

19. SOLID와 OCP - 복습 체크리스트

오래 버티는 코드의 원칙

이 페이지의 핵심

기능이 많아질수록 무작정 고치는 코드는 깨지기 쉽다. SOLID는 변경을 작게 만드는 기준이다.

복습 질문

SOLID와 OCP은 control plane, media plane, data plane, ops plane 중 어디에 가까운가?

체크리스트

경로가 상수화되어 있는가? DTO가 분리되어 있는가? 실패 테스트가 있는가? 운영 지표가 있는가? secret이 노출되지 않았는가?

다음 학습

관련 코드 파일 하나와 테스트 파일 하나를 함께 열어 구현과 검증이 어떻게 연결되는지 읽는다.

기억할 문장

- SOLID와 OCP은 '오래 버티는 코드의 원칙'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

20. 객체지향과 디자인 패턴 - 개념 지도

복잡한 코드를 이름 있는 구조로 정리하기

이 페이지의 핵심

디자인 패턴은 자주 반복되는 문제를 검증된 형태로 푸는 방법이다.

왜 필요한가

디자인 패턴은 자주 반복되는 문제를 검증된 형태로 푸는 방법이다.

서버에서 맡는 역할

Proxy, Adapter, Strategy, Repository, Factory Method, Facade, Observer, DI가 실제로 쓰인다.

초보자 비유

요리 레시피처럼 문제별로 자주 쓰는 구조가 정리되어 있다고 보면 된다.

기억할 문장

- 객체지향과 디자인 패턴은 '복잡한 코드를 이름 있는 구조로 정리하기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

20. 객체지향과 디자인 패턴 - 서버 적용 위치

복잡한 코드를 이름 있는 구조로 정리하기

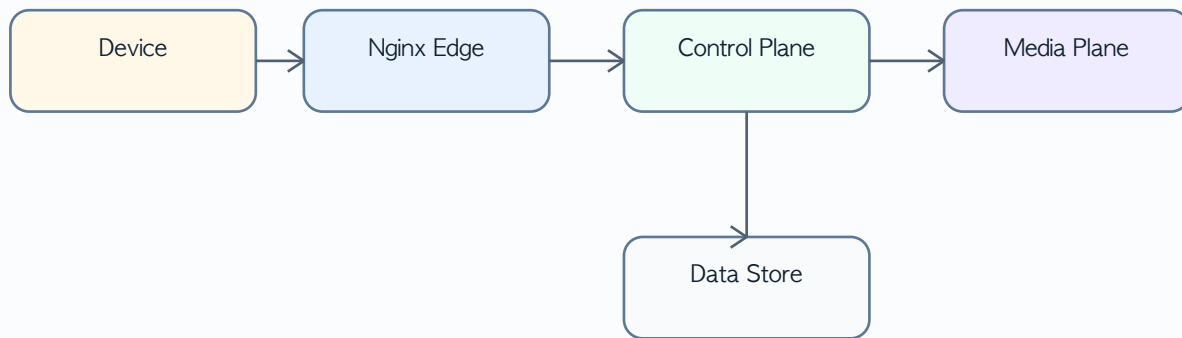


그림 해석: 복잡한 코드를 이름 있는 구조로 정리하기 - interface count, adapter coverage, duplicated creation rule, constructor validation

서버 적용 위치

Proxy, Adapter, Strategy, Repository, Factory Method, Facade, Observer, DI가 실제로 쓰인다.

연결해서 볼 것

객체지향과 디자인 패턴은 단독 기술이 아니라 Nginx, Docker Compose, 인증, 미디어 경로, 테스트와 함께 동작한다.

운영 지표

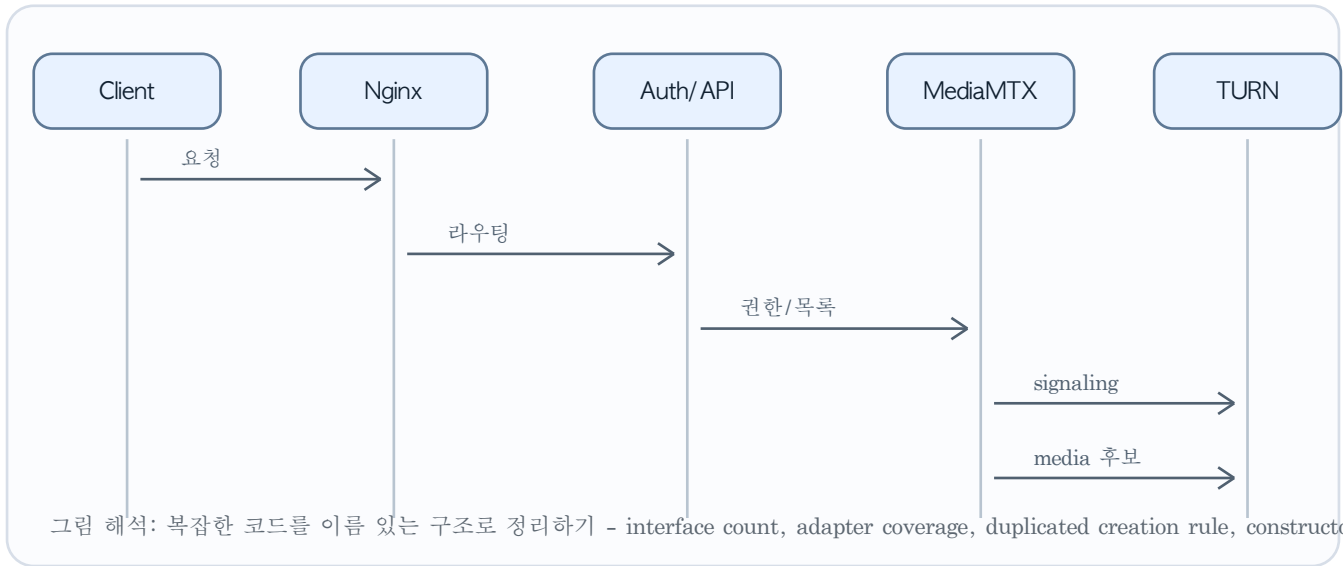
interface count, adapter coverage, duplicated creation rule, constructor validation을 본다.

기억할 문장

- 객체지향과 디자인 패턴은 '복잡한 코드를 이름 있는 구조로 정리하기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

20. 객체지향과 디자인 패턴 - 요청/데이터 흐름

복잡한 코드를 이름 있는 구조로 정리하기



흐름 읽기

요청은 항상 입구, 정책 판단, 실제 처리, 응답의 순서로 본다. 객체지향과 디자인 패턴에서도 같은 순서로 로그를 따라가면 원인 파악이 빨라진다.

확인할 데이터

interface count, adapter coverage, duplicated creation rule, constructor validation을 본다.

다음 단계

Repository와 Factory Method를 작은 예제로 직접 만들어 본다.

기억할 문장

- 객체지향과 디자인 패턴은 '복잡한 코드를 이름 있는 구조로 정리하기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

20. 객체지향과 디자인 패턴 - 코드 작성 원칙

복잡한 코드를 이름 있는 구조로 정리하기



OCP 핵심

새 구현체를 추가해 확장하고, 이미 검증된 호출부는 되도록 건드리지 않는다.

그림 해석: 복잡한 코드를 이름 있는 구조로 정리하기 - interface count, adapter coverage, duplicated creation rule, constructor validation

코드 원칙

패턴을 먼저 고르지 말고 중복, 변경점, 테스트 어려움을 보고 필요한 패턴만 도입한다.

설계 연결

SRP는 책임을 나누고, OCP는 새 구현체를 붙이기 쉽게 만들며, DIP는 구체 구현 대신 interface를 바라보게 만든다.

작은 예시

문자열 endpoint를 컨트롤러에 직접 쓰지 말고 RouteContract 또는 ApiRoutes 같은 상수 객체에 모은다.

기억할 문장

- 객체지향과 디자인 패턴은 '복잡한 코드를 이름 있는 구조로 정리하기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

20. 객체지향과 디자인 패턴 - 테스트와 검증

복잡한 코드를 이름 있는 구조로 정리하기

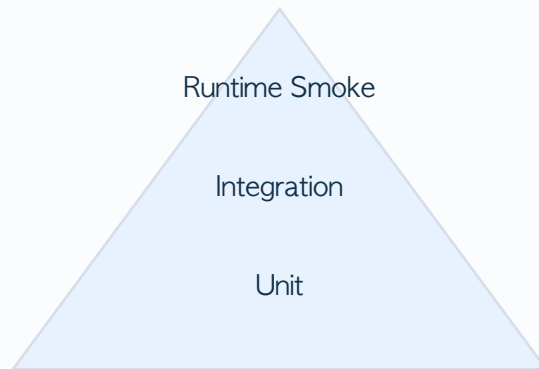


그림 해석: 복잡한 코드를 이름 있는 구조로 정리하기 - interface count, adapter coverage, duplicated creation rule, constructor validation

테스트 원칙

구현체 교체 테스트, factory validation test, adapter contract test를 둔다.

TDD 관점

먼저 실패하는 계약 테스트를 만들고, 그 계약을 만족하는 가장 작은 구현을 작성한 뒤, 실제 runtime smoke로 연결을 확인한다.

통과 기준

interface count, adapter coverage, duplicated creation rule, constructor validation을 본다.가 기대 범위 안에 있고, 실패 시 원인이 로그와 테스트 이름으로 드러나야 한다.

기억할 문장

- 객체지향과 디자인 패턴은 '복잡한 코드를 이름 있는 구조로 정리하기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

20. 객체지향과 디자인 패턴 - 운영 체크

복잡한 코드를 이름 있는 구조로 정리하기

이 페이지의 핵심

디자인 패턴은 자주 반복되는 문제를 검증된 형태로 푸는 방법이다.

운영 원칙

운영에 영향을 주는 패턴 변경은 로그와 장애 대응 흐름도 같이 확인한다.

장애 대응

status code, container health, application log, network path,
secret/config 누락을 순서대로 확인한다.

기록할 것

명령어, 실행 시각, 결과, 실패 원인, 수정 파일, 남은 리스크를 보고서에 남긴다.

기억할 문장

- 객체지향과 디자인 패턴은 '복잡한 코드를 이름 있는 구조로 정리하기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

20. 객체지향과 디자인 패턴 - 자주 나는 실수

복잡한 코드를 이름 있는 구조로 정리하기

이 페이지의 핵심

디자인 패턴은 자주 반복되는 문제를 검증된 형태로 푸는 방법이다.

자주 나는 실수

패턴을 과하게 쓰면 코드가 더 숨고 초보자가 읽기 어려워진다.

왜 위험한가

초반에는 빠르게 보이지만, 운영 중 장애 원인을 숨기거나 변경 범위를 키워 결국 더 오래 걸리게 만든다.

예방 습관

계약값 중앙화, DTO 분리, 테스트 추가, 로그/지표 확인을 PR 기준으로 둔다.

기억할 문장

- 객체지향과 디자인 패턴은 '복잡한 코드를 이름 있는 구조로 정리하기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

20. 객체지향과 디자인 패턴 - 초보자 실습

복잡한 코드를 이름 있는 구조로 정리하기

이 페이지의 핵심

디자인 패턴은 자주 반복되는 문제를 검증된 형태로 푸는 방법이다.

따라 해보기

Repository와 Factory Method를 작은 예제로 직접 만들어 본다.

관찰하기

작업 전후로 interface count, adapter coverage, duplicated creation rule, constructor validation을 본다.를 기록한다. 숫자를 모르면 개선인지 착각인지 구분할 수 없다.

질문 만들기

'객체지향과 디자인 패턴이 깨지면 사용자는 무엇을 보게 되는가?'를 한 문장으로 답해 본다.

기억할 문장

- 객체지향과 디자인 패턴은 '복잡한 코드를 이름 있는 구조로 정리하기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

20. 객체지향과 디자인 패턴 - 복습 체크리스트

복잡한 코드를 이름 있는 구조로 정리하기

이 페이지의 핵심

디자인 패턴은 자주 반복되는 문제를 검증된 형태로 푸는 방법이다.

복습 질문

객체지향과 디자인 패턴은 control plane, media plane, data plane, ops plane 중 어디에 가까운가?

체크리스트

경로가 상수화되어 있는가? DTO가 분리되어 있는가? 실패 테스트가 있는가? 운영 지표가 있는가? secret이 노출되지 않았는가?

다음 학습

관련 코드 파일 하나와 테스트 파일 하나를 함께 열어 구현과 검증이 어떻게 연결되는지 읽는다.

기억할 문장

- 객체지향과 디자인 패턴은 '복잡한 코드를 이름 있는 구조로 정리하기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

21. DTO, VO, API Contract - 개념 지도

외부 약속과 내부 규칙 분리

이 페이지의 핵심

API는 한번 공개되면 클라이언트와의 약속이 되므로 field와 route를 쉽게 바꾸면 안 된다.

왜 필요한가

API는 한번 공개되면 클라이언트와의 약속이 되므로 field와 route를 쉽게 바꾸면 안 된다.

서버에서 맡는 역할

DTO는 외부 JSON 계약, VO는 의미 있는 값, domain model은 내부 규칙을 담당한다.

초보자 비유

택배 송장 양식과 창고 내부 정리 규칙이 다른 것과 같다.

기억할 문장

- DTO, VO, API Contract은 '외부 약속과 내부 규칙 분리'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

21. DTO, VO, API Contract - 서버 적용 위치

외부 약속과 내부 규칙 분리

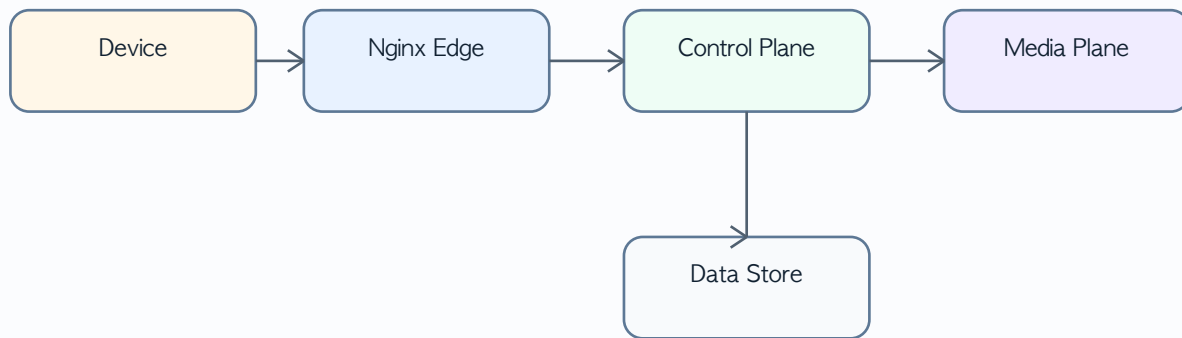


그림 해석: 외부 약속과 내부 규칙 분리 - contract drift, missing field, undocumented route, deprecated call count를 본다.

서버 적용 위치

DTO는 외부 JSON 계약, VO는 의미 있는 값, domain model은 내부 규칙을 담당한다.

연결해서 볼 것

DTO, VO, API Contract은 단독 기술이 아니라 Nginx, Docker Compose, 인증, 미디어 경로, 테스트와 함께 동작한다.

운영 지표

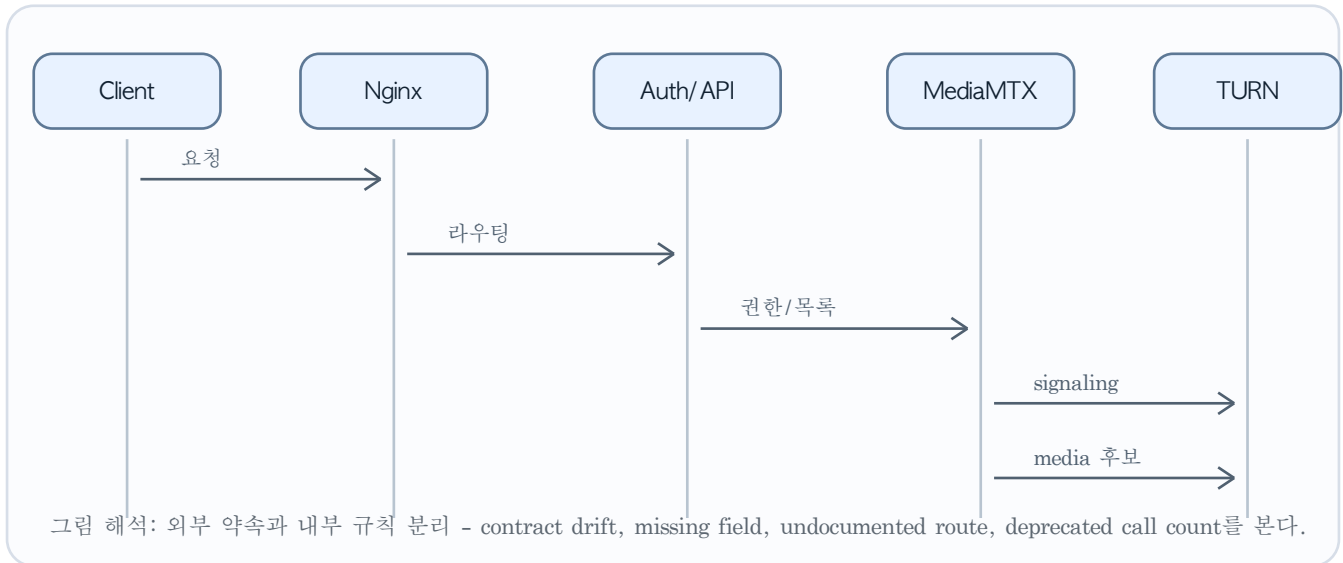
contract drift, missing field, undocumented route, deprecated call count를 본다.

기억할 문장

- DTO, VO, API Contract은 '외부 약속과 내부 규칙 분리'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

21. DTO, VO, API Contract - 요청/데이터 흐름

외부 약속과 내부 규칙 분리



흐름 읽기

요청은 항상 입구, 정책 판단, 실제 처리, 응답의 순서로 본다. DTO, VO, API Contract에서도 같은 순서로 로그를 따라가면 원인 파악이 빨라진다.

확인할 데이터

contract drift, missing field, undocumented route, deprecated call count를 본다.

다음 단계

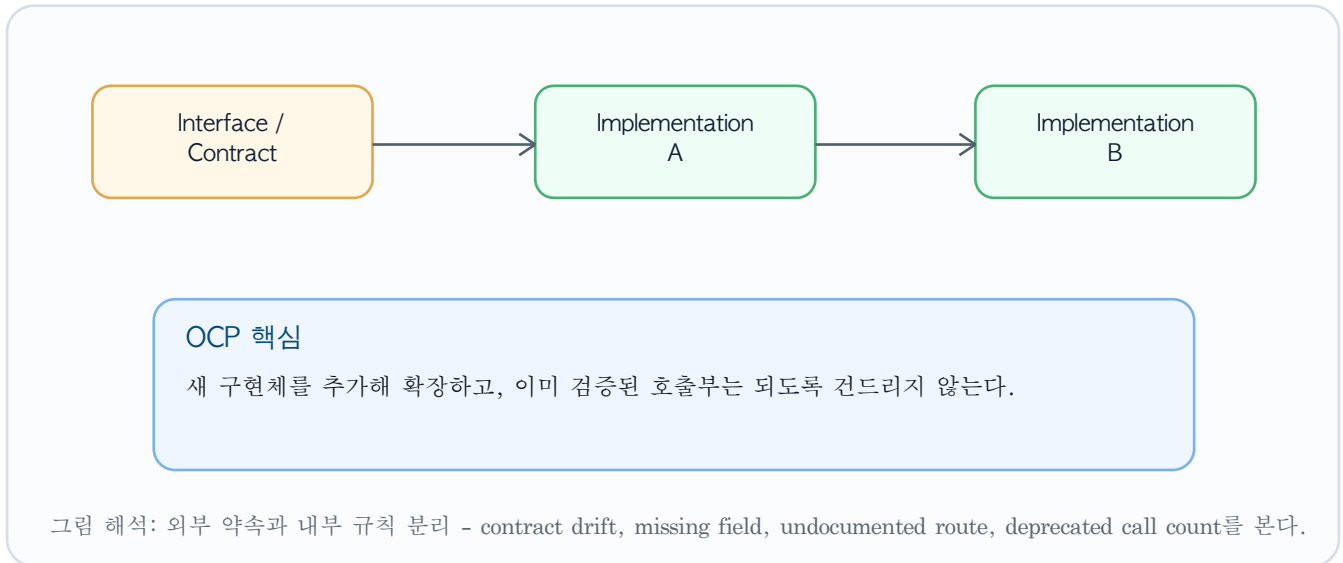
하나의 response JSON을 DTO, VO, domain model로 나눠 그려 본다.

기억할 문장

- DTO, VO, API Contract은 '외부 약속과 내부 규칙 분리'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

21. DTO, VO, API Contract - 코드 작성 원칙

외부 약속과 내부 규칙 분리



코드 원칙

endpoint, header, error, DTO field, protocol string은 contract/config/constants로 분리한다.

설계 연결

SRP는 책임을 나누고, OCP는 새 구현체를 붙이기 쉽게 만들며, DIP는 구체 구현 대신 interface를 바라보게 만든다.

작은 예시

문자열 endpoint를 컨트롤러에 직접 쓰지 말고 RouteContract 또는 ApiRoutes 같은 상수 객체에 모은다.

기억할 문장

- DTO, VO, API Contract은 '외부 약속과 내부 규칙 분리'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

21. DTO, VO, API Contract - 테스트와 검증

외부 약속과 내부 규칙 분리

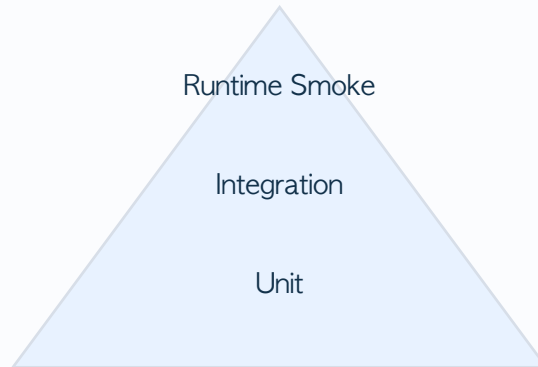


그림 해석: 외부 약속과 내부 규칙 분리 - contract drift, missing field, undocumented route, deprecated call count를 본다.

테스트 원칙

route contract test, DTO field contract test, integration response test를 둔다.

TDD 관점

먼저 실패하는 계약 테스트를 만들고, 그 계약을 만족하는 가장 작은 구현을 작성한 뒤, 실제 runtime smoke로 연결을 확인한다.

통과 기준

contract drift, missing field, undocumented route, deprecated call count를 본다.가 기대 범위 안에 있고, 실패 시 원인이 로그와 테스트 이름으로 드러나야 한다.

기억할 문장

- DTO, VO, API Contract은 '외부 약속과 내부 규칙 분리'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

21. DTO, VO, API Contract - 운영 체크

외부 약속과 내부 규칙 분리

이 페이지의 핵심

API는 한번 공개되면 클라이언트와의 약속이 되므로 field와 route를 쉽게 바꾸면 안 된다.

운영 원칙

버전 변경과 deprecated route에는 문서와 replacement route를 함께 제공한다.

장애 대응

status code, container health, application log, network path, secret/config 누락을 순서대로 확인한다.

기록할 것

명령어, 실행 시각, 결과, 실패 원인, 수정 파일, 남은 리스크를 보고서에 남긴다.

기억할 문장

- DTO, VO, API Contract은 '외부 약속과 내부 규칙 분리'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

21. DTO, VO, API Contract - 자주 나는 실수

외부 약속과 내부 규칙 분리

이 페이지의 핵심

API는 한번 공개되면 클라이언트와의 약속이 되므로 field와 route를 쉽게 바꾸면 안 된다.

자주 나는 실수

컨트롤러에서 DB entity를 그대로 반환하면 내부 구조가 외부 계약이 되어버린다.

왜 위험한가

초반에는 빠르게 보이지만, 운영 중 장애 원인을 숨기거나 변경 범위를 키워 결국 더 오래 걸리게 만든다.

예방 습관

계약값 중앙화, DTO 분리, 테스트 추가, 로그/지표 확인을 PR 기준으로 둔다.

기억할 문장

- DTO, VO, API Contract은 '외부 약속과 내부 규칙 분리'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

21. DTO, VO, API Contract - 초보자 실습

외부 약속과 내부 규칙 분리

이 페이지의 핵심

API는 한번 공개되면 클라이언트와의 약속이 되므로 field와 route를 쉽게 바꾸면 안 된다.

따라 해보기

하나의 response JSON을 DTO, VO, domain model로 나눠 그려 본다.

관찰하기

작업 전후로 contract drift, missing field, undocumented route, deprecated call count를 본다.를 기록한다. 숫자를 모르면 개선인지 착각인지 구분할 수 없다.

질문 만들기

'DTO, VO, API Contract이 깨지면 사용자는 무엇을 보게 되는가?'를 한 문장으로 답해 본다.

기억할 문장

- DTO, VO, API Contract은 '외부 약속과 내부 규칙 분리'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

21. DTO, VO, API Contract - 복습 체크리스트

외부 약속과 내부 규칙 분리

이 페이지의 핵심

API는 한번 공개되면 클라이언트와의 약속이 되므로 field와 route를 쉽게 바꾸면 안 된다.

복습 질문

DTO, VO, API Contract은 control plane, media plane, data plane, ops plane 중 어디에 가까운가?

체크리스트

경로가 상수화되어 있는가? DTO가 분리되어 있는가? 실패 테스트가 있는가? 운영 지표가 있는가? secret이 노출되지 않았는가?

다음 학습

관련 코드 파일 하나와 테스트 파일 하나를 함께 열어 구현과 검증이 어떻게 연결되는지 읽는다.

기억할 문장

- DTO, VO, API Contract은 '외부 약속과 내부 규칙 분리'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

22. 동시성과 멀티스레드 - 개념 지도

동시에 일어나면 안 되는 일을 막기

이 페이지의 핵심

로그인 refresh, stream registry update, telemetry upsert는 동시에 요청될 수 있다.

왜 필요한가

로그인 refresh, stream registry update, telemetry upsert는 동시에 요청될 수 있다.

서버에서 맡는 역할

Redis atomic operation, synchronized, lock scope, idempotent update로 경쟁 상태를 줄인다.

초보자 비유

두 사람이 동시에 같은 좌석을 예매하지 못하게 막는 원리와 비슷하다.

기억할 문장

- 동시성과 멀티스레드는 '동시에 일어나면 안 되는 일을 막기'를 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

22. 동시성과 멀티스레드 - 서버 적용 위치

동시에 일어나면 안 되는 일을 막기

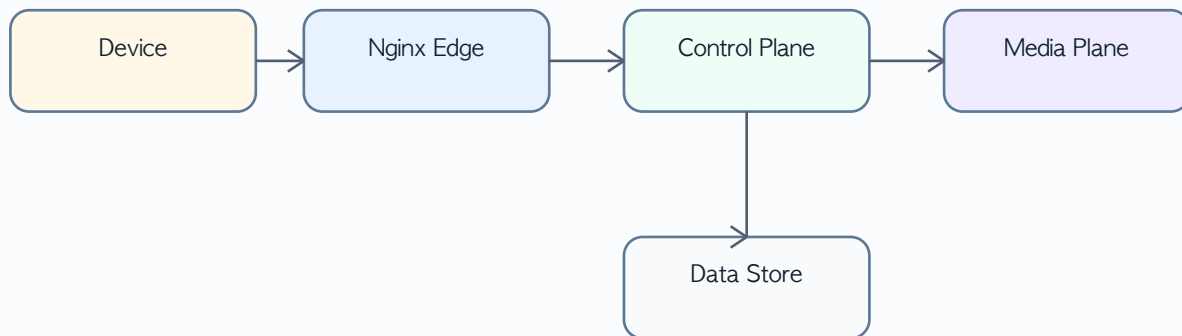


그림 해석: 동시에 일어나면 안 되는 일을 막기 - lock wait, duplicate update, lost update, Redis command atomicity를 본다.

서버 적용 위치

Redis atomic operation, synchronized, lock scope, idempotent update로 경쟁 상태를 줄인다.

연결해서 볼 것

동시성과 멀티스레드는 단독 기술이 아니라 Nginx, Docker Compose, 인증, 미디어 경로, 테스트와 함께 동작한다.

운영 지표

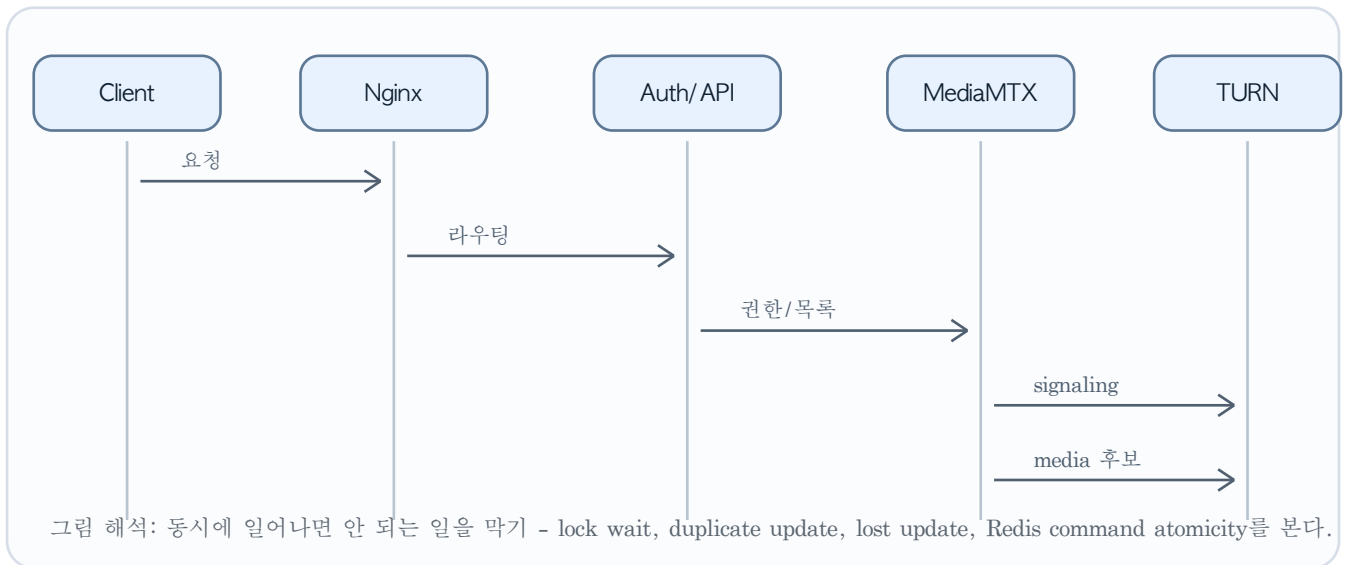
lock wait, duplicate update, lost update, Redis command atomicity를 본다.

기억할 문장

- 동시성과 멀티스레드는 '동시에 일어나면 안 되는 일을 막기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

22. 동시성과 멀티스레드 - 요청/데이터 흐름

동시에 일어나면 안 되는 일을 막기



흐름 읽기

요청은 항상 입구, 정책 판단, 실제 처리, 응답의 순서로 본다. 동시성과 멀티스레드에서도 같은 순서로 로그를 따라가 원인 파악이 빨라진다.

확인할 데이터

lock wait, duplicate update, lost update, Redis command atomicity를 본다.

다음 단계

같은 refresh token을 동시에 두 번 쓰는 테스트를 설계해 본다.

기억할 문장

- 동시성과 멀티스레드는 '동시에 일어나면 안 되는 일을 막기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

22. 동시성과 멀티스레드 - 코드 작성 원칙

동시에 일어나면 안 되는 일을 막기



OCP 핵심

새 구현체를 추가해 확장하고, 이미 검증된 호출부는 되도록 건드리지 않는다.

그림 해석: 동시에 일어나면 안 되는 일을 막기 - lock wait, duplicate update, lost update, Redis command atomicity를 본다.

코드 원칙

공유 상태는 작게 만들고, 불변 객체와 일급 컬렉션으로 변경 범위를 줄인다.

설계 연결

SRP는 책임을 나누고, OCP는 새 구현체를 붙이기 쉽게 만들며, DIP는 구체 구현 대신 interface를 바라보게 만든다.

작은 예시

문자열 endpoint를 컨트롤러에 직접 쓰지 말고 RouteContract 또는 ApiRoutes 같은 상수 객체에 모은다.

기억할 문장

- 동시성과 멀티스레드는 '동시에 일어나면 안 되는 일을 막기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

22. 동시성과 멀티스레드 - 테스트와 검증

동시에 일어나면 안 되는 일을 막기

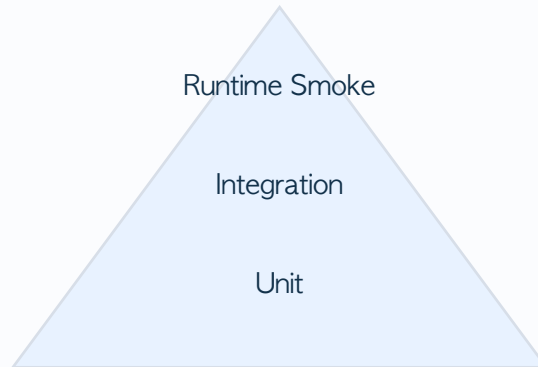


그림 해석: 동시에 일어나면 안 되는 일을 막기 - lock wait, duplicate update, lost update, Redis command atomicity를 본다.

테스트 원칙

refresh token consume, stream registry update, telemetry upsert
concurrency test를 둔다.

TDD 관점

먼저 실패하는 계약 테스트를 만들고, 그 계약을 만족하는 가장 작은 구현을 작성한 뒤, 실제 runtime smoke로 연결을 확인한다.

통과 기준

lock wait, duplicate update, lost update, Redis command atomicity를
본다.가 기대 범위 안에 있고, 실패 시 원인이 로그와 테스트 이름으로 드러나야 한다.

기억할 문장

- 동시성과 멀티스레드는 '동시에 일어나면 안 되는 일을 막기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

22. 동시성과 멀티스레드 - 운영 체크

동시에 일어나면 안 되는 일을 막기

이 페이지의 핵심

로그인 refresh, stream registry update, telemetry upsert는 동시에 요청될 수 있다.

운영 원칙

락은 너무 넓으면 병목이 되고 너무 좁으면 데이터가 깨질 수 있다.

장애 대응

status code, container health, application log, network path, secret/config 누락을 순서대로 확인한다.

기록할 것

명령어, 실행 시각, 결과, 실패 원인, 수정 파일, 남은 리스크를 보고서에 남긴다.

기억할 문장

- 동시성과 멀티스레드는 '동시에 일어나면 안 되는 일을 막기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

22. 동시성과 멀티스레드 - 자주 나는 실수

동시에 일어나면 안 되는 일을 막기

이 페이지의 핵심

로그인 refresh, stream registry update, telemetry upsert는 동시에 요청될 수 있다.

자주 나는 실수

테스트가 단일 요청만 보면 동시성 버그는 운영에서 처음 터진다.

왜 위험한가

초반에는 빠르게 보이지만, 운영 중 장애 원인을 숨기거나 변경 범위를 키워 결국 더 오래 걸리게 만든다.

예방 습관

계약값 중앙화, DTO 분리, 테스트 추가, 로그/지표 확인을 PR 기준으로 둔다.

기억할 문장

- 동시성과 멀티스레드는 '동시에 일어나면 안 되는 일을 막기'를 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

22. 동시성과 멀티스레드 - 초보자 실습

동시에 일어나면 안 되는 일을 막기

이 페이지의 핵심

로그인 refresh, stream registry update, telemetry upsert는 동시에 요청될 수 있다.

따라 해보기

같은 refresh token을 동시에 두 번 쓰는 테스트를 설계해 본다.

관찰하기

작업 전후로 lock wait, duplicate update, lost update, Redis command atomicity를 본다.를 기록한다. 숫자를 모르면 개선인지 착각인지 구분할 수 없다.

질문 만들기

'동시성과 멀티스레드가 깨지면 사용자는 무엇을 보게 되는가?'를 한 문장으로 답해 본다.

기억할 문장

- 동시성과 멀티스레드는 '동시에 일어나면 안 되는 일을 막기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

22. 동시성과 멀티스레드 - 복습 체크리스트

동시에 일어나면 안 되는 일을 막기

이 페이지의 핵심

로그인 refresh, stream registry update, telemetry upsert는 동시에 요청될 수 있다.

복습 질문

동시성과 멀티스레드는 control plane, media plane, data plane, ops plane 중 어디에 가까운가?

체크리스트

경로가 상수화되어 있는가? DTO가 분리되어 있는가? 실패 테스트가 있는가? 운영 지표가 있는가? secret이 노출되지 않았는가?

다음 학습

관련 코드 파일 하나와 테스트 파일 하나를 함께 열어 구현과 검증이 어떻게 연결되는지 읽는다.

기억할 문장

- 동시성과 멀티스레드는 '동시에 일어나면 안 되는 일을 막기'를 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

23. 폐쇄망 배포 - 개념 지도

인터넷 없이도 돌아가는 시스템

이 페이지의 핵심

군/산업 현장에서는 인터넷 연결이 제한되므로 외부 API와 registry 의존을 줄여야 한다.

왜 필요한가

군/산업 현장에서는 인터넷 연결이 제한되므로 외부 API와 registry 의존을 줄여야 한다.

서버에서 맡는 역할

자체 STUN/TURN, 내부 지도 타일, 내부 시간 서버, offline package/image 배포가 필요하다.

초보자 비유

섬 안에서도 발전기, 지도, 시계, 창고가 있어야 하는 상황과 같다.

기억할 문장

- 폐쇄망 배포는 '인터넷 없이도 돌아가는 시스템'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

23. 폐쇄망 배포 - 서버 적용 위치

인터넷 없이도 돌아가는 시스템

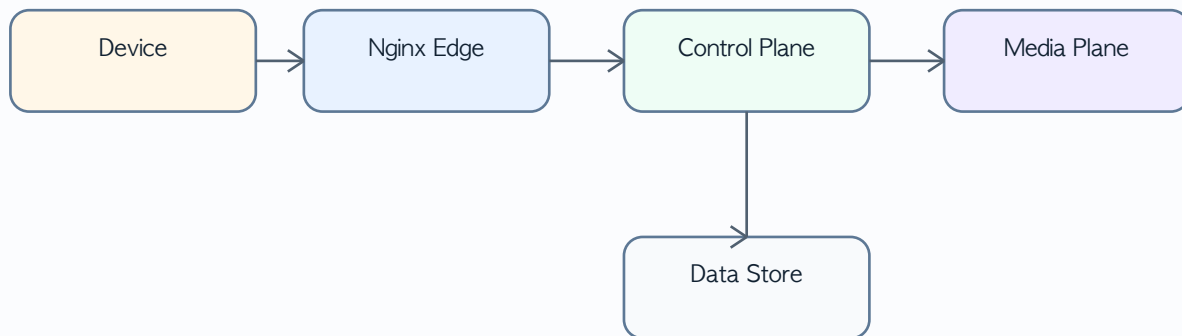


그림 해석: 인터넷 없이도 돌아가는 시스템 - external request count, offline build success, local STUN/TURN latency를 본다.

서버 적용 위치

자체 STUN/TURN, 내부 지도 타일, 내부 시간 서버, offline package/image 배포가 필요하다.

연결해서 볼 것

폐쇄망 배포는 단독 기술이 아니라 Nginx, Docker Compose, 인증, 미디어 경로, 테스트와 함께 동작한다.

운영 지표

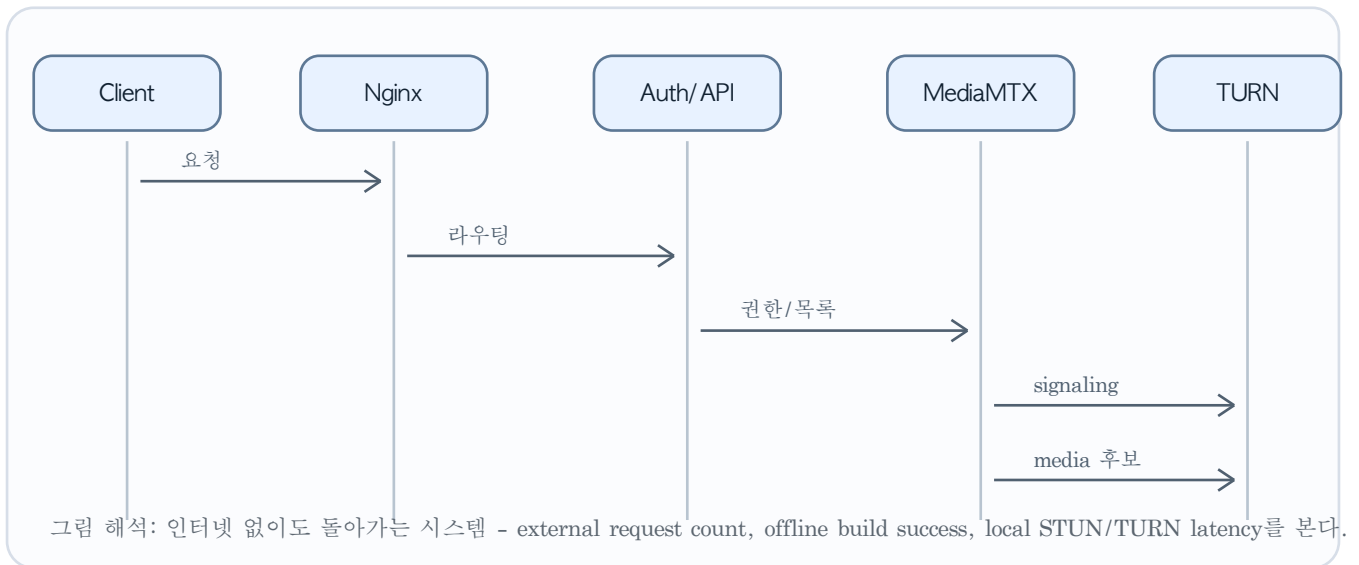
external request count, offline build success, local STUN/TURN latency를 본다.

기억할 문장

- 폐쇄망 배포는 '인터넷 없이도 돌아가는 시스템'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

23. 폐쇄망 배포 - 요청/데이터 흐름

인터넷 없이도 돌아가는 시스템



흐름 읽기

요청은 항상 입구, 정책 판단, 실제 처리, 응답의 순서로 본다. 폐쇄망 배포에서도 같은 순서로 로그를 따라가면 원인 파악이 빨라진다.

확인할 데이터

external request count, offline build success, local STUN/TURN latency를 본다.

다음 단계

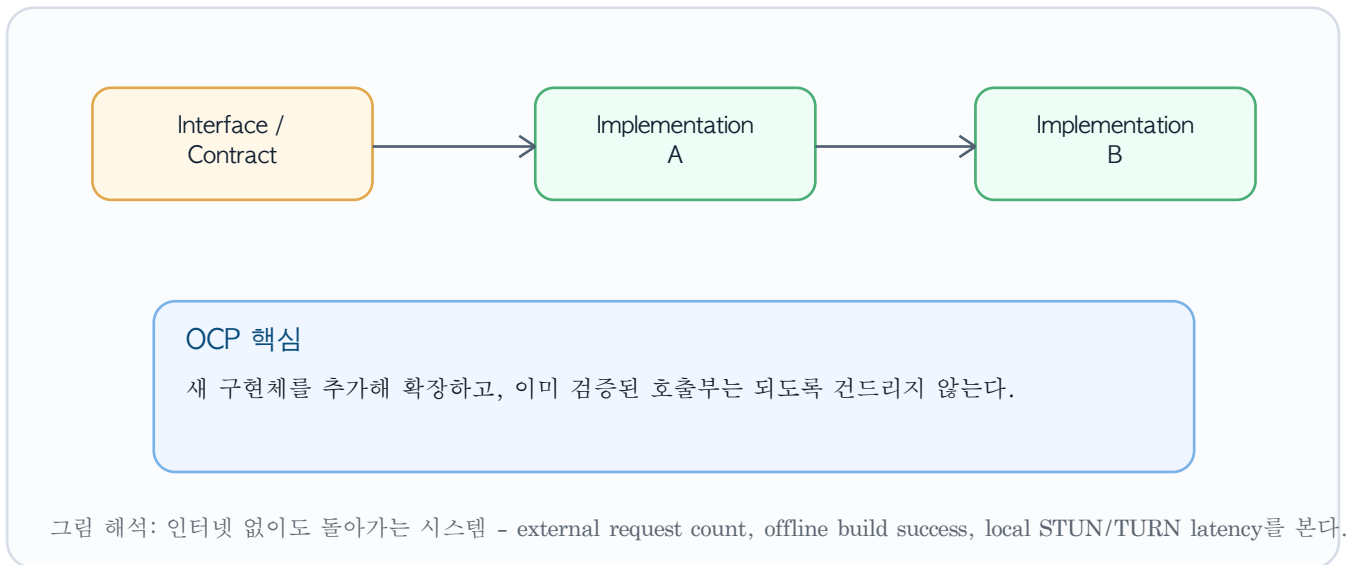
외부 인터넷을 끊었다고 가정하고 필요한 파일 목록을 만들어 본다.

기억할 문장

- 폐쇄망 배포는 '인터넷 없이도 돌아가는 시스템'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

23. 폐쇄망 배포 - 코드 작성 원칙

인터넷 없이도 돌아가는 시스템



코드 원칙

config profile로 public, closed, hybrid 모드를 나누고 코드에는 직접 값을 넣지 않는다.

설계 연결

SRP는 책임을 나누고, OCP는 새 구현체를 붙이기 쉽게 만들며, DIP는 구체 구현 대신 interface를 바라보게 만든다.

작은 예시

문자열 endpoint를 컨트롤러에 직접 쓰지 말고 RouteContract 또는 ApiRoutes 같은 상수 객체에 모은다.

기억할 문장

- 폐쇄망 배포는 '인터넷 없이도 돌아가는 시스템'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

23. 폐쇄망 배포 - 테스트와 검증

인터넷 없이도 돌아가는 시스템

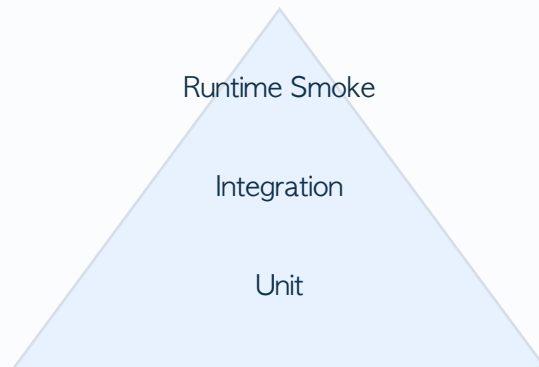


그림 해석: 인터넷 없이도 돌아가는 시스템 - external request count, offline build success, local STUN/TURN latency를 본다.

테스트 원칙

closed-network static check, compose config, image load/run smoke를 둔다.

TDD 관점

먼저 실패하는 계약 테스트를 만들고, 그 계약을 만족하는 가장 작은 구현을 작성한 뒤, 실제 runtime smoke로 연결을 확인한다.

통과 기준

external request count, offline build success, local STUN/TURN latency를 본다.가 기대 범위 안에 있고, 실패 시 원인이 로그와 테스트 이름으로 드러나야 한다.

기억할 문장

- 폐쇄망 배포는 '인터넷 없이도 돌아가는 시스템'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

23. 폐쇄망 배포 - 운영 체크

인터넷 없이도 돌아가는 시스템

이 페이지의 핵심

군/산업 현장에서는 인터넷 연결이 제한되므로 외부 API와 registry 의존을 줄여야 한다.

운영 원칙

Docker가 없는 환경까지 고려하면 설치 패키지, systemd, binary 배포 전략도 필요하다.

장애 대응

status code, container health, application log, network path, secret/config 누락을 순서대로 확인한다.

기록할 것

명령어, 실행 시각, 결과, 실패 원인, 수정 파일, 남은 리스크를 보고서에 남긴다.

기억할 문장

- 폐쇄망 배포는 '인터넷 없이도 돌아가는 시스템'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

23. 폐쇄망 배포 - 자주 나는 실수

인터넷 없이도 돌아가는 시스템

이 페이지의 핵심

군/산업 현장에서는 인터넷 연결이 제한되므로 외부 API와 registry 의존을 줄여야 한다.

자주 나는 실수

개발 중 npm install이나 docker pull에 숨어 있는 인터넷 의존을 놓치기 쉽다.

왜 위험한가

초반에는 빠르게 보이지만, 운영 중 장애 원인을 숨기거나 변경 범위를 키워 결국 더 오래 걸리게 만든다.

예방 습관

계약값 중앙화, DTO 분리, 테스트 추가, 로그/지표 확인을 PR 기준으로 둔다.

기억할 문장

- 폐쇄망 배포는 '인터넷 없이도 돌아가는 시스템'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

23. 폐쇄망 배포 - 초보자 실습

인터넷 없이도 돌아가는 시스템

이 페이지의 핵심

군/산업 현장에서는 인터넷 연결이 제한되므로 외부 API와 registry 의존을 줄여야 한다.

따라 해보기

외부 인터넷을 끊었다고 가정하고 필요한 파일 목록을 만들어 본다.

관찰하기

작업 전후로 external request count, offline build success, local STUN/TURN latency를 본다.를 기록한다. 숫자를 모르면 개선인지 착각인지 구분할 수 없다.

질문 만들기

'폐쇄망 배포이 깨지면 사용자는 무엇을 보게 되는가?'를 한 문장으로 답해 본다.

기억할 문장

- 폐쇄망 배포는 '인터넷 없이도 돌아가는 시스템'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

23. 폐쇄망 배포 - 복습 체크리스트

인터넷 없이도 돌아가는 시스템

이 페이지의 핵심

군/산업 현장에서는 인터넷 연결이 제한되므로 외부 API와 registry 의존을 줄여야 한다.

복습 질문

폐쇄망 배포는 control plane, media plane, data plane, ops plane 중 어디에 가까운가?

체크리스트

경로가 상수화되어 있는가? DTO가 분리되어 있는가? 실패 테스트가 있는가? 운영 지표가 있는가? secret이 노출되지 않았는가?

다음 학습

관련 코드 파일 하나와 테스트 파일 하나를 함께 열어 구현과 검증이 어떻게 연결되는지 읽는다.

기억할 문장

- 폐쇄망 배포는 '인터넷 없이도 돌아가는 시스템'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

24. 성능 튜닝 - 개념 지도

빠르고 덜 낭비하게 만들기

이 페이지의 핵심

실시간 스트리밍은 네트워크, CPU, DB, 브라우저 렌더링이 모두 지연 원인이 될 수 있다.

왜 필요한가

실시간 스트리밍은 네트워크, CPU, DB, 브라우저 렌더링이 모두 지연 원인이 될 수 있다.

서버에서 맡는 역할

WebRTC media path, Redis cache, DB index, React render, Nginx keepalive를 함께 본다.

초보자 비유

도로 정체를 줄일 때 차선, 신호, 운전자, 주차장을 모두 보는 것과 같다.

기억할 문장

- 성능 튜닝은 '빠르고 덜 낭비하게 만들기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

24. 성능 튜닝 - 서버 적용 위치

빠르고 덜 낭비하게 만들기

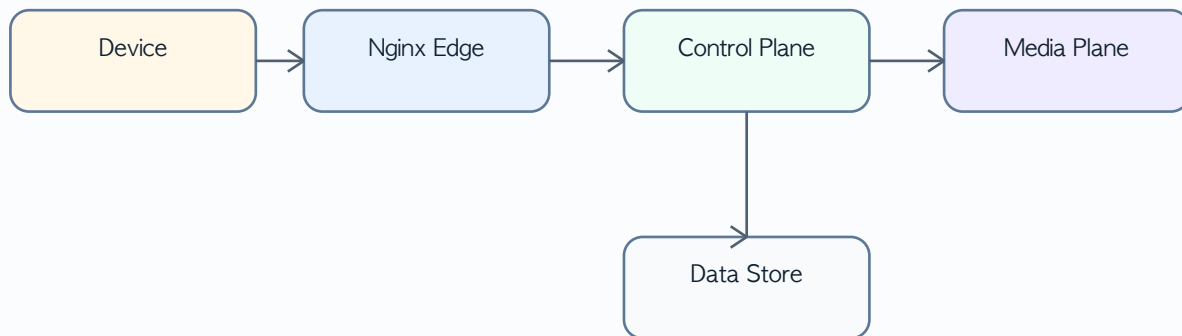


그림 해석: 빠르고 덜 낭비하게 만들기 - p95 latency, CPU, memory, rows examined, render count, packet loss를 본다.

서버 적용 위치

WebRTC media path, Redis cache, DB index, React render, Nginx keepalive를 함께 본다.

연결해서 볼 것

성능 튜닝은 단독 기술이 아니라 Nginx, Docker Compose, 인증, 미디어 경로, 테스트와 함께 동작한다.

운영 지표

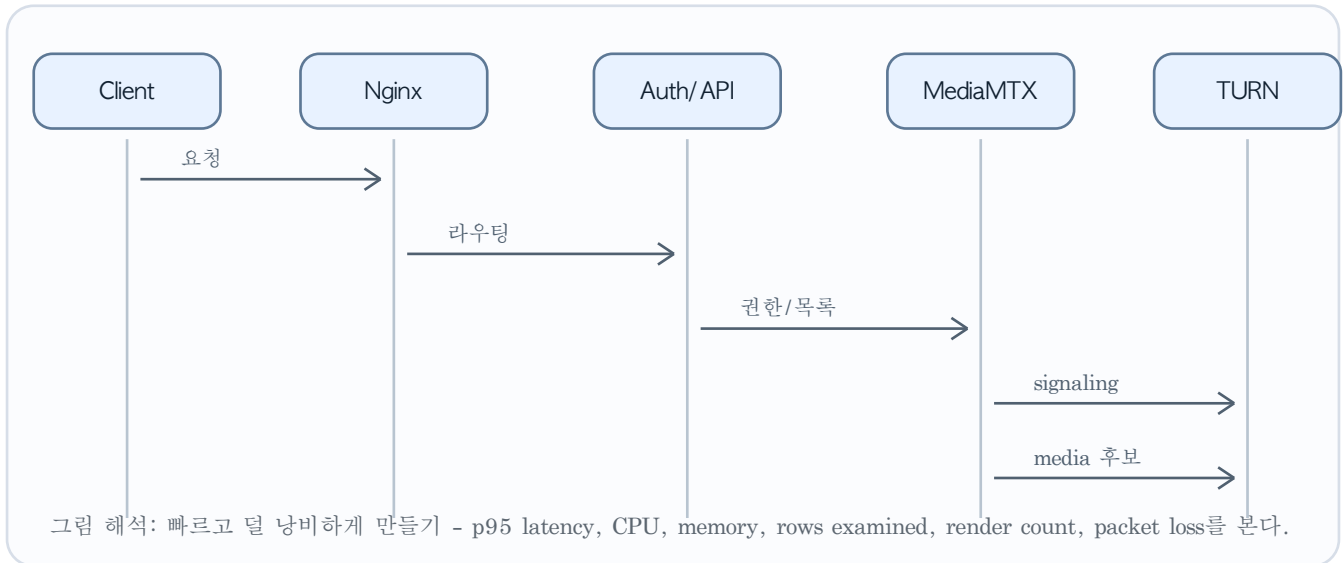
p95 latency, CPU, memory, rows examined, render count, packet loss를 본다.

기억할 문장

- 성능 튜닝은 '빠르고 덜 낭비하게 만들기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

24. 성능 튜닝 - 요청/데이터 흐름

빠르고 덜 낭비하게 만들기



흐름 읽기

요청은 항상 입구, 정책 판단, 실제 처리, 응답의 순서로 본다. 성능 튜닝에서도 같은 순서로 로그를 따라가면 원인 파악이 빨라진다.

확인할 데이터

p95 latency, CPU, memory, rows examined, render count, packet loss를 본다.

다음 단계

하나의 느린 경로를 API, DB, frontend, media로 나눠 시간을 재 본다.

기억할 문장

- 성능 튜닝은 '빠르고 덜 낭비하게 만들기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

24. 성능 튜닝 - 코드 작성 원칙

빠르고 덜 낭비하게 만들기



OCP 핵심

새 구현체를 추가해 확장하고, 이미 검증된 호출부는 되도록 건드리지 않는다.

그림 해석: 빠르고 덜 낭비하게 만들기 - p95 latency, CPU, memory, rows examined, render count, packet loss를 본다.

코드 원칙

먼저 계층하고, 병목을 찾고, 가장 큰 비용부터 줄인다.

설계 연결

SRP는 책임을 나누고, OCP는 새 구현체를 붙이기 쉽게 만들며, DIP는 구체 구현 대신 interface를 바라보게 만든다.

작은 예시

문자열 endpoint를 컨트롤러에 직접 쓰지 말고 RouteContract 또는 ApiRoutes 같은 상수 객체에 모은다.

기억할 문장

- 성능 튜닝은 '빠르고 덜 낭비하게 만들기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

24. 성능 튜닝 - 테스트와 검증

빠르고 덜 낭비하게 만들기

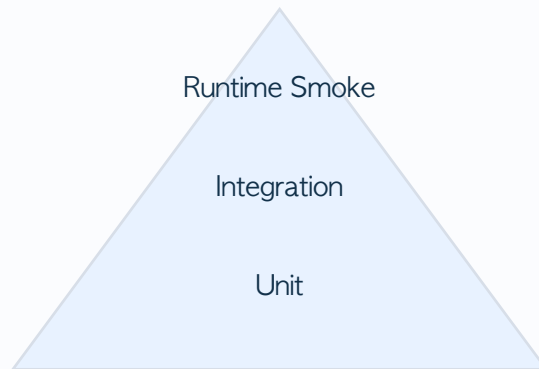


그림 해석: 빠르고 덜 낭비하게 만들기 - p95 latency, CPU, memory, rows examined, render count, packet loss를 본다.

테스트 원칙

benchmark 전후 수치, query plan, render profile, smoke latency를 기록한다.

TDD 관점

먼저 실패하는 계약 테스트를 만들고, 그 계약을 만족하는 가장 작은 구현을 작성한 뒤, 실제 runtime smoke로 연결을 확인한다.

통과 기준

p95 latency, CPU, memory, rows examined, render count, packet loss를 본다.가 기대 범위 안에 있고, 실패 시 원인이 로그와 테스트 이름으로 드러나야 한다.

기억할 문장

- 성능 튜닝은 '빠르고 덜 낭비하게 만들기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

24. 성능 튜닝 - 운영 체크

빠르고 덜 낭비하게 만들기

이 페이지의 핵심

실시간 스트리밍은 네트워크, CPU, DB, 브라우저 렌더링이 모두 지연 원인이 될 수 있다.

운영 원칙

최적화는 기능 변경과 분리해 PR에 근거와 부작용을 함께 적는다.

장애 대응

status code, container health, application log, network path, secret/config 누락을 순서대로 확인한다.

기록할 것

명령어, 실행 시각, 결과, 실패 원인, 수정 파일, 남은 리스크를 보고서에 남긴다.

기억할 문장

- 성능 튜닝은 '빠르고 덜 낭비하게 만들기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

24. 성능 튜닝 - 자주 나는 실수

빠르고 덜 낭비하게 만들기

이 페이지의 핵심

실시간 스트리밍은 네트워크, CPU, DB, 브라우저 렌더링이 모두 지연 원인이 될 수 있다.

자주 나는 실수

측정 없이 넣은 useMemo나 cache는 오히려 복잡도만 늘릴 수 있다.

왜 위험한가

초반에는 빠르게 보이지만, 운영 중 장애 원인을 숨기거나 변경 범위를 키워 결국 더 오래 걸리게 만든다.

예방 습관

계약값 중앙화, DTO 분리, 테스트 추가, 로그/지표 확인을 PR 기준으로 둔다.

기억할 문장

- 성능 튜닝은 '빠르고 덜 낭비하게 만들기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

24. 성능 튜닝 - 초보자 실습

빠르고 덜 낭비하게 만들기

이 페이지의 핵심

실시간 스트리밍은 네트워크, CPU, DB, 브라우저 렌더링이 모두 지연 원인이 될 수 있다.

따라 해보기

하나의 느린 경로를 API, DB, frontend, media로 나눠 시간을 재 본다.

관찰하기

작업 전후로 p95 latency, CPU, memory, rows examined, render count, packet loss를 본다.를 기록한다. 숫자를 모르면 개선인지 착각인지 구분할 수 없다.

질문 만들기

'성능 튜닝이 깨지면 사용자는 무엇을 보게 되는가?'를 한 문장으로 답해 본다.

기억할 문장

- 성능 튜닝은 '빠르고 덜 낭비하게 만들기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

24. 성능 튜닝 - 복습 체크리스트

빠르고 덜 낭비하게 만들기

이 페이지의 핵심

실시간 스트리밍은 네트워크, CPU, DB, 브라우저 렌더링이 모두 지연 원인이 될 수 있다.

복습 질문

성능 튜닝은 control plane, media plane, data plane, ops plane 중 어디에 가까운가?

체크리스트

경로가 상수화되어 있는가? DTO가 분리되어 있는가? 실패 테스트가 있는가? 운영 지표가 있는가? secret이 노출되지 않았는가?

다음 학습

관련 코드 파일 하나와 테스트 파일 하나를 함께 열어 구현과 검증이 어떻게 연결되는지 읽는다.

기억할 문장

- 성능 튜닝은 '빠르고 덜 낭비하게 만들기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

25. 운영 Runbook과 릴리스 - 개념 지도

개발 결과를 안전하게 서버에 올리기

이 페이지의 핵심

코드가 좋아도 배포와 복구 절차가 없으면 운영 시스템으로 보기 어렵다.

왜 필요한가

코드가 좋아도 배포와 복구 절차가 없으면 운영 시스템으로 보기 어렵다.

서버에서 맡는 역할

release tag, PR, issue, server release directory, current symlink, health check가 배포 흐름을 이룬다.

초보자 비유

공연 전 리허설과 비상구 확인까지 해야 실제 무대에 올릴 수 있다.

기억할 문장

- 운영 Runbook과 릴리스는 '개발 결과를 안전하게 서버에 올리기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

25. 운영 Runbook과 릴리스 - 서버 적용 위치

개발 결과를 안전하게 서버에 올리기

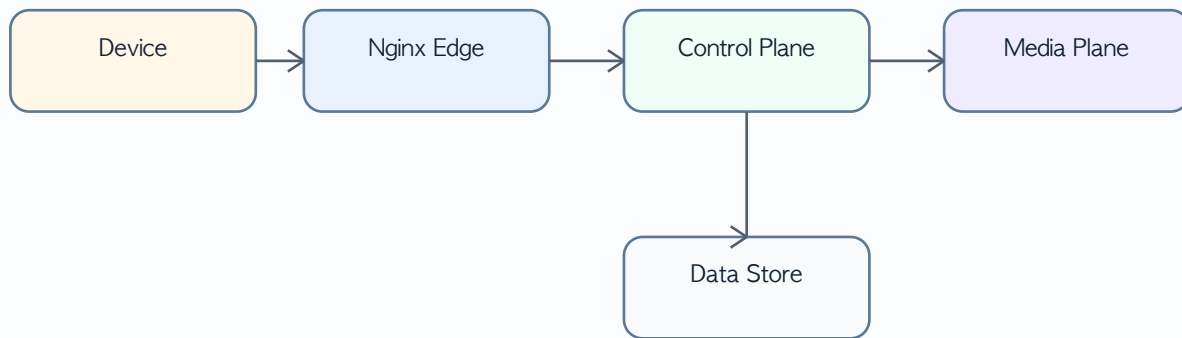


그림 해석: 개발 결과를 안전하게 서버에 올리기 - release sha, image digest, container uptime, external status, rollback time을 본다.

서버 적용 위치

release tag, PR, issue, server release directory, current symlink, health check가 배포 흐름을 이룬다.

연결해서 볼 것

운영 Runbook과 릴리스는 단독 기술이 아니라 Nginx, Docker Compose, 인증, 미디어 경로, 테스트와 함께 동작한다.

운영 지표

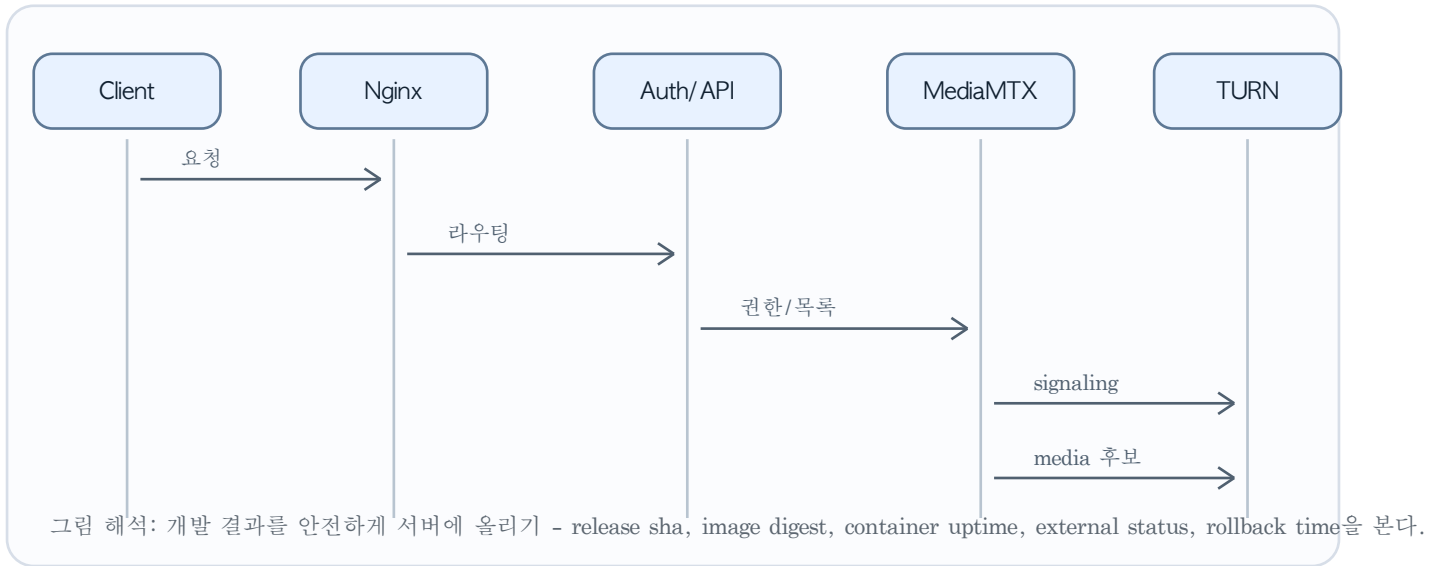
release sha, image digest, container uptime, external status, rollback time을 본다.

기억할 문장

- 운영 Runbook과 릴리스는 '개발 결과를 안전하게 서버에 올리기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

25. 운영 Runbook과 릴리스 - 요청/데이터 흐름

개발 결과를 안전하게 서버에 올리기



흐름 읽기

요청은 항상 입구, 정책 판단, 실제 처리, 응답의 순서로 본다. 운영 Runbook과 릴리스에서도 같은 순서로 로그를 따라가면 원인 파악이 빨라진다.

확인할 데이터

release sha, image digest, container uptime, external status, rollback time을 본다.

다음 단계

배포 보고서에 명령, 결과, 실패 원인, 복구 방법을 한 줄씩 적어 본다.

기억할 문장

- 운영 Runbook과 릴리스는 '개발 결과를 안전하게 서버에 올리기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

25. 운영 Runbook과 릴리스 - 코드 작성 원칙

개발 결과를 안전하게 서버에 올리기



OCP 핵심

새 구현체를 추가해 확장하고, 이미 검증된 호출부는 되도록 건드리지 않는다.

그림 해석: 개발 결과를 안전하게 서버에 올리기 - release sha, image digest, container uptime, external status, rollback time을 본다.

코드 원칙

릴리스는 커밋, 이미지, env, compose, smoke 결과가 서로 맞아야 한다.

설계 연결

SRP는 책임을 나누고, OCP는 새 구현체를 붙이기 쉽게 만들며, DIP는 구체 구현 대신 interface를 바라보게 만든다.

작은 예시

문자열 endpoint를 컨트롤러에 직접 쓰지 말고 RouteContract 또는 ApiRoutes 같은 상수 객체에 모은다.

기억할 문장

- 운영 Runbook과 릴리스는 '개발 결과를 안전하게 서버에 올리기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

25. 운영 Runbook과 릴리스 - 테스트와 검증

개발 결과를 안전하게 서버에 올리기

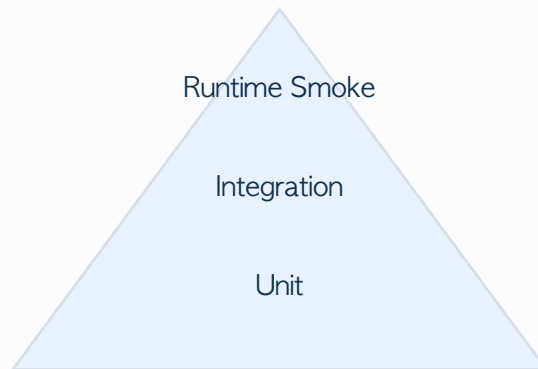


그림 해석: 개발 결과를 안전하게 서버에 올리기 - release sha, image digest, container uptime, external status, rollback time을 본다.

테스트 원칙

배포 전 test suite, compose config, external curl, WebRTC smoke를 확인한다.

TDD 관점

먼저 실패하는 계약 테스트를 만들고, 그 계약을 만족하는 가장 작은 구현을 작성한 뒤, 실제 runtime smoke로 연결을 확인한다.

통과 기준

release sha, image digest, container uptime, external status, rollback time을 본다.가 기대 범위 안에 있고, 실패 시 원인이 로그와 테스트 이름으로 드러나야 한다.

기억할 문장

- 운영 Runbook과 릴리스는 '개발 결과를 안전하게 서버에 올리기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

25. 운영 Runbook과 릴리스 - 운영 체크

개발 결과를 안전하게 서버에 올리기

이 페이지의 핵심

코드가 좋아도 배포와 복구 절차가 없으면 운영 시스템으로 보기 어렵다.

운영 원칙

서버에는 이전 릴리스를 남겨 rollback 가능성을 유지한다.

장애 대응

status code, container health, application log, network path,
secret/config 누락을 순서대로 확인한다.

기록할 것

명령어, 실행 시각, 결과, 실패 원인, 수정 파일, 남은 리스크를 보고서에 남긴다.

기억할 문장

- 운영 Runbook과 릴리스는 '개발 결과를 안전하게 서버에 올리기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

25. 운영 Runbook과 릴리스 - 자주 나는 실수

개발 결과를 안전하게 서버에 올리기

이 페이지의 핵심

코드가 좋아도 배포와 복구 절차가 없으면 운영 시스템으로 보기 어렵다.

자주 나는 실수

current symlink만 바꾸고 실제 컨테이너가 옛 이미지면 배포됐다고 착각할 수 있다.

왜 위험한가

초반에는 빠르게 보이지만, 운영 중 장애 원인을 숨기거나 변경 범위를 키워 결국 더 오래 걸리게 만든다.

예방 습관

계약값 중앙화, DTO 분리, 테스트 추가, 로그/지표 확인을 PR 기준으로 둔다.

기억할 문장

- 운영 Runbook과 릴리스는 '개발 결과를 안전하게 서버에 올리기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

25. 운영 Runbook과 릴리스 - 초보자 실습

개발 결과를 안전하게 서버에 올리기

이 페이지의 핵심

코드가 좋아도 배포와 복구 절차가 없으면 운영 시스템으로 보기 어렵다.

따라 해보기

배포 보고서에 명령, 결과, 실패 원인, 복구 방법을 한 줄씩 적어 본다.

관찰하기

작업 전후로 release sha, image digest, container uptime, external status, rollback time을 본다.를 기록한다. 숫자를 모르면 개선인지 착각인지 구분할 수 없다.

질문 만들기

'운영 Runbook과 릴리스이 깨지면 사용자는 무엇을 보게 되는가?'를 한 문장으로 답해 본다.

기억할 문장

- 운영 Runbook과 릴리스은 '개발 결과를 안전하게 서버에 올리기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

25. 운영 Runbook과 릴리스 - 복습 체크리스트

개발 결과를 안전하게 서버에 올리기

이 페이지의 핵심

코드가 좋아도 배포와 복구 절차가 없으면 운영 시스템으로 보기 어렵다.

복습 질문

운영 Runbook과 릴리스는 control plane, media plane, data plane, ops plane
중 어디에 가까운가?

체크리스트

경로가 상수화되어 있는가? DTO가 분리되어 있는가? 실패 테스트가 있는가? 운영 지표가 있는가? secret이 노출되지 않았는가?

다음 학습

관련 코드 파일 하나와 테스트 파일 하나를 함께 열어 구현과 검증이 어떻게 연결되는지 읽는다.

기억할 문장

- 운영 Runbook과 릴리스는 '개발 결과를 안전하게 서버에 올리기'을 해결하기 위한 기술이다.
- 설계 원칙은 코드 예쁘게 보이기가 아니라 변경 비용과 장애 범위를 줄이는 장치다.
- 테스트는 기능 완료의 장식이 아니라 운영 가능한 상태를 증명하는 최소 조건이다.

부록 01. 용어 사전 1

실무에서 바로 다시 보는 요약

요약

WebRTC, ICE, SDP, RTP, Opus, H264 같은 미디어 용어를 한 줄 정의로 복습한다.

핵심 포인트

- 먼저 목적을 확인한다. 이 기술은 어떤 위험이나 비용을 줄이기 위해 들어왔는가?
- 다음으로 서버 위치를 본다. entry, control, media, data, ops 중 어디인가?
- 마지막으로 테스트를 본다. 이 기술이 깨졌을 때 어떤 자동 검증이 실패해야 하는가?

실무 메모

초보자는 명령어를 외우기보다 장애 흐름을 먼저 그리는 편이 좋다. 예를 들어 영상이 안 나오면 로그인 문제인지, 스트림 문제인지, WHIP/WHEP signaling 문제인지, ICE/TURN 문제인지, 실제 RTP packet 문제인지 순서대로 나누어 본다.

보고서에 남길 문장

- 무엇을 확인했는가: 명령, endpoint, 테스트 이름을 적는다.
- 왜 실패했는가: 원인을 계층별로 분리해서 적는다.
- 어떻게 수정했는가: 파일, 책임, 테스트를 함께 적는다.
- 남은 리스크는 무엇인가: 후속 이슈로 분리할 조건을 적는다.

부록 02. 용어 사전 2

실무에서 바로 다시 보는 요약

요약

JWT, CSRF, CORS, CSP, HSTS, HttpOnly cookie
같은 보안 용어를 복습한다.

핵심 포인트

- 먼저 목적을 확인한다. 이 기술은 어떤 위협이나 비용을 줄이기 위해 들어왔는가?
- 다음으로 서버 위치를 본다. entry, control, media, data, ops 중 어디인가?
- 마지막으로 테스트를 본다. 이 기술이 깨졌을 때 어떤 자동 검증이 실패해야 하는가?

실무 메모

초보자는 명령어를 외우기보다 장애 흐름을 먼저 그리는 편이 좋다. 예를 들어 영상이 안 나오면 로그인 문제인지, 스트림 문제인지, WHIP/WHEP signaling 문제인지, ICE/TURN 문제인지, 실제 RTP packet 문제인지 순서대로 나누어 본다.

보고서에 남길 문장

- 무엇을 확인했는가: 명령, endpoint, 테스트 이름을 적는다.
- 왜 실패했는가: 원인을 계층별로 분리해서 적는다.
- 어떻게 수정했는가: 파일, 책임, 테스트를 함께 적는다.
- 남은 리스크는 무엇인가: 후속 이슈로 분리할 조건을 적는다.

부록 03. 용어 사전 3

실무에서 바로 다시 보는 요약

요약

DTO, VO, Domain, Repository, Adapter, Strategy, Factory를 코드 관점에서 복습한다.

핵심 포인트

- 먼저 목적을 확인한다. 이 기술은 어떤 위험이나 비용을 줄이기 위해 들어왔는가?
- 다음으로 서버 위치를 본다. entry, control, media, data, ops 중 어디인가?
- 마지막으로 테스트를 본다. 이 기술이 깨졌을 때 어떤 자동 검증이 실패해야 하는가?

실무 메모

초보자는 명령어를 외우기보다 장애 흐름을 먼저 그리는 편이 좋다. 예를 들어 영상이 안 나오면 로그인 문제인지, 스트림 문제인지, WHIP/WHEP signaling 문제인지, ICE/TURN 문제인지, 실제 RTP packet 문제인지 순서대로 나누어 본다.

보고서에 남길 문장

- 무엇을 확인했는가: 명령, endpoint, 테스트 이름을 적는다.
- 왜 실패했는가: 원인을 계층별로 분리해서 적는다.
- 어떻게 수정했는가: 파일, 책임, 테스트를 함께 적는다.
- 남은 리스크는 무엇인가: 후속 이슈로 분리할 조건을 적는다.

부록 04. 운영 명령 모음

실무에서 바로 다시 보는 요약

요약

curl, docker compose, healthcheck, smoke script를 어떤 순서로 쓰는지 정리한다.

핵심 포인트

- 먼저 목적을 확인한다. 이 기술은 어떤 위험이나 비용을 줄이기 위해 들어왔는가?
- 다음으로 서버 위치를 본다. entry, control, media, data, ops 중 어디인가?
- 마지막으로 테스트를 본다. 이 기술이 깨졌을 때 어떤 자동 검증이 실패해야 하는가?

실무 메모

초보자는 명령어를 외우기보다 장애 흐름을 먼저 그리는 편이 좋다. 예를 들어 영상이 안 나오면 로그인 문제인지, 스트림 문제인지, WHIP/WHEP signaling 문제인지, ICE/TURN 문제인지, 실제 RTP packet 문제인지 순서대로 나누어 본다.

보고서에 남길 문장

- 무엇을 확인했는가: 명령, endpoint, 테스트 이름을 적는다.
- 왜 실패했는가: 원인을 계층별로 분리해서 적는다.
- 어떻게 수정했는가: 파일, 책임, 테스트를 함께 적는다.
- 남은 리스크는 무엇인가: 후속 이슈로 분리할 조건을 적는다.

부록 05. WebRTC 장애 분류

실무에서 바로 다시 보는 요약

요약

signaling 실패, ICE 실패, media packet loss, codec 문제를 구분한다.

핵심 포인트

- 먼저 목적을 확인한다. 이 기술은 어떤 위험이나 비용을 줄이기 위해 들어왔는가?
- 다음으로 서버 위치를 본다. entry, control, media, data, ops 중 어디인가?
- 마지막으로 테스트를 본다. 이 기술이 깨졌을 때 어떤 자동 검증이 실패해야 하는가?

실무 메모

초보자는 명령어를 외우기보다 장애 흐름을 먼저 그리는 편이 좋다. 예를 들어 영상이 안 나오면 로그인 문제인지, 스트림 문제인지, WHIP/WHEP signaling 문제인지, ICE/TURN 문제인지, 실제 RTP packet 문제인지 순서대로 나누어 본다.

보고서에 남길 문장

- 무엇을 확인했는가: 명령, endpoint, 테스트 이름을 적는다.
- 왜 실패했는가: 원인을 계층별로 분리해서 적는다.
- 어떻게 수정했는가: 파일, 책임, 테스트를 함께 적는다.
- 남은 리스크는 무엇인가: 후속 이슈로 분리할 조건을 적는다.

부록 06. 오디오 지연 분류

실무에서 바로 다시 보는 요약

요약

jitter, jitter buffer, packet loss, browser
audio processing을 구분한다.

핵심 포인트

- 먼저 목적을 확인한다. 이 기술은 어떤 위험이나 비용을 줄이기 위해 들어왔는가?
- 다음으로 서버 위치를 본다. entry, control, media, data, ops 중 어디인가?
- 마지막으로 테스트를 본다. 이 기술이 깨졌을 때 어떤 자동 검증이 실패해야 하는가?

실무 메모

초보자는 명령어를 외우기보다 장애 흐름을 먼저 그리는 편이 좋다. 예를 들어 영상이 안 나오면 로그인 문제인지, 스트림 문제인지, WHIP/WHEP signaling 문제인지, ICE/TURN 문제인지, 실제 RTP packet 문제인지 순서대로 나누어 본다.

보고서에 남길 문장

- 무엇을 확인했는가: 명령, endpoint, 테스트 이름을 적는다.
- 왜 실패했는가: 원인을 계층별로 분리해서 적는다.
- 어떻게 수정했는가: 파일, 책임, 테스트를 함께 적는다.
- 남은 리스크는 무엇인가: 후속 이슈로 분리할 조건을 적는다.

부록 07. DB 튜닝 요약

실무에서 바로 다시 보는 요약

요약

EXPLAIN, index, covering index, no-offset
paging, buffer pool을 복습한다.

핵심 포인트

- 먼저 목적을 확인한다. 이 기술은 어떤 위험이나 비용을 줄이기 위해 들어왔는가?
- 다음으로 서버 위치를 본다. entry, control, media, data, ops 중 어디인가?
- 마지막으로 테스트를 본다. 이 기술이 깨졌을 때 어떤 자동 검증이 실패해야 하는가?

실무 메모

초보자는 명령어를 외우기보다 장애 흐름을 먼저 그리는 편이 좋다. 예를 들어 영상이 안 나오면 로그인 문제인지, 스트림 문제인지, WHIP/WHEP signaling 문제인지, ICE/TURN 문제인지, 실제 RTP packet 문제인지 순서대로 나누어 본다.

보고서에 남길 문장

- 무엇을 확인했는가: 명령, endpoint, 테스트 이름을 적는다.
- 왜 실패했는가: 원인을 계층별로 분리해서 적는다.
- 어떻게 수정했는가: 파일, 책임, 테스트를 함께 적는다.
- 남은 리스크는 무엇인가: 후속 이슈로 분리할 조건을 적는다.

부록 08. Redis 튜닝 요약

실무에서 바로 다시 보는 요약

요약

TTL, eviction, SCAN, atomic consume, cache stampede를 복습한다.

핵심 포인트

- 먼저 목적을 확인한다. 이 기술은 어떤 위험이나 비용을 줄이기 위해 들어왔는가?
- 다음으로 서버 위치를 본다. entry, control, media, data, ops 중 어디인가?
- 마지막으로 테스트를 본다. 이 기술이 깨졌을 때 어떤 자동 검증이 실패해야 하는가?

실무 메모

초보자는 명령어를 외우기보다 장애 흐름을 먼저 그리는 편이 좋다. 예를 들어 영상이 안 나오면 로그인 문제인지, 스트림 문제인지, WHIP/WHEP signaling 문제인지, ICE/TURN 문제인지, 실제 RTP packet 문제인지 순서대로 나누어 본다.

보고서에 남길 문장

- 무엇을 확인했는가: 명령, endpoint, 테스트 이름을 적는다.
- 왜 실패했는가: 원인을 계층별로 분리해서 적는다.
- 어떻게 수정했는가: 파일, 책임, 테스트를 함께 적는다.
- 남은 리스크는 무엇인가: 후속 이슈로 분리할 조건을 적는다.

부록 09. 프론트 렌더링 점검

실무에서 바로 다시 보는 요약

요약

useEffect, useReducer, memo, lazy loading, player sizing을 점검한다.

핵심 포인트

- 먼저 목적을 확인한다. 이 기술은 어떤 위험이나 비용을 줄이기 위해 들어왔는가?
- 다음으로 서버 위치를 본다. entry, control, media, data, ops 중 어디인가?
- 마지막으로 테스트를 본다. 이 기술이 깨졌을 때 어떤 자동 검증이 실패해야 하는가?

실무 메모

초보자는 명령어를 외우기보다 장애 흐름을 먼저 그리는 편이 좋다. 예를 들어 영상이 안 나오면 로그인 문제인지, 스트림 문제인지, WHIP/WHEP signaling 문제인지, ICE/TURN 문제인지, 실제 RTP packet 문제인지 순서대로 나누어 본다.

보고서에 남길 문장

- 무엇을 확인했는가: 명령, endpoint, 테스트 이름을 적는다.
- 왜 실패했는가: 원인을 계층별로 분리해서 적는다.
- 어떻게 수정했는가: 파일, 책임, 테스트를 함께 적는다.
- 남은 리스크는 무엇인가: 후속 이슈로 분리할 조건을 적는다.

부록 10. 보안 체크리스트

실무에서 바로 다시 보는 요약

요약

secret, token, cookie, HTTPS, origin, CSP, logging을 점검한다.

핵심 포인트

- 먼저 목적을 확인한다. 이 기술은 어떤 위협이나 비용을 줄이기 위해 들어왔는가?
- 다음으로 서버 위치를 본다. entry, control, media, data, ops 중 어디인가?
- 마지막으로 테스트를 본다. 이 기술이 깨졌을 때 어떤 자동 검증이 실패해야 하는가?

실무 메모

초보자는 명령어를 외우기보다 장애 흐름을 먼저 그리는 편이 좋다. 예를 들어 영상이 안 나오면 로그인 문제인지, 스트림 문제인지, WHIP/WHEP signaling 문제인지, ICE/TURN 문제인지, 실제 RTP packet 문제인지 순서대로 나누어 본다.

보고서에 남길 문장

- 무엇을 확인했는가: 명령, endpoint, 테스트 이름을 적는다.
- 왜 실패했는가: 원인을 계층별로 분리해서 적는다.
- 어떻게 수정했는가: 파일, 책임, 테스트를 함께 적는다.
- 남은 리스크는 무엇인가: 후속 이슈로 분리할 조건을 적는다.

부록 11. 폐쇄망 체크리스트

실무에서 바로 다시 보는 요약

요약

외부 API, npm, Docker image, 지도, 시간 서버, STUN/TURN 의존을 점검한다.

핵심 포인트

- 먼저 목적을 확인한다. 이 기술은 어떤 위험이나 비용을 줄이기 위해 들어왔는가?
- 다음으로 서버 위치를 본다. entry, control, media, data, ops 중 어디인가?
- 마지막으로 테스트를 본다. 이 기술이 깨졌을 때 어떤 자동 검증이 실패해야 하는가?

실무 메모

초보자는 명령어를 외우기보다 장애 흐름을 먼저 그리는 편이 좋다. 예를 들어 영상이 안 나오면 로그인 문제인지, 스트림 문제인지, WHIP/WHEP signaling 문제인지, ICE/TURN 문제인지, 실제 RTP packet 문제인지 순서대로 나누어 본다.

보고서에 남길 문장

- 무엇을 확인했는가: 명령, endpoint, 테스트 이름을 적는다.
- 왜 실패했는가: 원인을 계층별로 분리해서 적는다.
- 어떻게 수정했는가: 파일, 책임, 테스트를 함께 적는다.
- 남은 리스크는 무엇인가: 후속 이슈로 분리할 조건을 적는다.

부록 12. 테스트 체크리스트

실무에서 바로 다시 보는 요약

요약

unit, integration, smoke, coverage, runtime validation을 PR 전에 확인한다.

핵심 포인트

- 먼저 목적을 확인한다. 이 기술은 어떤 위험이나 비용을 줄이기 위해 들어왔는가?
- 다음으로 서버 위치를 본다. entry, control, media, data, ops 중 어디인가?
- 마지막으로 테스트를 본다. 이 기술이 깨졌을 때 어떤 자동 검증이 실패해야 하는가?

실무 메모

초보자는 명령어를 외우기보다 장애 흐름을 먼저 그리는 편이 좋다. 예를 들어 영상이 안 나오면 로그인 문제인지, 스트림 문제인지, WHIP/WHEP signaling 문제인지, ICE/TURN 문제인지, 실제 RTP packet 문제인지 순서대로 나누어 본다.

보고서에 남길 문장

- 무엇을 확인했는가: 명령, endpoint, 테스트 이름을 적는다.
- 왜 실패했는가: 원인을 계층별로 분리해서 적는다.
- 어떻게 수정했는가: 파일, 책임, 테스트를 함께 적는다.
- 남은 리스크는 무엇인가: 후속 이슈로 분리할 조건을 적는다.

부록 13. PR 작성 템플릿

실무에서 바로 다시 보는 요약

요약

문제 원인, 수정 내용, 테스트, 남은 리스크, 운영 영향 범위를 적는다.

핵심 포인트

- 먼저 목적을 확인한다. 이 기술은 어떤 위험이나 비용을 줄이기 위해 들어왔는가?
- 다음으로 서버 위치를 본다. entry, control, media, data, ops 중 어디인가?
- 마지막으로 테스트를 본다. 이 기술이 깨졌을 때 어떤 자동 검증이 실패해야 하는가?

실무 메모

초보자는 명령어를 외우기보다 장애 흐름을 먼저 그리는 편이 좋다. 예를 들어 영상이 안 나오면 로그인 문제인지, 스트림 문제인지, WHIP/WHEP signaling 문제인지, ICE/TURN 문제인지, 실제 RTP packet 문제인지 순서대로 나누어 본다.

보고서에 남길 문장

- 무엇을 확인했는가: 명령, endpoint, 테스트 이름을 적는다.
- 왜 실패했는가: 원인을 계층별로 분리해서 적는다.
- 어떻게 수정했는가: 파일, 책임, 테스트를 함께 적는다.
- 남은 리스크는 무엇인가: 후속 이슈로 분리할 조건을 적는다.

부록 14. Issue 작성 템플릿

실무에서 바로 다시 보는 요약

요약

배경, 수락 기준, 보안 주의, 테스트 계획, 후속 분리 기준을 적는다.

핵심 포인트

- 먼저 목적을 확인한다. 이 기술은 어떤 위험이나 비용을 줄이기 위해 들어왔는가?
- 다음으로 서버 위치를 본다. entry, control, media, data, ops 중 어디인가?
- 마지막으로 테스트를 본다. 이 기술이 깨졌을 때 어떤 자동 검증이 실패해야 하는가?

실무 메모

초보자는 명령어를 외우기보다 장애 흐름을 먼저 그리는 편이 좋다. 예를 들어 영상이 안 나오면 로그인 문제인지, 스트림 문제인지, WHIP/WHEP signaling 문제인지, ICE/TURN 문제인지, 실제 RTP packet 문제인지 순서대로 나누어 본다.

보고서에 남길 문장

- 무엇을 확인했는가: 명령, endpoint, 테스트 이름을 적는다.
- 왜 실패했는가: 원인을 계층별로 분리해서 적는다.
- 어떻게 수정했는가: 파일, 책임, 테스트를 함께 적는다.
- 남은 리스크는 무엇인가: 후속 이슈로 분리할 조건을 적는다.

부록 15. 서버 배포 템플릿

실무에서 바로 다시 보는 요약

요약

릴리스 디렉터리, env 복사, compose config, build, up, external curl 순서를 적는다.

핵심 포인트

- 먼저 목적을 확인한다. 이 기술은 어떤 위험이나 비용을 줄이기 위해 들어왔는가?
- 다음으로 서버 위치를 본다. entry, control, media, data, ops 중 어디인가?
- 마지막으로 테스트를 본다. 이 기술이 깨졌을 때 어떤 자동 검증이 실패해야 하는가?

실무 메모

초보자는 명령어를 외우기보다 장애 흐름을 먼저 그리는 편이 좋다. 예를 들어 영상이 안 나오면 로그인 문제인지, 스트림 문제인지, WHIP/WHEP signaling 문제인지, ICE/TURN 문제인지, 실제 RTP packet 문제인지 순서대로 나누어 본다.

보고서에 남길 문장

- 무엇을 확인했는가: 명령, endpoint, 테스트 이름을 적는다.
- 왜 실패했는가: 원인을 계층별로 분리해서 적는다.
- 어떻게 수정했는가: 파일, 책임, 테스트를 함께 적는다.
- 남은 리스크는 무엇인가: 후속 이슈로 분리할 조건을 적는다.

부록 16. 장애 보고 템플릿

실무에서 바로 다시 보는 요약

요약

발생 시각, 영향 범위, 로그, 원인, 조치, 재발 방지를 적는다.

핵심 포인트

- 먼저 목적을 확인한다. 이 기술은 어떤 위험이나 비용을 줄이기 위해 들어왔는가?
- 다음으로 서버 위치를 본다. entry, control, media, data, ops 중 어디인가?
- 마지막으로 테스트를 본다. 이 기술이 깨졌을 때 어떤 자동 검증이 실패해야 하는가?

실무 메모

초보자는 명령어를 외우기보다 장애 흐름을 먼저 그리는 편이 좋다. 예를 들어 영상이 안 나오면 로그인 문제인지, 스트림 문제인지, WHIP/WHEP signaling 문제인지, ICE/TURN 문제인지, 실제 RTP packet 문제인지 순서대로 나누어 본다.

보고서에 남길 문장

- 무엇을 확인했는가: 명령, endpoint, 테스트 이름을 적는다.
- 왜 실패했는가: 원인을 계층별로 분리해서 적는다.
- 어떻게 수정했는가: 파일, 책임, 테스트를 함께 적는다.
- 남은 리스크는 무엇인가: 후속 이슈로 분리할 조건을 적는다.

부록 17. 초보자 4주 학습 계획

실무에서 바로 다시 보는 요약

요약

HTTP/Docker, React/API, WebRTC, 보안/테스트 순서로 공부한다.

핵심 포인트

- 먼저 목적을 확인한다. 이 기술은 어떤 위험이나 비용을 줄이기 위해 들어왔는가?
- 다음으로 서버 위치를 본다. entry, control, media, data, ops 중 어디인가?
- 마지막으로 테스트를 본다. 이 기술이 깨졌을 때 어떤 자동 검증이 실패해야 하는가?

실무 메모

초보자는 명령어를 외우기보다 장애 흐름을 먼저 그리는 편이 좋다. 예를 들어 영상이 안 나오면 로그인 문제인지, 스트림 문제인지, WHIP/WHEP signaling 문제인지, ICE/TURN 문제인지, 실제 RTP packet 문제인지 순서대로 나누어 본다.

보고서에 남길 문장

- 무엇을 확인했는가: 명령, endpoint, 테스트 이름을 적는다.
- 왜 실패했는가: 원인을 계층별로 분리해서 적는다.
- 어떻게 수정했는가: 파일, 책임, 테스트를 함께 적는다.
- 남은 리스크는 무엇인가: 후속 이슈로 분리할 조건을 적는다.

부록 18. 코드 리뷰 관점

실무에서 바로 다시 보는 요약

요약

계약값, DTO, 불변성, 동시성, 테스트, secret 노출을 본다.

핵심 포인트

- 먼저 목적을 확인한다. 이 기술은 어떤 위험이나 비용을 줄이기 위해 들어왔는가?
- 다음으로 서버 위치를 본다. entry, control, media, data, ops 중 어디인가?
- 마지막으로 테스트를 본다. 이 기술이 깨졌을 때 어떤 자동 검증이 실패해야 하는가?

실무 메모

초보자는 명령어를 외우기보다 장애 흐름을 먼저 그리는 편이 좋다. 예를 들어 영상이 안 나오면 로그인 문제인지, 스트림 문제인지, WHIP/WHEP signaling 문제인지, ICE/TURN 문제인지, 실제 RTP packet 문제인지 순서대로 나누어 본다.

보고서에 남길 문장

- 무엇을 확인했는가: 명령, endpoint, 테스트 이름을 적는다.
- 왜 실패했는가: 원인을 계층별로 분리해서 적는다.
- 어떻게 수정했는가: 파일, 책임, 테스트를 함께 적는다.
- 남은 리스크는 무엇인가: 후속 이슈로 분리할 조건을 적는다.

부록 19. 성능 리뷰 관점

실무에서 바로 다시 보는 요약

요약

p95 latency, first frame, DB rows examined,
Redis hit ratio, render count를 본다.

핵심 포인트

- 먼저 목적을 확인한다. 이 기술은 어떤 위험이나 비용을 줄이기 위해 들어왔는가?
- 다음으로 서버 위치를 본다. entry, control, media, data, ops 중 어디인가?
- 마지막으로 테스트를 본다. 이 기술이 깨졌을 때 어떤 자동 검증이 실패해야 하는가?

실무 메모

초보자는 명령어를 외우기보다 장애 흐름을 먼저 그리는 편이 좋다. 예를 들어 영상이 안 나오면 로그인 문제인지, 스트림 문제인지, WHIP/WHEP signaling 문제인지, ICE/TURN 문제인지, 실제 RTP packet 문제인지 순서대로 나누어 본다.

보고서에 남길 문장

- 무엇을 확인했는가: 명령, endpoint, 테스트 이름을 적는다.
- 왜 실패했는가: 원인을 계층별로 분리해서 적는다.
- 어떻게 수정했는가: 파일, 책임, 테스트를 함께 적는다.
- 남은 리스크는 무엇인가: 후속 이슈로 분리할 조건을 적는다.

부록 20. 마지막 한 장

실무에서 바로 다시 보는 요약

요약

시스템 전체를 control plane, media plane, data plane, ops plane으로 다시 요약한다.

핵심 포인트

- 먼저 목적을 확인한다. 이 기술은 어떤 위험이나 비용을 줄이기 위해 들어왔는가?
- 다음으로 서버 위치를 본다. entry, control, media, data, ops 중 어디인가?
- 마지막으로 테스트를 본다. 이 기술이 깨졌을 때 어떤 자동 검증이 실패해야 하는가?

실무 메모

초보자는 명령어를 외우기보다 장애 흐름을 먼저 그리는 편이 좋다. 예를 들어 영상이 안 나오면 로그인 문제인지, 스트림 문제인지, WHIP/WHEP signaling 문제인지, ICE/TURN 문제인지, 실제 RTP packet 문제인지 순서대로 나누어 본다.

보고서에 남길 문장

- 무엇을 확인했는가: 명령, endpoint, 테스트 이름을 적는다.
- 왜 실패했는가: 원인을 계층별로 분리해서 적는다.
- 어떻게 수정했는가: 파일, 책임, 테스트를 함께 적는다.
- 남은 리스크는 무엇인가: 후속 이슈로 분리할 조건을 적는다.